

Imports

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from mltools import *
from sklearn.model_selection import train_test_split
from functools import partial
import seaborn as sns
```

I. Perceptron et classe Linéaire (batch)

1) Fonctions d'apprentissage

```
In [ ]: def perceptron_loss(w, x, y):
        """
        Compute the perceptron loss for a set of predictions.

        Params:
            w (np.ndarray): weight vector
            x (np.ndarray): input features
            y (np.ndarray): labels (+1 or -1)

        Returns:
            float: average loss over all samples
        """
        margins = -y * (x @ w)
        loss = np.maximum(0, margins).mean()
        return loss
```

```
In [ ]: def perceptron_grad(w, x, y):
        """
        Compute the gradient of the perceptron loss.

        Params:
            w (np.ndarray): weight vector
            x (np.ndarray): input features
            y (np.ndarray): labels (+1 or -1)

        Returns:
            np.ndarray: gradient of the loss w.r.t. weights
        """
        margins = -y * (x @ w)
        mask = margins > 0
        grad = -(x[mask].T @ y[mask]) / x.shape[0] #seulement exemple mal classé
        return grad
```

```
In [ ]: class Lineaire:
        def __init__(self, loss, loss_g, max_iter=100, eps=0.01):
            """
            Linear classifier trained with custom loss and gradient.
```

```
Params:
    loss (callable): loss function (e.g. perceptron_loss)
    loss_g (callable): gradient of the loss function
    max_iter (int): number of training iterations
    eps (float): learning rate
"""

self.loss = loss #perceptron loss
self.loss_g = loss_g # perceptron grad
self.max_iter = max_iter
self.eps = eps
self.w = None
self.loss_history = []

def predict(self, datax):
    """
    Predict class labels for given input data.

    Params:
        datax (np.ndarray): input features

    Returns:
        np.ndarray: predicted labels (+1 or -1)
    """

    return np.sign(datax @ self.w)

def score(self, datax, datay):
    """
    Compute accuracy score of the model.

    Params:
        datax (np.ndarray): input features
        datay (np.ndarray): true labels

    Returns:
        float: mean accuracy
    """

    y_pred = self.predict(datax)
    return (y_pred == datay).mean()

def fit(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model using gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Side Effects:
        Updates model weights, loss history and error metrics
    """
```

```

n, d = datax.shape
self.w = np.ones(d)
self.loss_history = []

self.train_errors = []
self.test_errors = []

for i in range(self.max_iter):
    loss_val = self.loss(self.w, datax, datay) #compute loss
    self.loss_history.append(loss_val) #stock it
    grad = self.loss_g(self.w, datax, datay) #compute grad w
    new_w = self.w - self.eps * grad
    self.w = new_w #update w
    if testx is not None and testy is not None:
        train_error = 1 - self.score(datax, datay) # calcul de l'err
        test_error = 1 - self.score(testx, testy)
        self.train_errors.append(train_error) #stock it
        self.test_errors.append(test_error)

```

```

In [ ]: def display_error(train_errors, test_errors) :
        """
        Display the learning curve showing training and test errors.

        Params:
            train_errors (list): list of training errors
            test_errors (list): list of test errors
        """

        plt.figure(figsize=(10,6))
        plt.plot(train_errors, label='Erreur entraînement')
        plt.plot(test_errors, label='Erreur test')
        plt.xlabel("Itérations")
        plt.ylabel("Erreur")
        plt.legend()
        plt.title("Courbe d'apprentissage (erreur par itérations)")
        plt.show()

```

2) Chargement des données USPS

```

In [ ]: def load_usps(fn):
        """
        Load the USPS dataset from a file.

        Params:
            fn (str): file path to the dataset

        Returns:
            tuple: input features (np.ndarray) and labels (np.ndarray)
        """

        with open(fn, "r") as f:
            f.readline()
            data = [[float(x) for x in l.split()] for l in f if len(l.split())>2]
            tmp=np.array(data)

```

```

    return tmp[:,1:],tmp[:,0].astype(int)

def get_usps(l,datax,datay):
    """
    Get the subset of data corresponding to specific labels.

    Params:
        l (int or list of ints): label(s) to retrieve
        datax (np.ndarray): input features
        datay (np.ndarray): labels

    Returns:
        tuple: subset of input features and corresponding labels
    """

    if type(l)!=list:
        resx = datax[datay==l,:]
        resy = datay[datay==l]
        return resx,resy
    tmp = list(zip(*[get_usps(i,datax,datay) for i in l]))
    tmpx,tmpy = np.vstack(tmp[0]),np.hstack(tmp[1])
    return tmpx,tmpy

def show_usps(data):
    """
    Display an image from the USPS dataset.

    Params:
        data (np.ndarray): flattened image data (size 256)
    """

    plt.imshow(data.reshape((16,16)),interpolation="nearest",cmap="gray")

```

```

In [ ]: def add_noise(data, eps=0.1):
    """
    Add Gaussian noise to the data.

    Params:
        data (np.ndarray): input data
        eps (float): standard deviation of the noise

    Returns:
        np.ndarray: noisy data, clipped between 0 and 1
    """

    noise = np.random.normal(0, eps, data.shape)
    noisy_data = data + noise
    noisy_data = np.clip(noisy_data, 0, 1)
    return noisy_data

```

```

In [8]: # Retrieve data
uspsdatatrain = "../data/USPS_train.txt"
uspsdatatest = "../data/USPS_test.txt"
alltrainx,alltrainy = load_usps(uspsdatatrain)
alltestx,alltesty = load_usps(uspsdatatest)

```

```

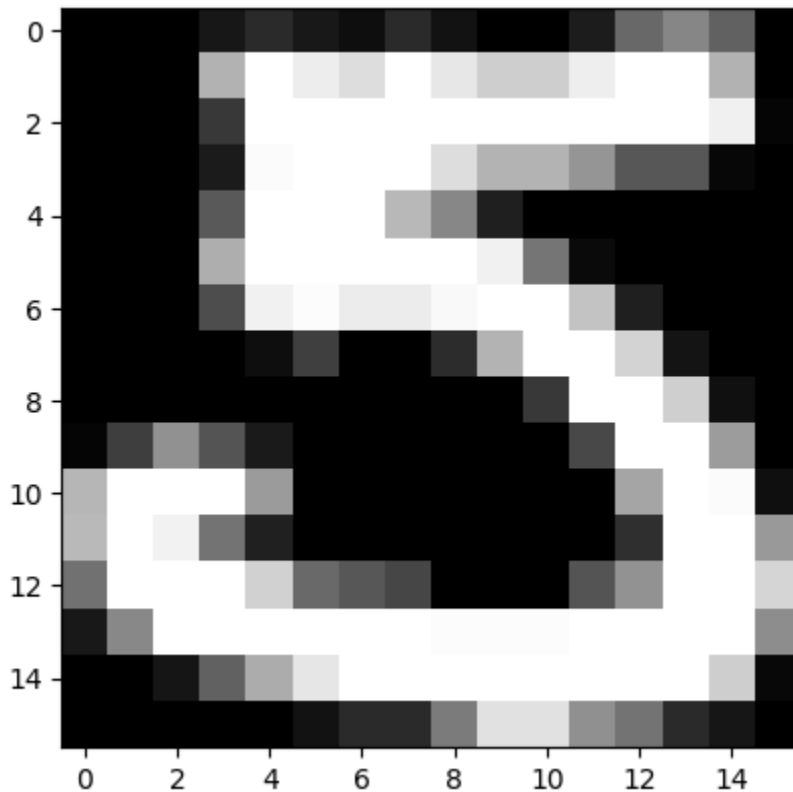
#filterer le dataset
neg = 5
pos = 6
datax, datay = get_usps([neg, pos], alltrainx, alltrainy)
testx, testy = get_usps([neg, pos], alltestx, alltesty)

# Affichage de l'image
show_usps(datax[0])
print(f"Shape of x data : {datax.shape}, y data : {datay.shape}")
plt.show()
print(f"L'image affichée correspond au label : {datay[0]}")

#transformer les labels des données 5, 6 en -1, 1
datay = np.where(datay == pos, 1, -1)
testy = np.where(testy == pos, 1, -1)
print(f"Après modification, l'image affichée correspond au label : {datay[0]}")

```

Shape of x data : (1220, 256), y data : (1220,)



L'image affichée correspond au label : 5

Après modification, l'image affichée correspond au label : -1

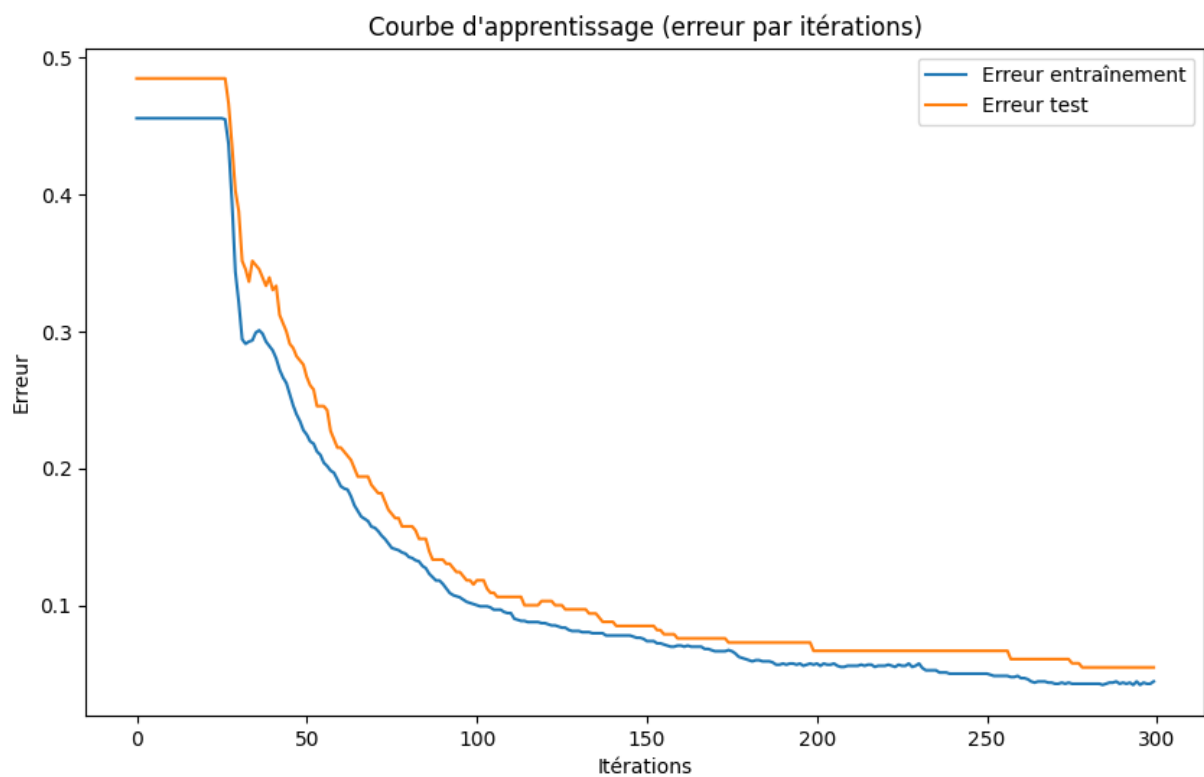
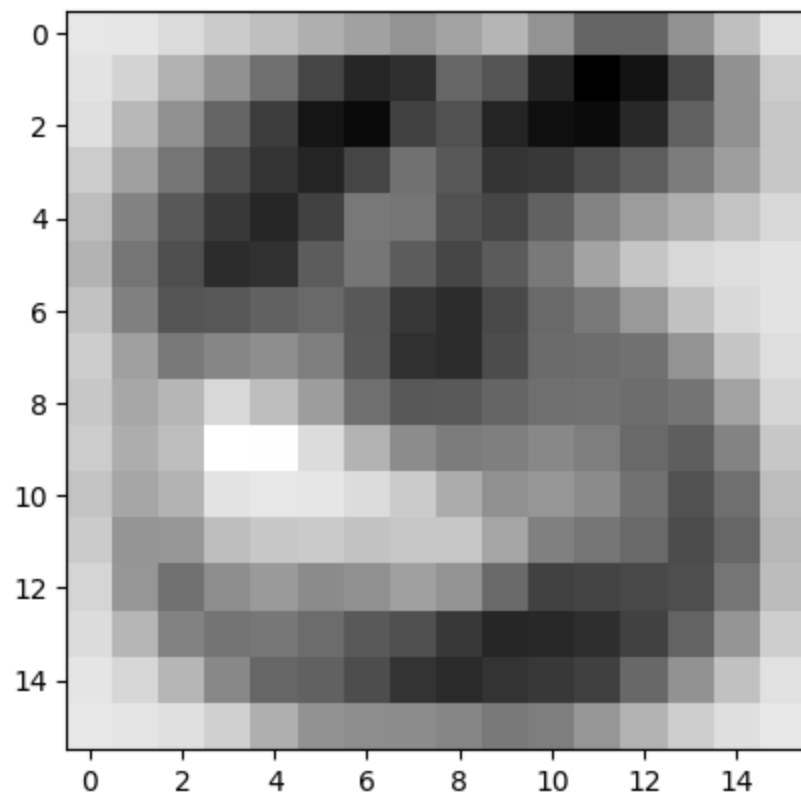
3) Tests et résultats

```

In [8]: lin = Lineaire(loss=perceptron_loss, loss_g=perceptron_grad, max_iter=300, e
lin.fit(datax, datay, testx, testy)
print("matrice de poids w après apprentissage : ")
show_usps(lin.w.reshape(16, 16))
display_error(lin.train_errors, lin.test_errors)

```

matrice de poids w après apprentissage :



II. Mini-batch et descente stochastique

1) Fonctions d'apprentissage

```

In [ ]: class Lineaire2:
    def __init__(self, loss, loss_g, n_epoch=100, eps=0.01):
        """
        Linear classifier trained with custom loss and gradient, supporting

        Params:
            loss (callable): loss function (e.g., perceptron_loss)
            loss_g (callable): gradient of the loss function
            n_epoch (int): number of training epochs
            eps (float): learning rate
        """

        self.loss = loss #perceptron loss
        self.loss_g = loss_g # perceptron grad
        self.n_epoch = n_epoch
        self.eps = eps
        self.w = None
        self.loss_history = []

    def predict(self, datax):
        """
        Predict class labels for given input data.

        Params:
            datax (np.ndarray): input features

        Returns:
            np.ndarray: predicted labels (+1 or -1)
        """

        return np.sign(datax @ self.w)

    def score(self, datax, datay):
        """
        Compute accuracy score of the model.

        Params:
            datax (np.ndarray): input features
            datay (np.ndarray): true labels

        Returns:
            float: mean accuracy
        """

        y_pred = self.predict(datax)
        return (y_pred == datay).mean()

    def fit_batch(self, datax, datay, testx=None, testy=None):
        """
        Train the linear model using batch gradient descent.

        Params:
            datax (np.ndarray): training input features
            datay (np.ndarray): training labels
            testx (np.ndarray, optional): test input features
            testy (np.ndarray, optional): test labels

```

```

Side Effects:
    Updates model weights, loss history and error metrics
    """

n, d = datax.shape
self.w = np.ones(d)
self.loss_history = []

self.train_errors = []
self.test_errors = []

for i in range(self.n_epoch):
    loss_val = self.loss(self.w, datax, datay) #compute loss
    self.loss_history.append(loss_val) #stock it
    grad = self.loss_g(self.w, datax, datay) #compute grad w
    new_w = self.w - self.eps * grad
    self.w = new_w #update w
    if testx is not None and testy is not None:
        train_error = 1 - self.score(datax, datay) # calcul de l'err
        test_error = 1 - self.score(testx, testy)
        self.train_errors.append(train_error) #stock it
        self.test_errors.append(test_error)

def fit_stochastic(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model using stochastic gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Side Effects:
        Updates model weights, loss history and error metrics
        """

n, d = datax.shape
self.w = np.ones(d)
self.loss_history = []

self.train_errors = []
self.test_errors = []

for epoch in range(self.n_epoch):
    indices = np.random.permutation(n)
    datax_shuffled = datax[indices]
    datay_shuffled = datay[indices]

    for i in range(0, n):
        xi = datax_shuffled[i:i+1]
        yi = datay_shuffled[i:i+1]
        grad = self.loss_g(self.w, xi, yi)
        self.w -= self.eps * grad

```



```

        train_error = self.loss(self.w, datax, datay)
        self.train_errors.append(train_error)

    if testx is not None and testy is not None:
        test_error = self.loss(self.w, testx, testy)
        self.test_errors.append(test_error)

def fit_minibatch(self, datax, datay, testx=None, testy=None, batch_size
    """
    Train the linear model using minibatch gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels
        batch_size (int): size of the minibatches

    Side Effects:
        Updates model weights, loss history and error metrics
    """

    n, d = datax.shape
    self.w = np.ones(d)
    self.loss_history = []

    self.train_errors = []
    self.test_errors = []

    for epoch in range(self.n_epoch):
        indices = np.random.permutation(n)
        datax_shuffled = datax[indices]
        datay_shuffled = datay[indices]

        for i in range(0, n, batch_size):
            x_batch = datax_shuffled[i:i + batch_size]
            y_batch = datay_shuffled[i:i + batch_size]
            grad = self.loss_g(self.w, x_batch, y_batch)
            self.w -= self.eps * grad

        # Ajout de l'erreur train
        train_error = 1 - self.score(datax, datay)
        self.train_errors.append(train_error)

        # Ajout de l'erreur test si dispo
        if testx is not None and testy is not None:
            test_error = 1 - self.score(testx, testy)
            self.test_errors.append(test_error)

```

2) Tests et résultats

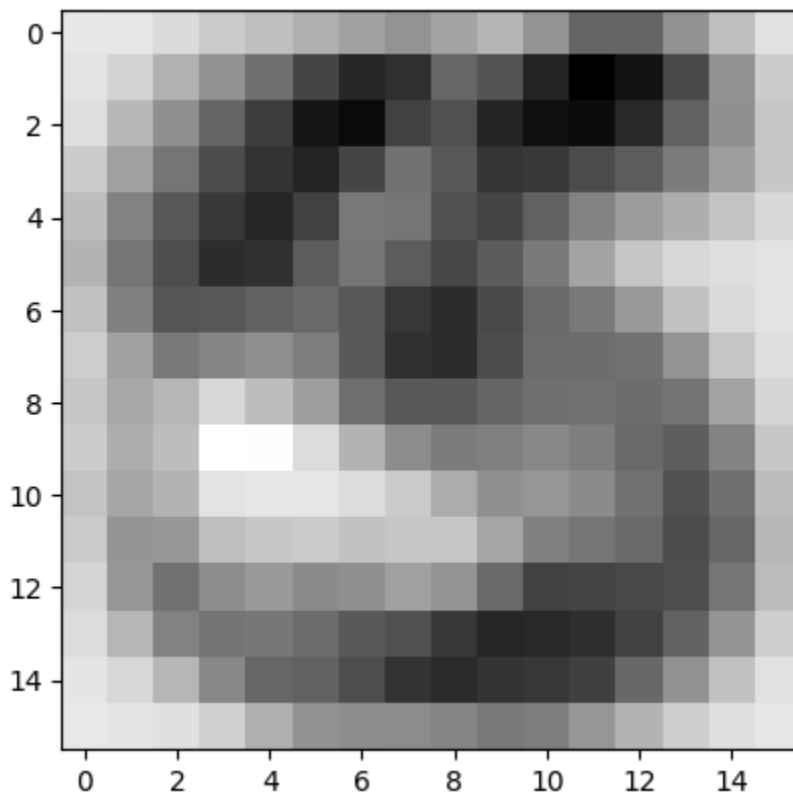
a) Apprentissage par batch

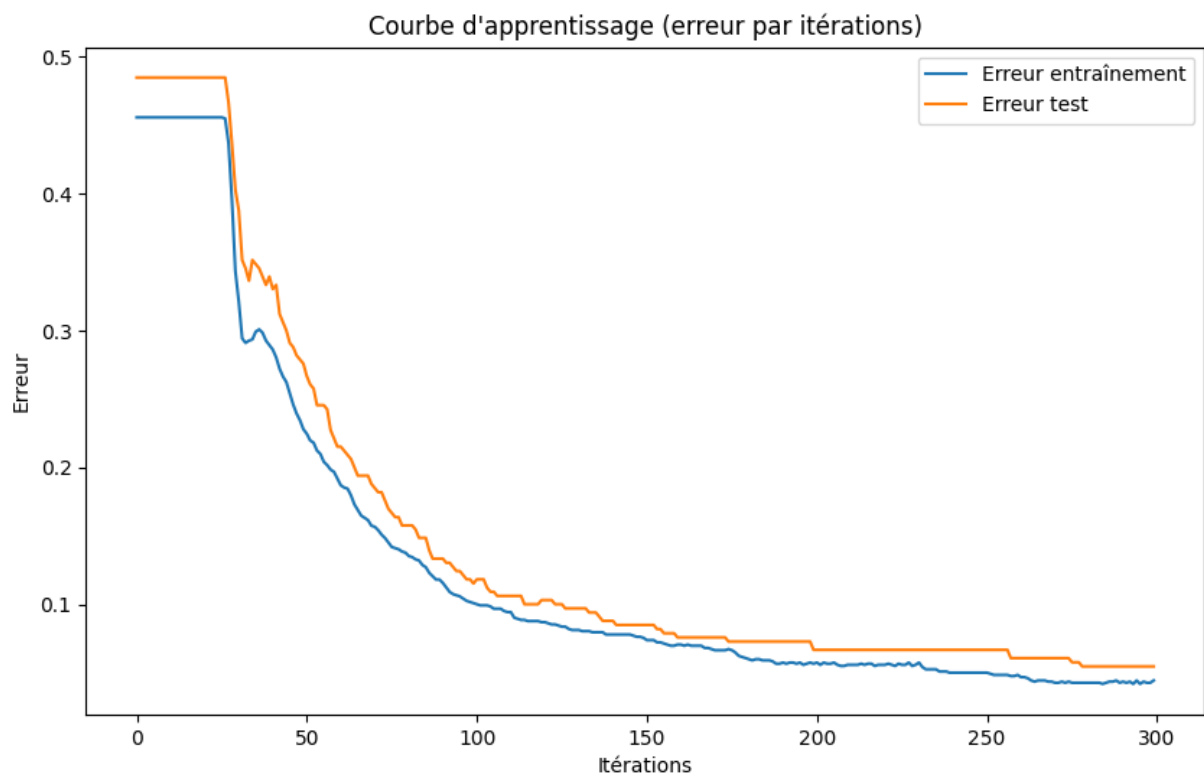
```
In [ ]: # données non bruitées
lin_batch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=
lin_batch.fit_batch(datax, datay, testx, testy)
print("matrice de poids w après apprentissage par batch: ")
show_usps(lin_batch.w.reshape(16, 16))
display_error(lin_batch.train_errors, lin_batch.test_errors)

# données bruitées 10%
lin_batch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad,
noisy_x_train = add_noise(datax, eps=0.10)
lin_batch_noised10.fit_batch(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage par batch pour données bruitée
show_usps(lin_batch_noised10.w.reshape(16, 16))
display_error(lin_batch_noised10.train_errors, lin_batch_noised10.test_error

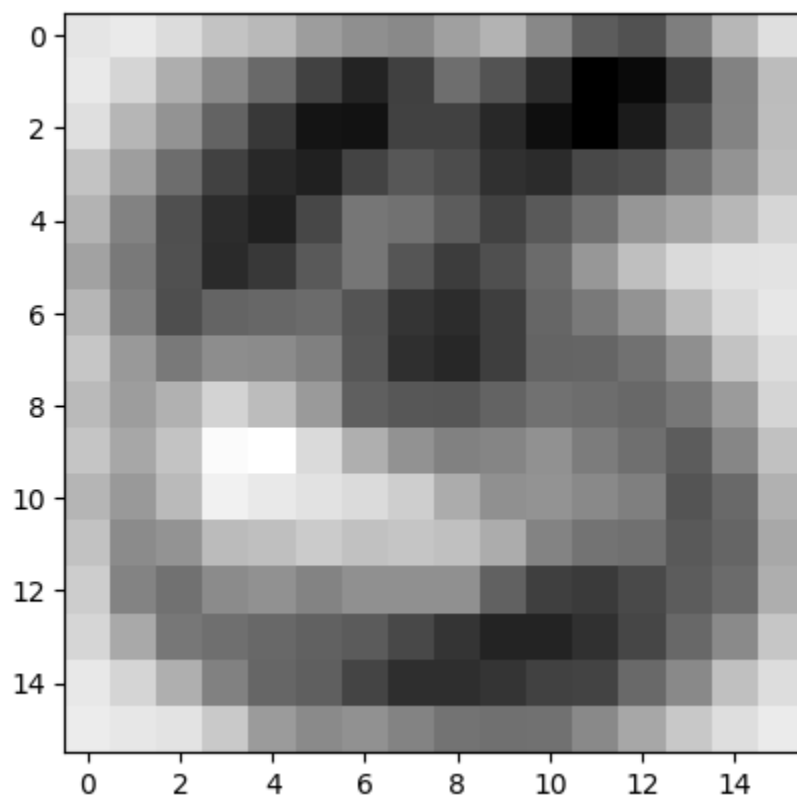
# données bruitées 50%
lin_batch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad,
noisy_x_train = add_noise(datax, eps=0.5)
lin_batch_noised50.fit_batch(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage par batch pour données bruitée
show_usps(lin_batch_noised50.w.reshape(16, 16))
display_error(lin_batch_noised50.train_errors, lin_batch_noised50.test_error
# données bruitées 70%
```

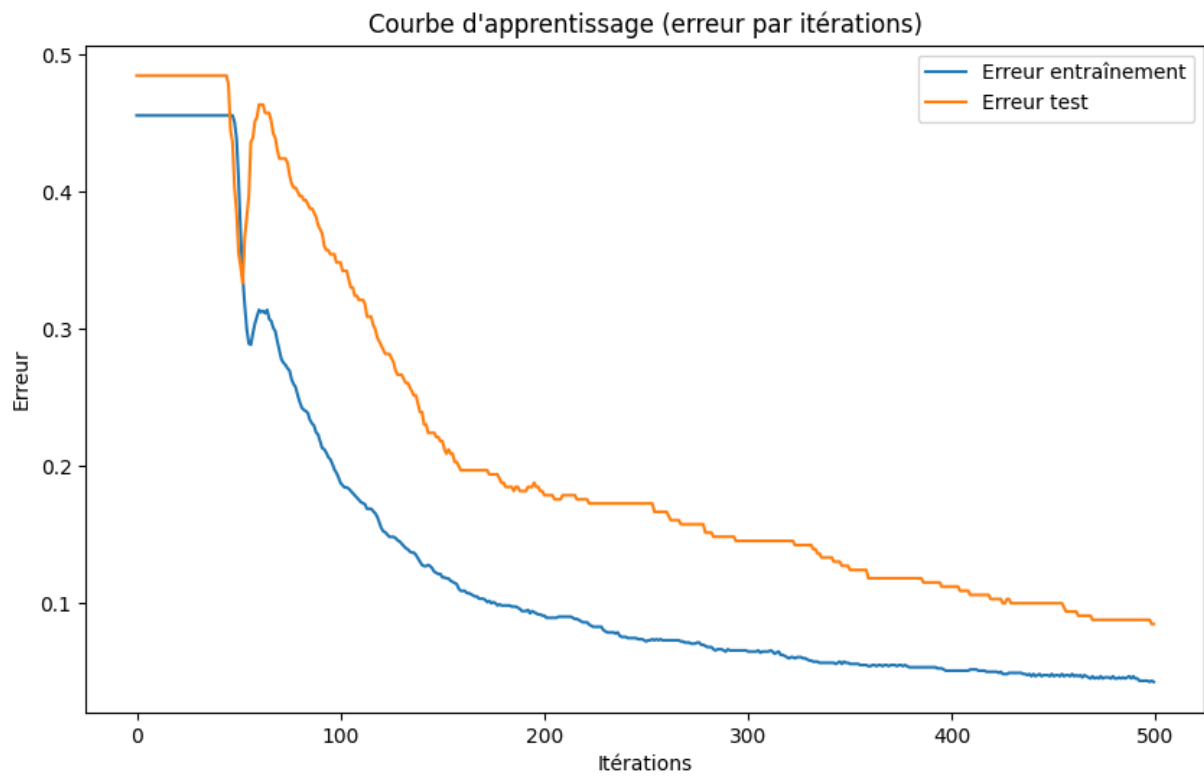
matrice de poids w après apprentissage par batch:



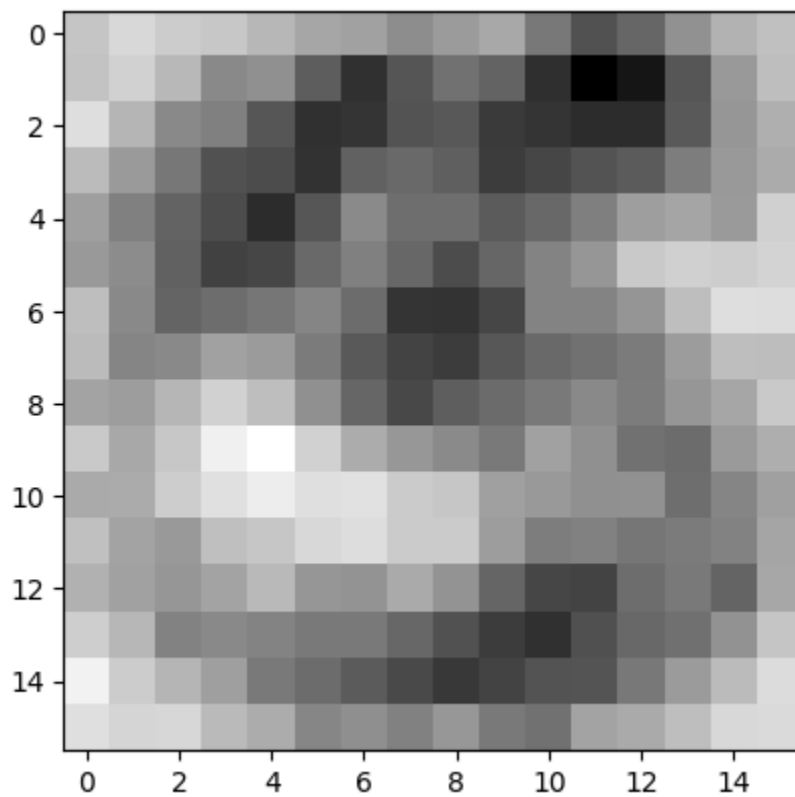


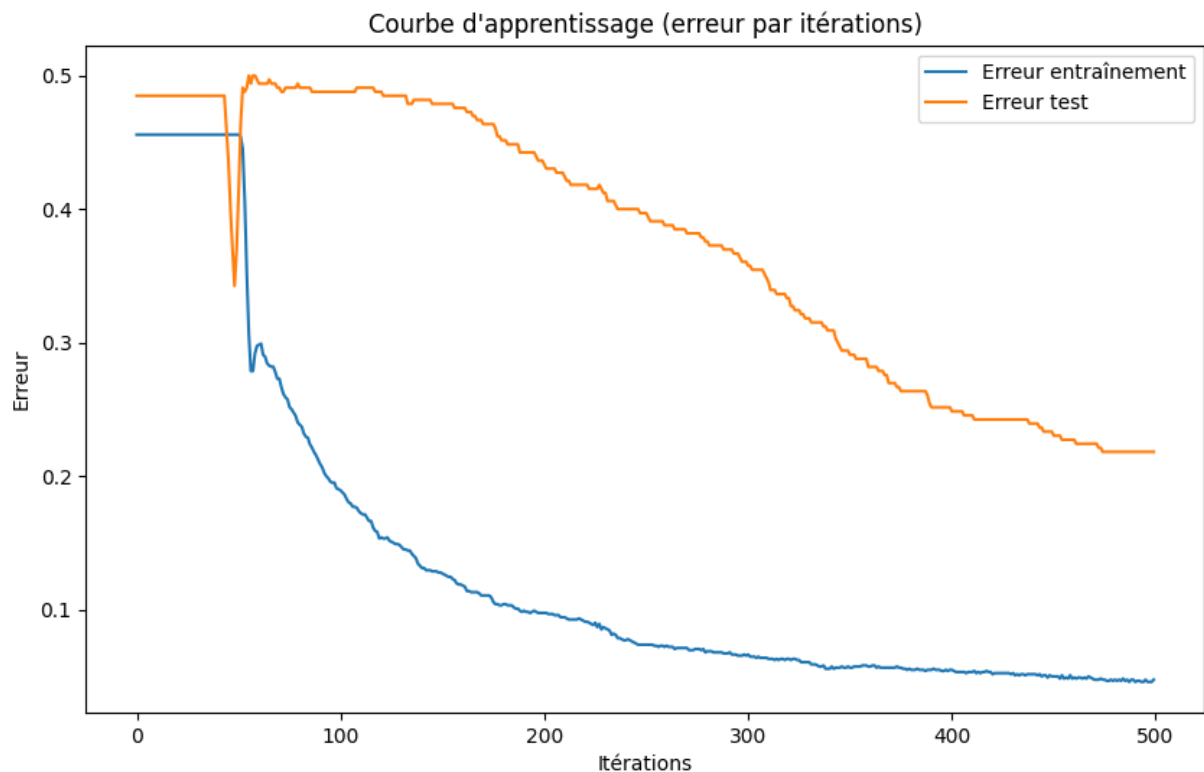
matrice de poids w après apprentissage par batch pour données bruitées (10%):





matrice de poids w après apprentissage par batch pour données bruitées (50%):





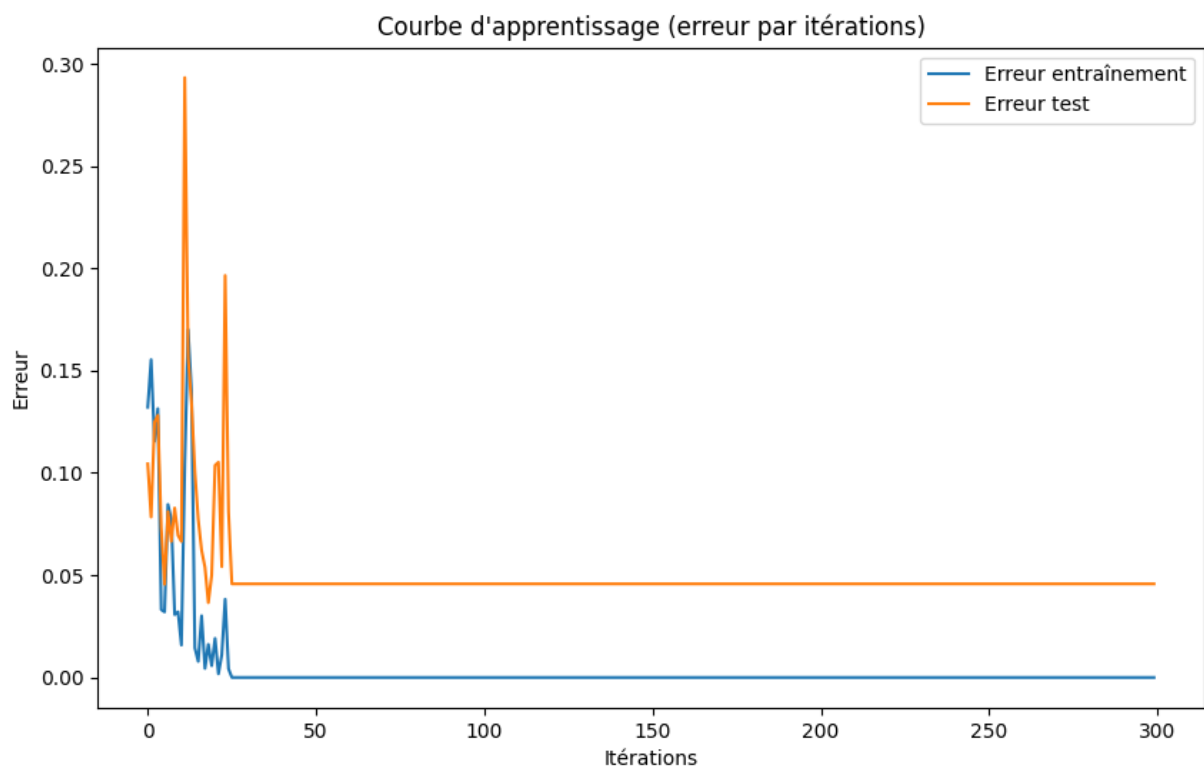
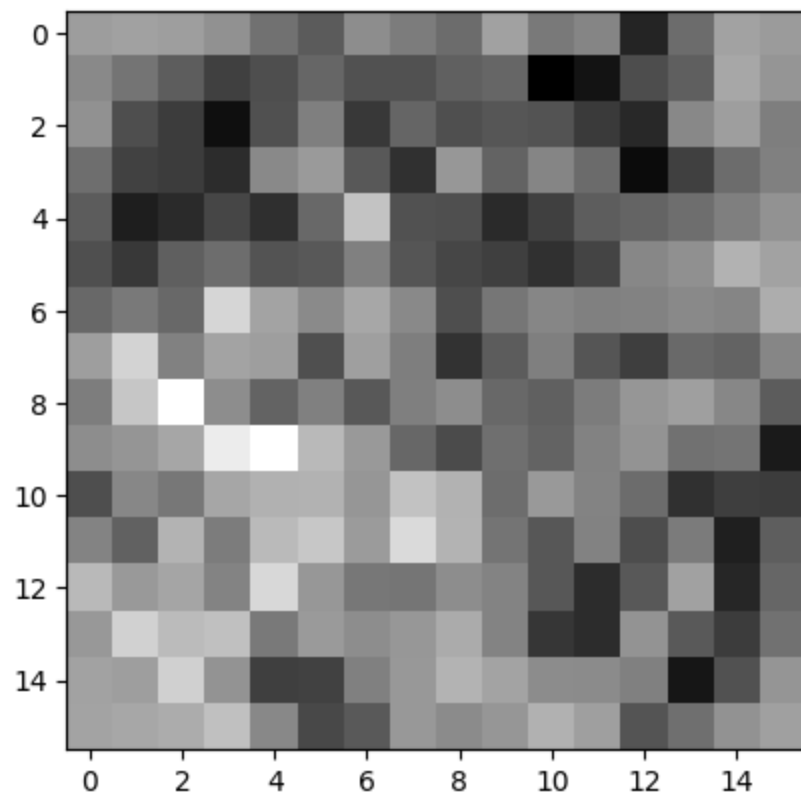
b) Apprentissage stochastique

```
In [30]: # Données non bruitées
lin_stoch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=
lin_stoch.fit_stochastic(datax, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique: ")
show_usps(lin_stoch.w.reshape(16, 16))
display_error(lin_stoch.train_errors, lin_stoch.test_errors)

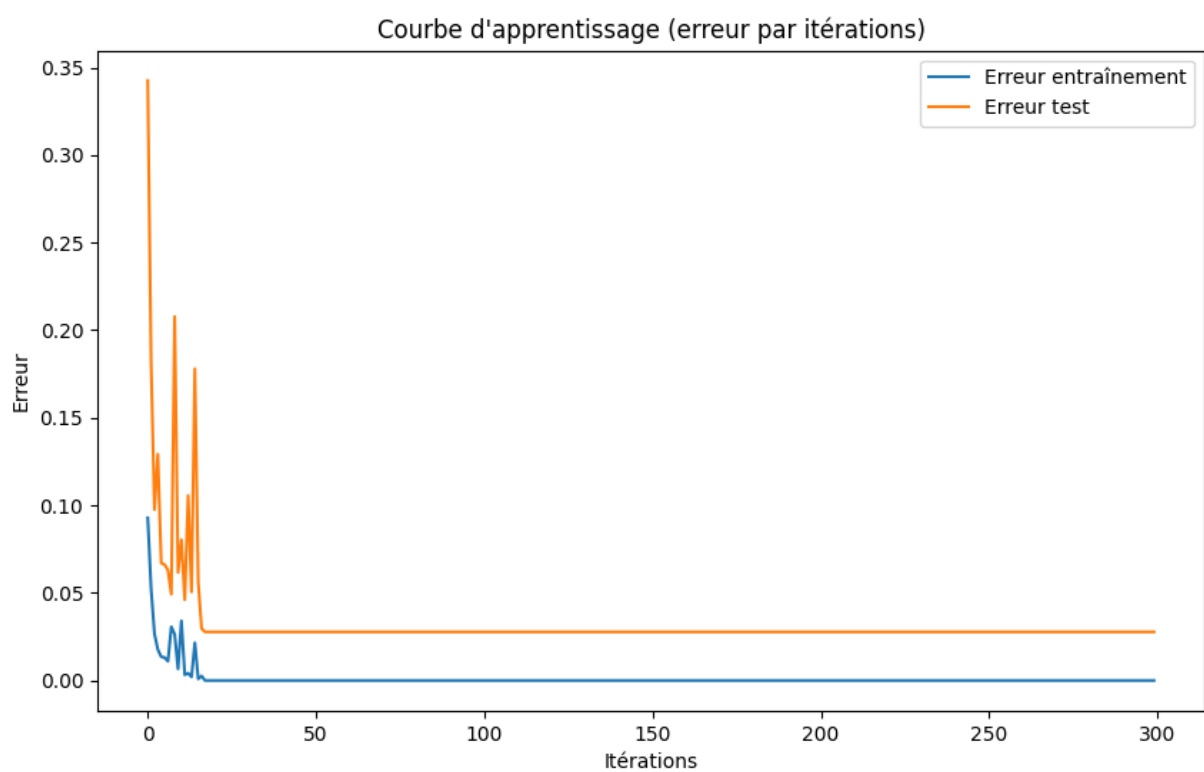
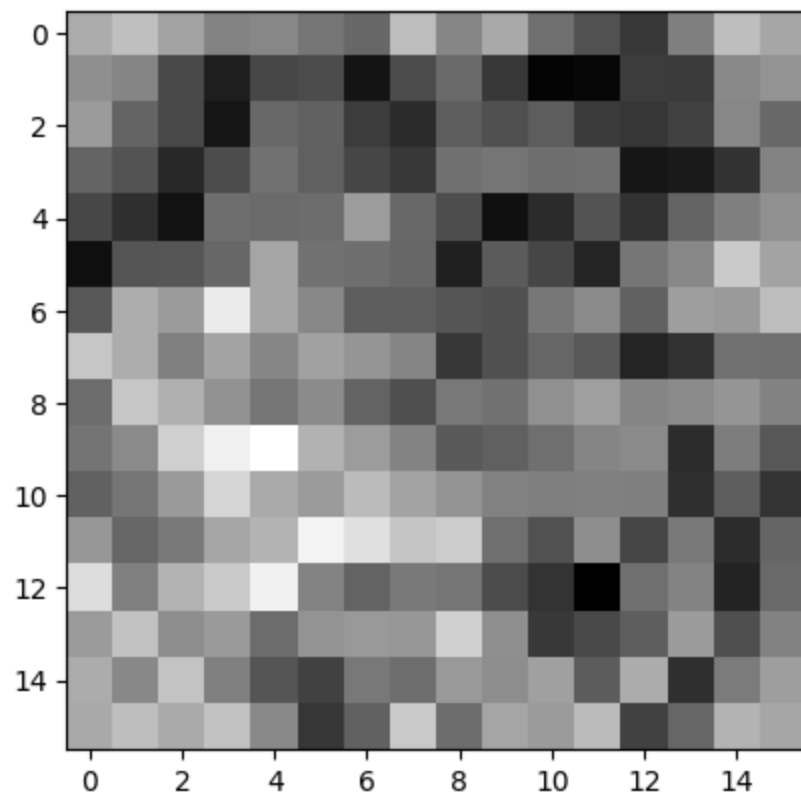
# Données bruitées 10%
lin_stoch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad,
noisy_x_train = add_noise(datax, eps=0.1)
lin_stoch_noised10.fit_stochastic(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique pour données brui
show_usps(lin_stoch_noised10.w.reshape(16, 16))
display_error(lin_stoch_noised10.train_errors, lin_stoch_noised10.test_error

# Données bruitées 50%
lin_stoch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad,
noisy_x_train = add_noise(datax, eps=0.5)
lin_stoch_noised50.fit_stochastic(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique pour données brui
show_usps(lin_stoch_noised50.w.reshape(16, 16))
display_error(lin_stoch_noised50.train_errors, lin_stoch_noised50.test_error
```

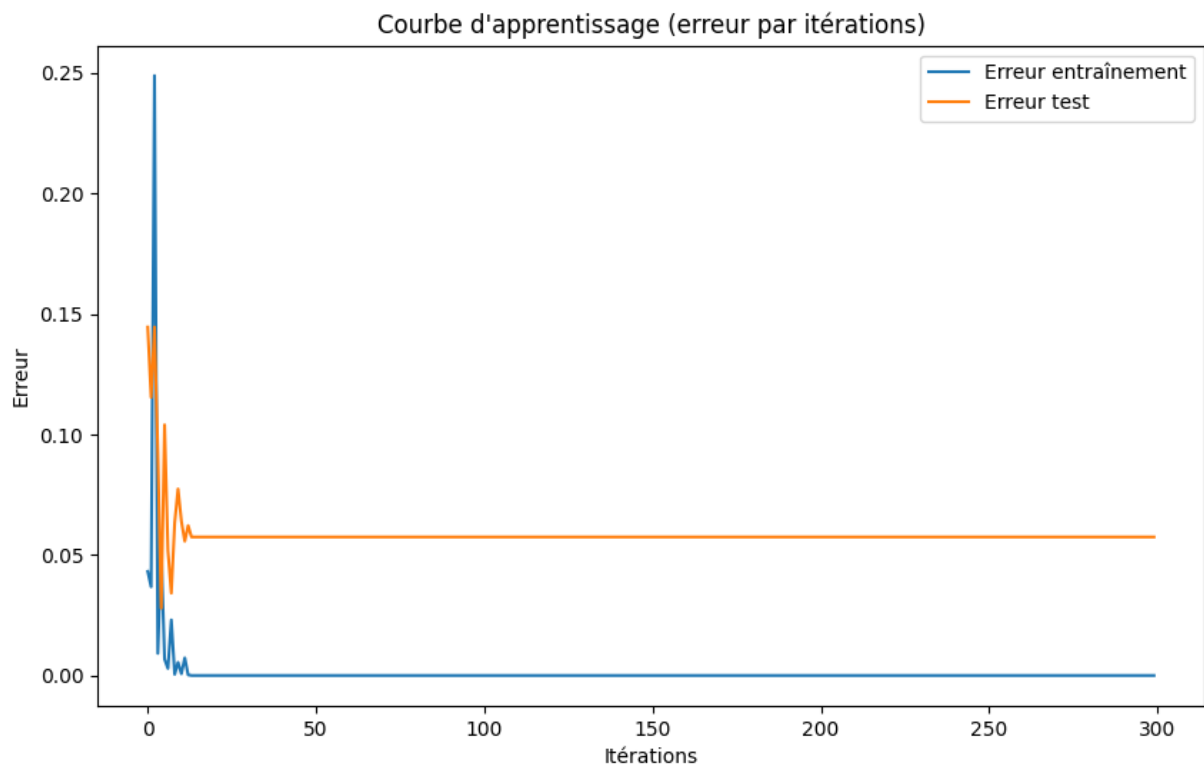
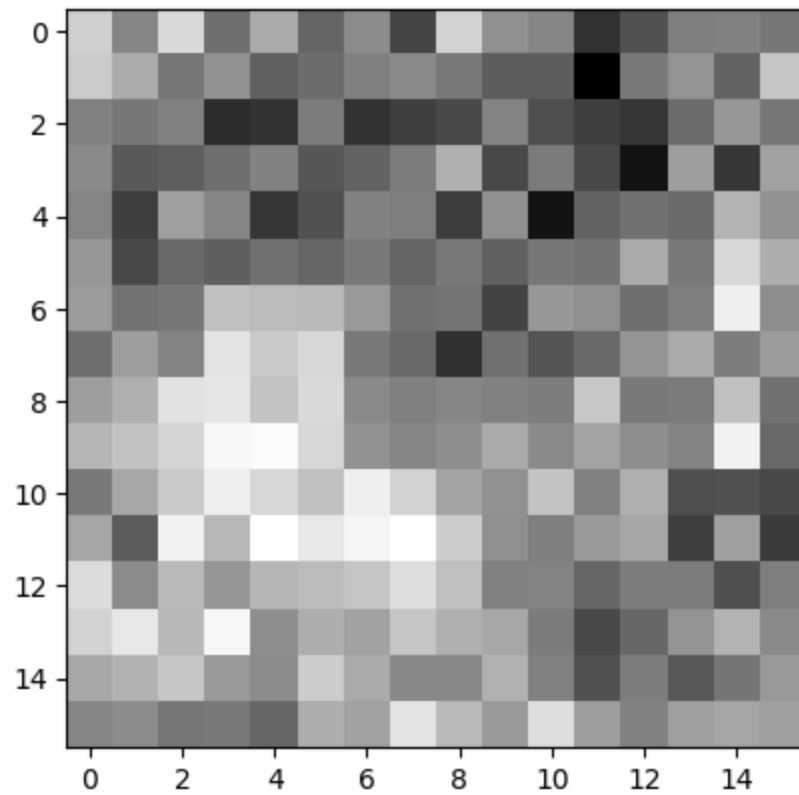
matrice de poids w après apprentissage stochastique:



matrice de poids w après apprentissage stochastique pour données bruitées (10%) :



matrice de poids w après apprentissage stochastique pour données bruitées (5 0%) :



c) Apprentissage par mini-batch

```
In [31]: # Données non bruitées
lin_mbatch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch
lin_mbatch.fit_minibatch(datax, datay, testx, testy, batch_size=16)
print("matrice de poids w après apprentissage par minibatch: ")
show_usps(lin_mbatch.w.reshape(16, 16))
```



```
display_error(lin_mbatch.train_errors, lin_mbatch.test_errors)

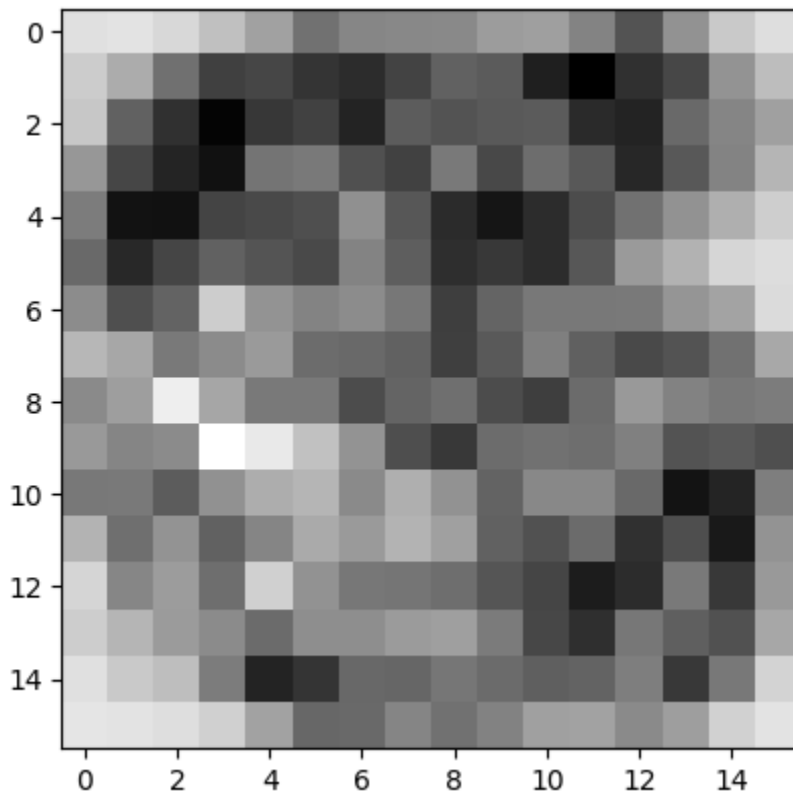
# Données bruitées 10%
lin_mbatch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad
noisy_x_train = add_noise(datax, eps=0.1)
lin_mbatch_noised10.fit_minibatch(noisy_x_train, datay, testx, testy, batch_

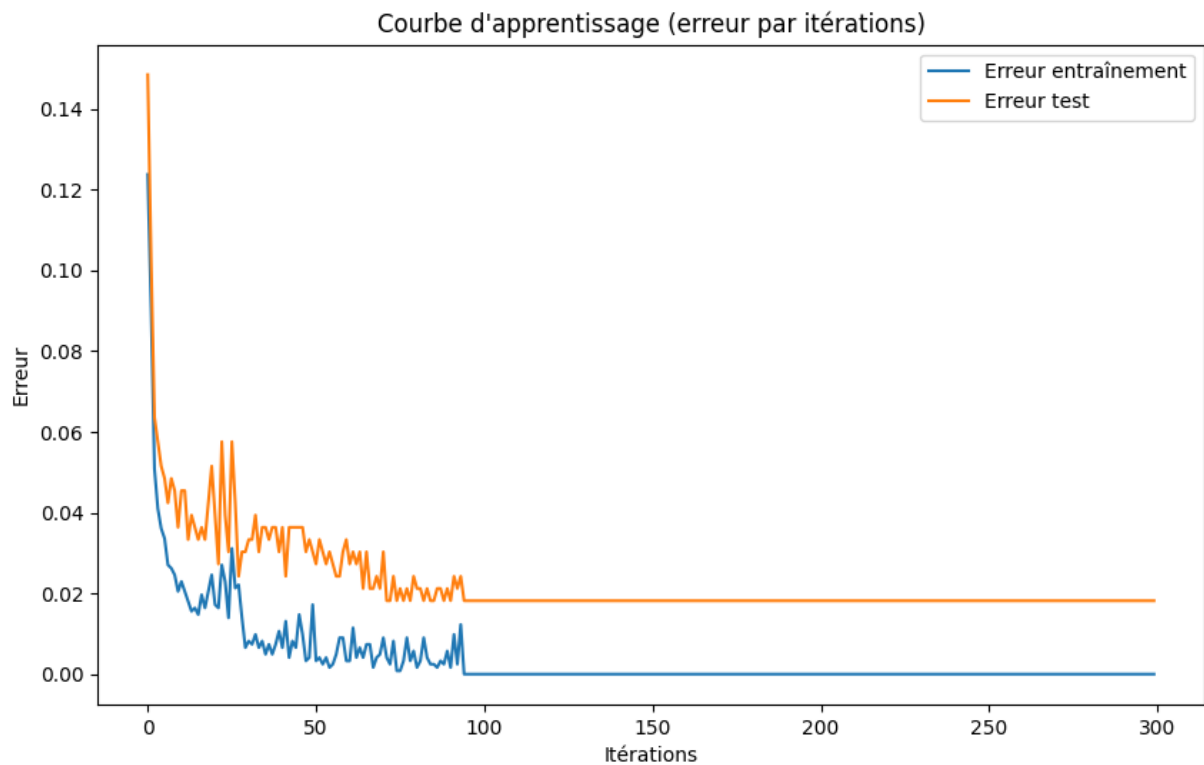
print("matrice de poids w après apprentissage par minibatch, pour des donnée
show_usps(lin_mbatch_noised10.w.reshape(16, 16))
display_error(lin_mbatch_noised10.train_errors, lin_mbatch_noised10.test_err

# Données bruitées 50%
lin_mbatch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad
noisy_x_train = add_noise(datax, eps=0.5)
lin_mbatch_noised50.fit_minibatch(noisy_x_train, datay, testx, testy, batch_

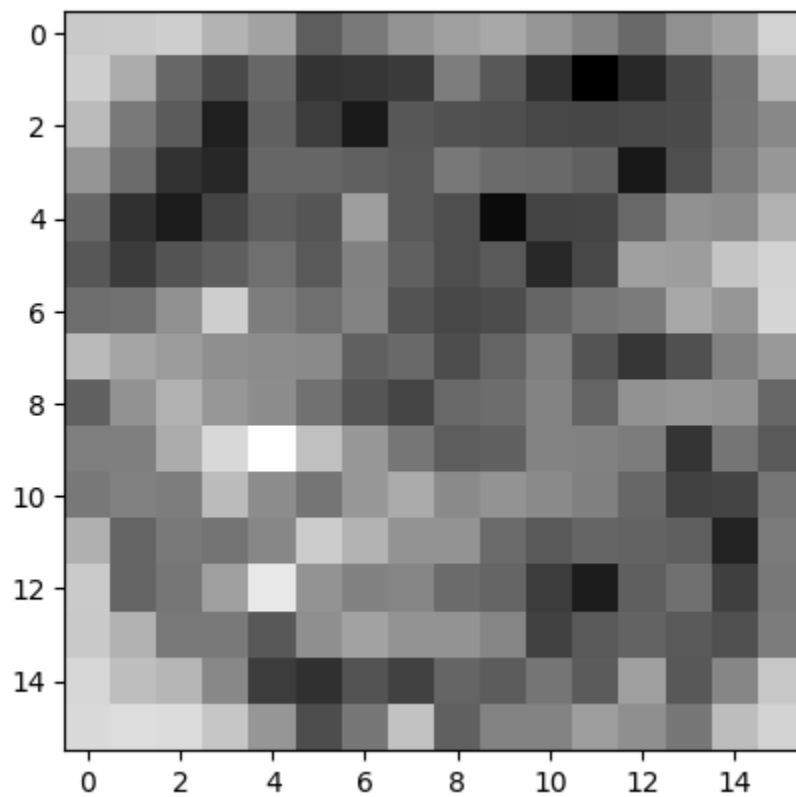
print("matrice de poids w après apprentissage par minibatch, pour des donnée
show_usps(lin_mbatch_noised50.w.reshape(16, 16))
display_error(lin_mbatch_noised50.train_errors, lin_mbatch_noised50.test_err
```

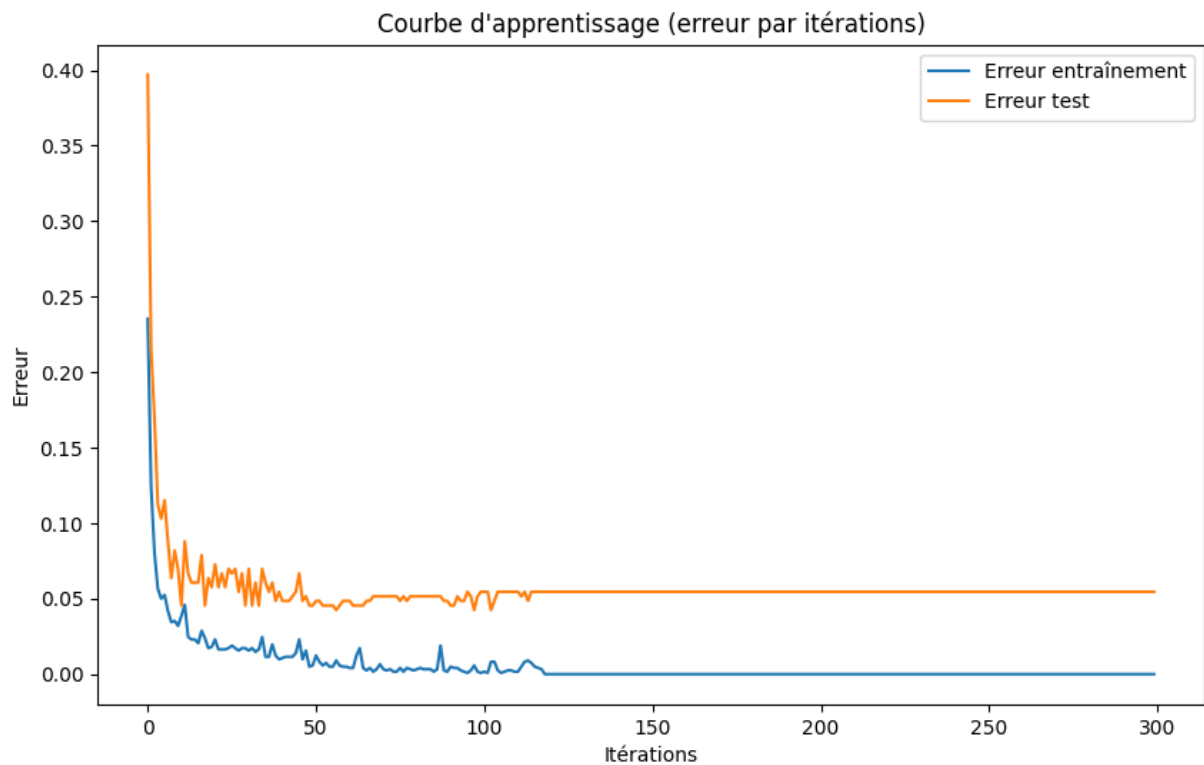
matrice de poids w après apprentissage par minibatch:



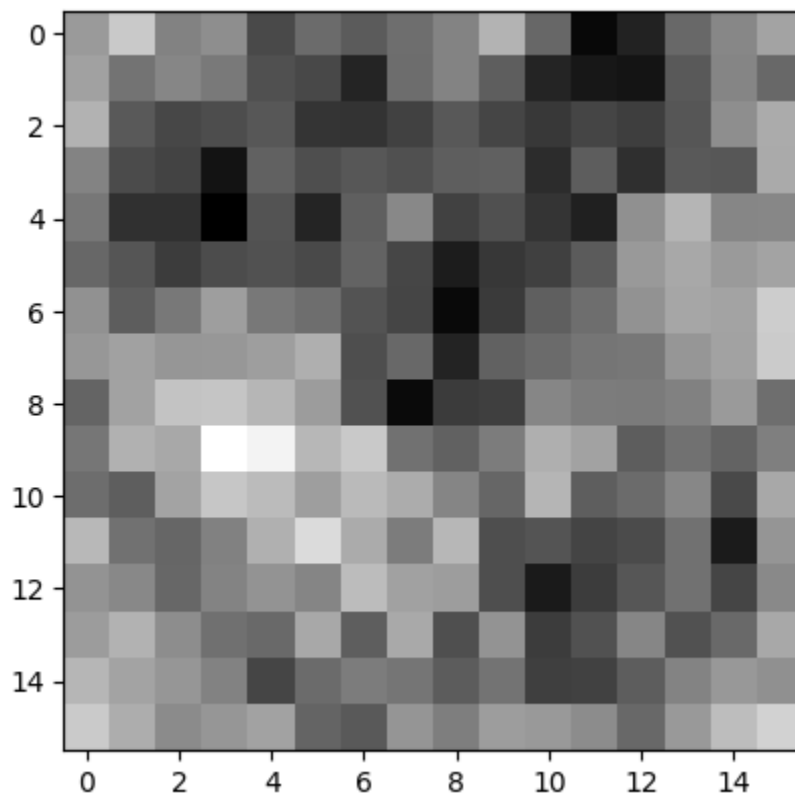


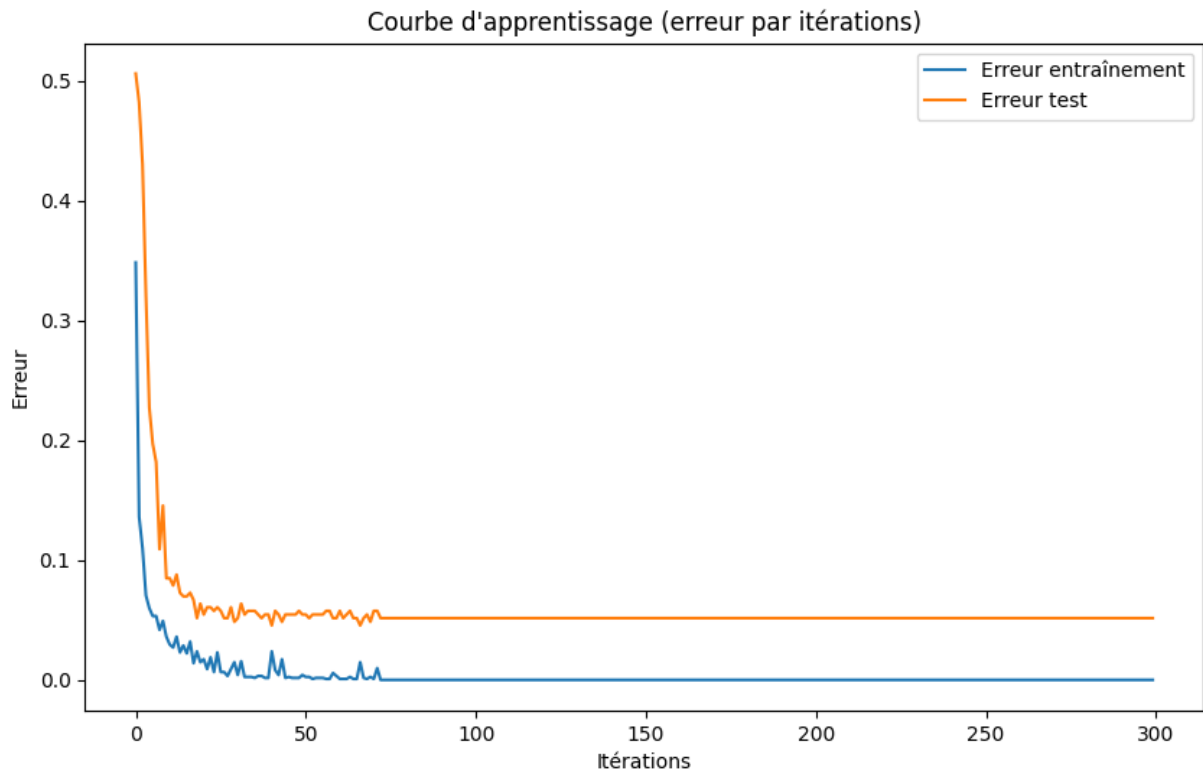
matrice de poids w après apprentissage par minibatch, pour des données bruitées (10%) :





matrice de poids w après apprentissage par minibatch, pour des données bruitées (50%) :





d) Comparaisons des vitesses de convergence selon la technique de descente de gradient (batch, mini-batch, stochastique) pour des données non bruitées

```
In [ ]: def compare_w(w_batch, w_stoch, w_mbatch) :
    """
    Compare the learned weights for batch, stochastic, and minibatch gradient descent

    Params:
        w_batch (np.ndarray): learned weights from batch gradient descent
        w_stoch (np.ndarray): learned weights from stochastic gradient descent
        w_mbatch (np.ndarray): learned weights from minibatch gradient descent
    """

    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.title("Batch")
    plt.imshow(w_batch.reshape(16, 16), cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.title("Stochastique")
    plt.imshow(w_stoch.reshape(16, 16), cmap='gray')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.title("Mini-batch")
    plt.imshow(w_mbatch.reshape(16, 16), cmap='gray')
    plt.axis('off')

    plt.suptitle("Comparaison des poids W appris")
    plt.tight_layout()
```

```
plt.show()
```

```
In [ ]: def compare_learning_curves(error_batch_train, error_batch_test, error_stoch
      """
      Compare the learning curves of batch, stochastic, and minibatch gradient
      descent.

      Params:
      error_batch_train (list): training error for batch gradient descent
      error_batch_test (list): test error for batch gradient descent
      error_stoch_train (list): training error for stochastic gradient descent
      error_stoch_test (list): test error for stochastic gradient descent
      error_mbatch_train (list): training error for minibatch gradient descent
      error_mbatch_test (list): test error for minibatch gradient descent
      """

      plt.figure(figsize=(10, 6))

      # Batch
      plt.plot(error_batch_train, label="Train - Batch", color='blue')
      plt.plot(error_batch_test, label="Test - Batch", color='skyblue')

      # Stochastique
      plt.plot(error_stoch_train, label="Train - Stochastique", color='green')
      plt.plot(error_stoch_test, label="Test - Stochastique", color='lightgreen')

      # Mini-batch
      plt.plot(error_mbatch_train, label="Train - Mini-batch", color='red')
      plt.plot(error_mbatch_test, label="Test - Mini-batch", color='orange')

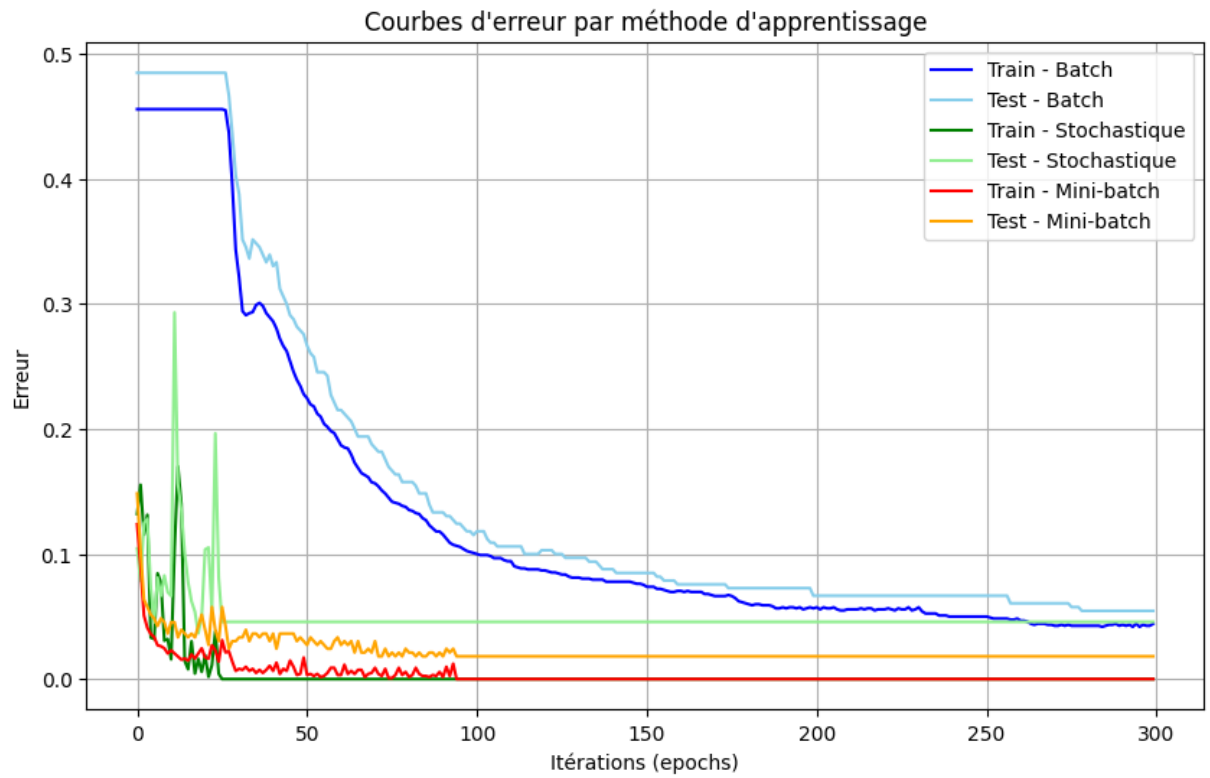
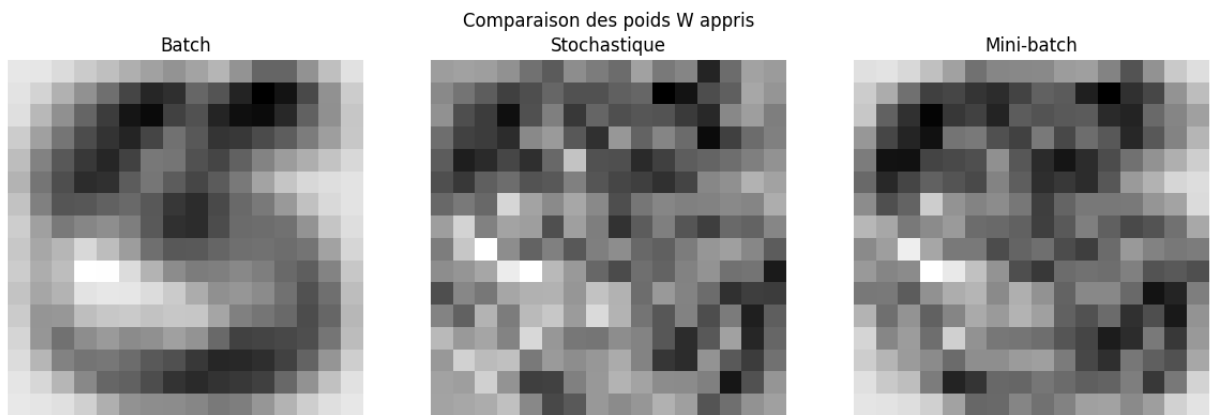
      plt.title("Courbes d'erreur par méthode d'apprentissage")
      plt.xlabel("Itérations (epochs)")
      plt.ylabel("Erreur")
      plt.legend()
      plt.grid(True)
      plt.show()
```

```
In [32]: # Données non bruitées
print("Données non bruitées")
compare_w(lin_batch.w, lin_stoch.w, lin_mbatch.w)
compare_learning_curves(lin_batch.train_errors, lin_batch.test_errors, lin_s

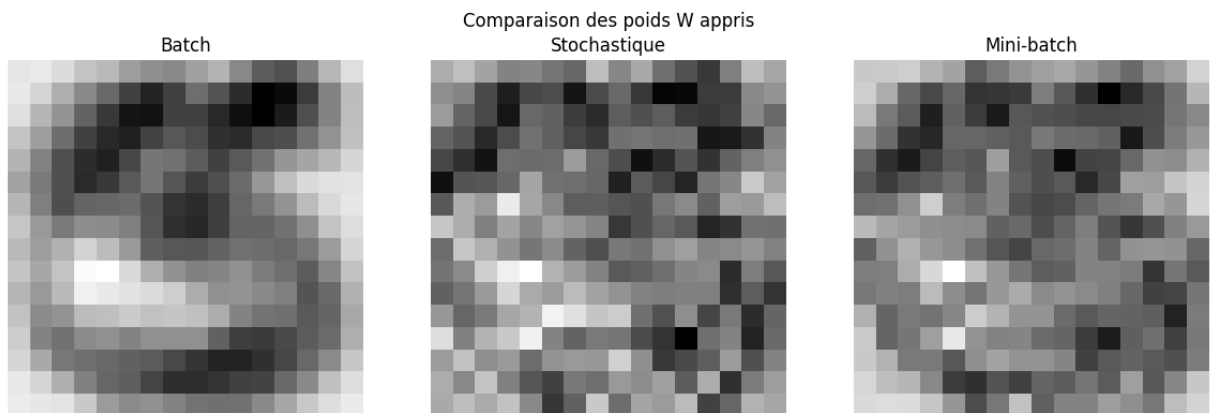
# Données bruitées 10%
print("Données bruitées 10%")
compare_w(lin_batch_noised10.w, lin_stoch_noised10.w, lin_mbatch_noised10.w)
compare_learning_curves(lin_batch_noised10.train_errors, lin_batch_noised10.

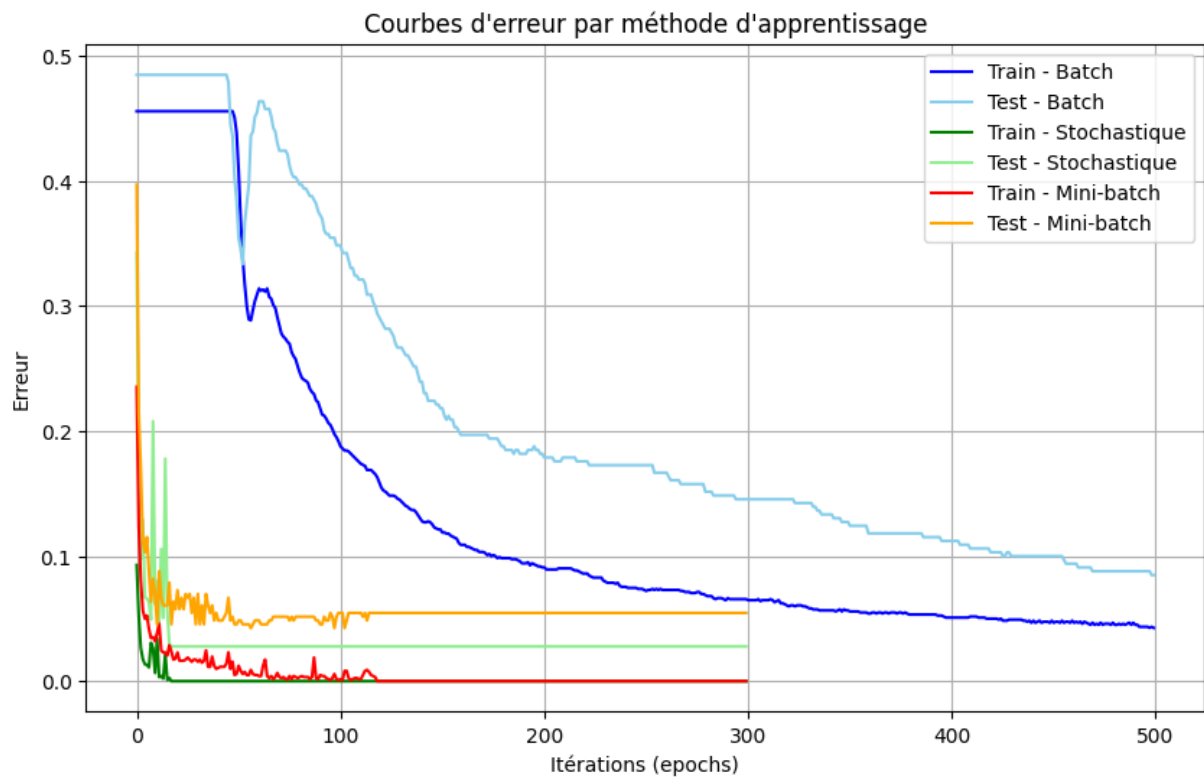
# Données bruitées 50%
print("Données bruitées 50%")
compare_w(lin_batch_noised50.w, lin_stoch_noised50.w, lin_mbatch_noised50.w)
compare_learning_curves(lin_batch_noised50.train_errors, lin_batch_noised50.
```

Données non bruitées

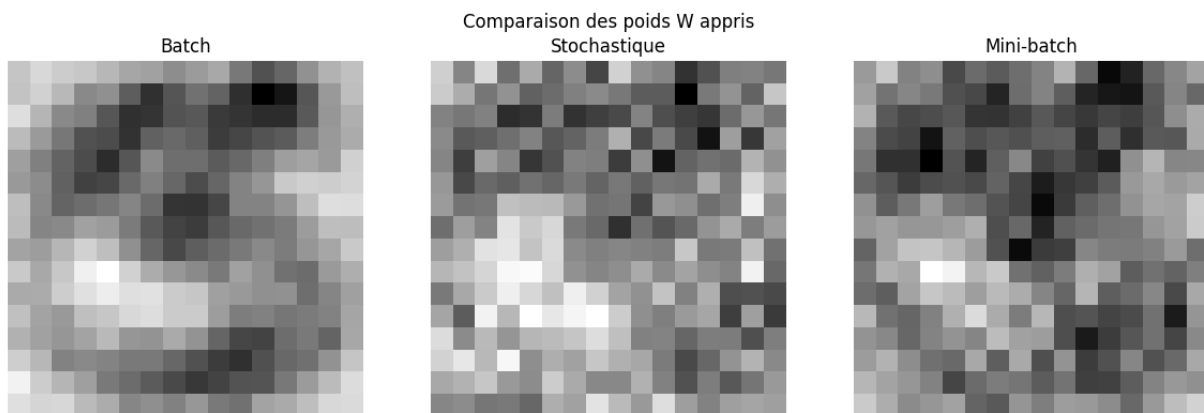


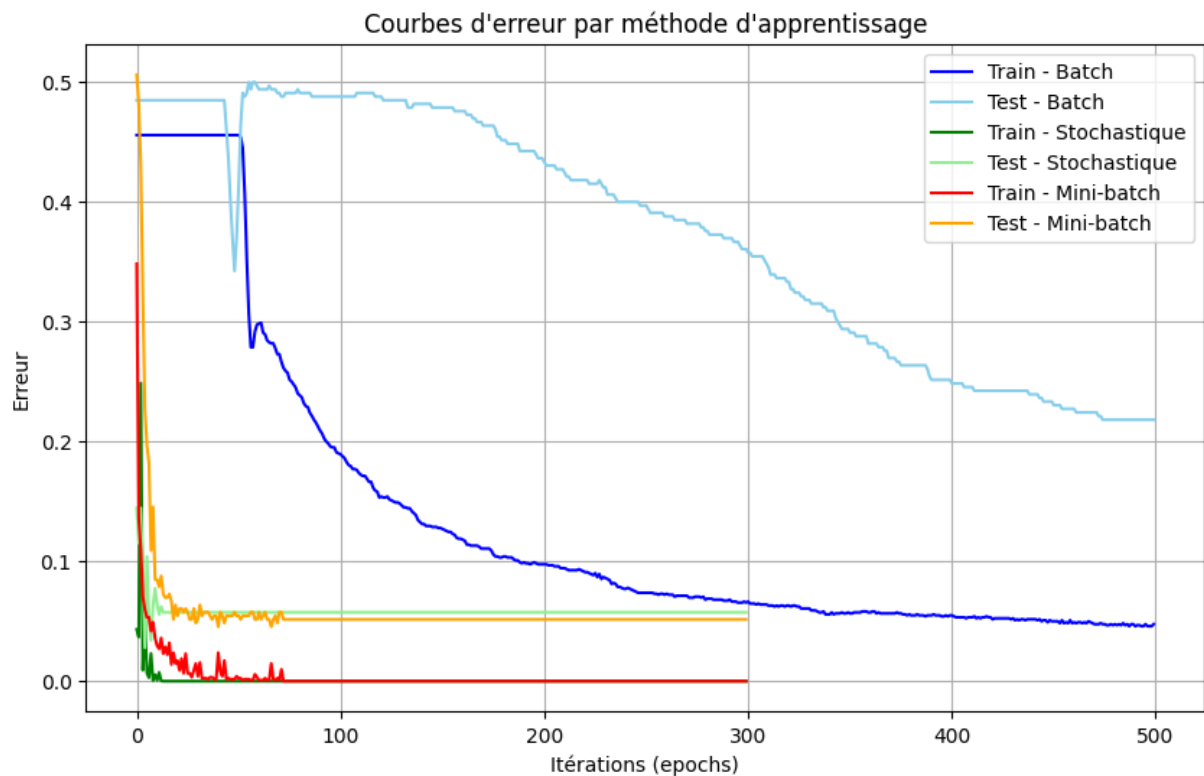
Données bruitées 10%





Données bruitées 50%





Interprétation des résultats

Les courbes d'erreurs pour les données train et tests nous permettent d'analyser les performances d'apprentissage des perceptrons linéaires.

En modifiant la technique de descente de gradient utilisée pour l'entraînement, on peut ainsi comparer les performances d'un même réseau de neurone pourvu d'une technique de descente de gradient différente (batch, mini-batch, stochastique) et donc élire potentiellement une technique de gradient plus optimale que les autres (par exemple).

Ici, on observe un apprentissage sur 300 itérations avec un pas d'apprentissage de $\epsilon = 0.07$.

- L'apprentissage par batch est celui qui met le plus d'itérations à converger (sur des données bruitées ou non). Il semble commencer à se stabiliser vraiment à 300 itérations pour les données non bruitées, et n'est toujours pas stabilisé à 300 itérations pour les données bruitées. Ce temps de convergence augmente proportionnellement à la proportion de bruit des données. La performance de test est plutôt proche de celle de train et sont inférieures à 0.1. D'un point de vue du temps d'exécution, l'apprentissage par batch est rapide.
- L'apprentissage stochastique est celui qui converge le plus rapidement. A 25 itérations, pour des données non bruitées il semble déjà stabilisé. Ce temps de convergence semble rester constant même en augmentant la proportion de bruit des données. Ses performances d'erreur sont bonnes ($< 0.1\%$ d'erreur) mais inférieures au mini batch. En terme de temps de calcul de l'algorithme, il est le plus long des trois algorithmes testés, à l'exécution.
- L'apprentissage par mini-batch semble être l'algorithme d'apprentissage le plus

intermédiaire. Il est à la fois rapide en terme d'exécution, présente de bonnes performances d'apprentissage avec des erreurs tests et trains proches entre elles et proche de 0 et converge assez rapidement (<100 itérations pour un bruit de 50% sur les données).

III - Projection et pénalisation

III - 1) Fonctions

a) Projections

```
In [ ]: def proj_poly(datax):  
    """  
    Project input data into polynomial features (linear, squared, and intera  
    Params:  
        datax (np.ndarray): input features  
    Returns:  
        np.ndarray: augmented feature set with bias, linear, squared, and in  
    """  
  
    n, d = datax.shape  
    bias = np.ones((n, 1))  
    lin = datax  
    squares = datax**2  
    interactions = []  
    for i in range(d):  
        for j in range(i+1, d):  
            interactions.append((datax[:, i]*datax[:, j]).reshape(-1, 1))  
    interactions = np.hstack(interactions) if interactions else np.empty((n,  
    return np.hstack([bias, lin, squares, interactions])
```

```
In [ ]: def proj_biais(datax):  
    """  
    Add a bias term (column of ones) to the input data.  
    Params:  
        datax (np.ndarray): input features  
    Returns:  
        np.ndarray: input data with an additional bias column  
    """  
  
    n = datax.shape[0]  
    return np.hstack([np.ones((n, 1)), datax])
```

```
In [ ]: def proj_gauss(datax, base, sigma):  
    """  
    Project input data onto a Gaussian radial basis function (RBF) space.
```

```

Params:
    datax (np.ndarray): input features
    base (np.ndarray): centers of the Gaussian basis functions
    sigma (float): standard deviation of the Gaussian function

Returns:
    np.ndarray: projected data using Gaussian RBF kernels
"""

n_samples = datax.shape[0]
n_bases = base.shape[0]

proj = np.zeros((n_samples, n_bases))
for i in range(n_bases):
    distances = np.linalg.norm(datax - base[i], axis=1)
    proj[:, i] = np.exp(- (distances ** 2) / (2 * sigma ** 2))

return proj

```

b) Apprentissage

```

In [ ]: class Lineaire3:
    def __init__(self, loss=perceptron_loss, loss_g=perceptron_grad, proj=None)
        """
        Linear classifier with optional projection (e.g., polynomial or Gauss

        Params:
            loss (callable): loss function (default is perceptron_loss)
            loss_g (callable): gradient of the loss function (default is per
            proj (callable, optional): projection function (e.g., for polyno
            n_epoch (int): number of training epochs
            eps (float): learning rate
        """

        self.loss = loss
        self.loss_g = loss_g
        self.proj = proj
        self.n_epoch = n_epoch
        self.eps = eps
        self.w = None
        self.loss_history = []
        self.train_errors = []
        self.test_errors = []
        self.gauss_base = None

    def predict(self, datax):
        """
        Predict class labels after applying the projection function.

        Params:
            datax (np.ndarray): input features

        Returns:
            np.ndarray: predicted labels (+1 or -1)
        """

```

```

        datax = self.proj(datax)
        return np.sign(datax @ self.w)

def score(self, datax, datay):
    """
    Compute accuracy score of the model after projection.

    Params:
        datax (np.ndarray): input features
        datay (np.ndarray): true labels

    Returns:
        float: mean accuracy
    """

    y_pred = self.predict(datax)
    return (y_pred == datay).mean()

def fit(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model with projection using gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Returns:
        list: history of loss values during training
    """

    datax_proj = self.proj(datax)
    if testx is not None:
        testx_proj = self.proj(testx)
    else:
        testx_proj = None

    n, d = datax_proj.shape
    self.w = np.ones(d)
    self.loss_history = []
    self.train_errors = []
    self.test_errors = []

    for i in range(self.n_epoch):
        loss_val = self.loss(self.w, datax_proj, datay)
        self.loss_history.append(loss_val)
        grad = self.loss_g(self.w, datax_proj, datay)
        new_w = self.w - self.eps * grad
        self.w = new_w

        if testx is not None and testy is not None:
            train_error = 1 - self.score(datax, datay) # calcul de l'err
            test_error = 1 - self.score(testx, testy)
            self.train_errors.append(train_error) #stock it

```

```
self.test_errors.append(test_error)

return self.loss_history
```

III.2) Tests et résultats

a) générer des données

```
In [ ]: x0, y0 = gen_arti(epsilon=0.8)
x0_train, x0_test, y0_train, y0_test = train_test_split(x0, y0, test_size=0.
x1, y1 = gen_arti(data_type=1, epsilon=0.8)
x1_train, x1_test, y1_train, y1_test = train_test_split(x1, y1, test_size=0.
x2, y2 = gen_arti(data_type=2, epsilon=0.8)
x2_train, x2_test, y2_train, y2_test = train_test_split(x2, y2, test_size=0.
```

b) Projection polynomiales

```
In [202... # apprentissage data 0
print("Apprentissage linéaire pour données 0 projetées poly")
pol0 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500,
pol0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())

plot_frontiere(x0, pol0.predict)
plot_data(x0, y0)
display_error(pol0.train_errors, pol0.test_errors)

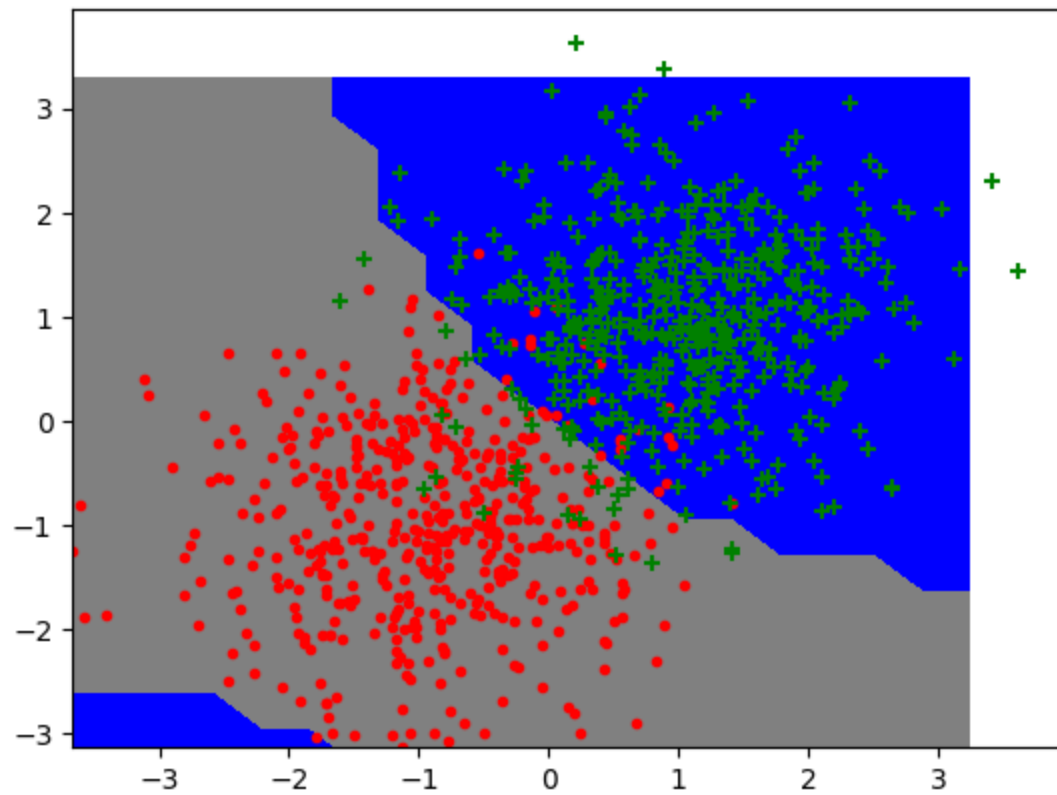
# apprentissage data 1
print("Apprentissage linéaire pour données 1 projetées poly")
pol1 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500,
pol1.fit(x1_train, y1_train.ravel(), x1_test, y1_test.ravel())

plot_frontiere(x1, pol1.predict)
plot_data(x1, y1)
display_error(pol1.train_errors, pol1.test_errors)

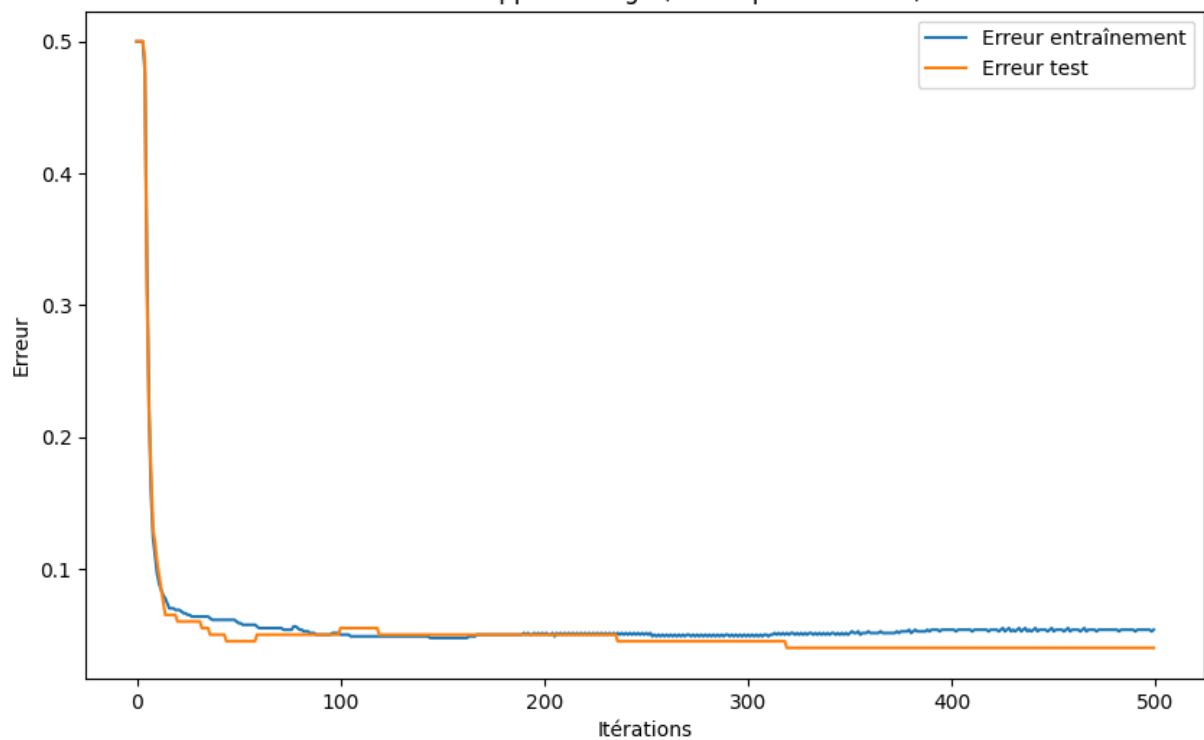
# apprentissage data 2
print("Apprentissage linéaire pour données 2 projetées poly")
pol2 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500,
pol2.fit(x2_train, y2_train.ravel(), x2_test, y2_test.ravel())

plot_frontiere(x2, pol2.predict)
plot_data(x2, y2)
display_error(pol2.train_errors, pol2.test_errors)
```

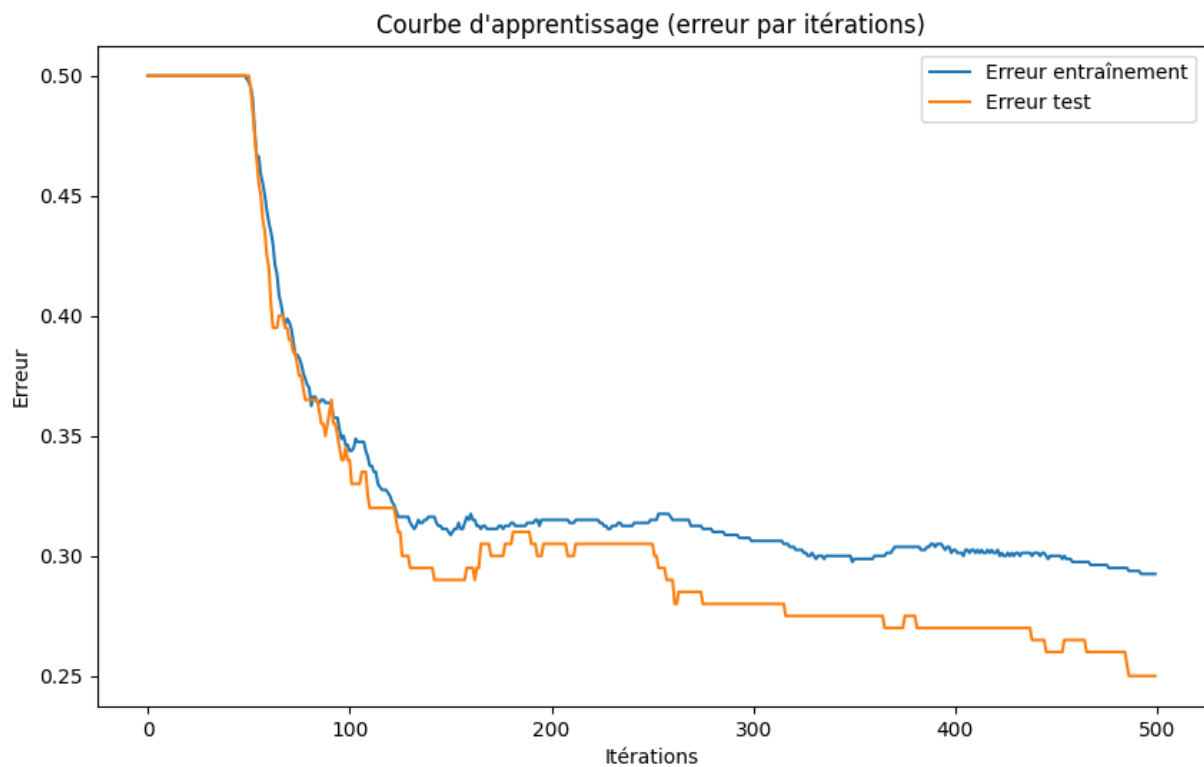
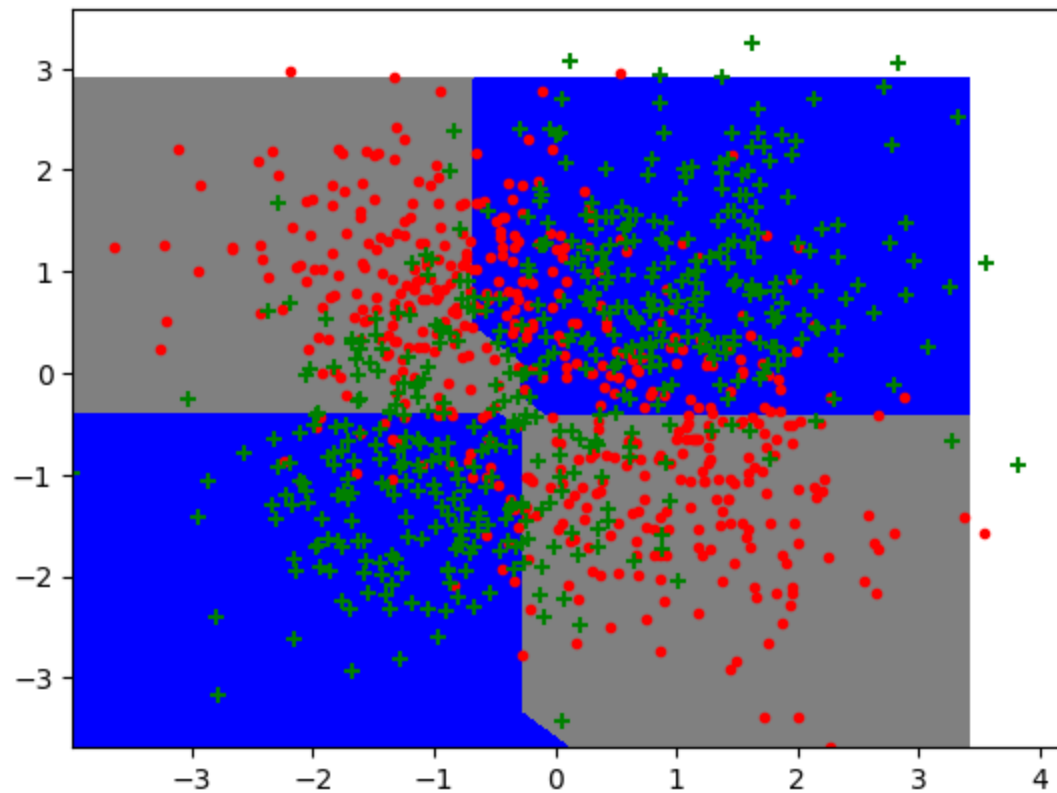
Apprentissage linéaire pour données 0 projetées poly



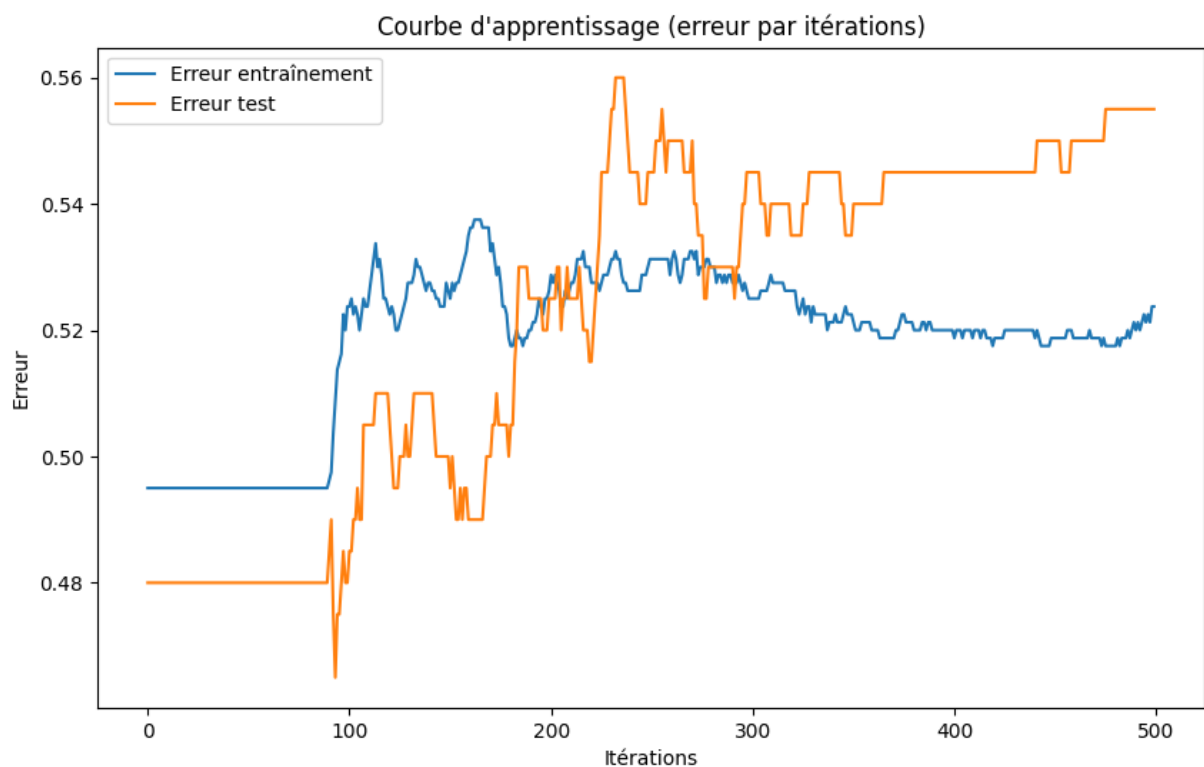
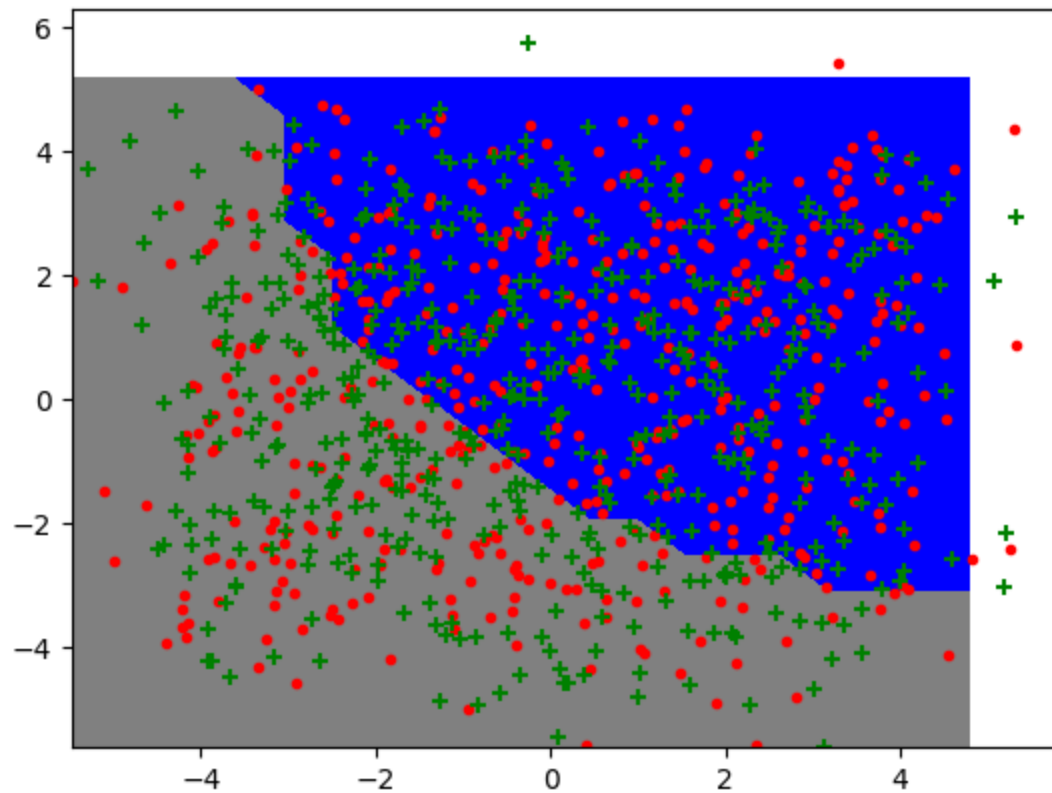
Courbe d'apprentissage (erreur par itérations)



Apprentissage linéaire pour données 1 projetées poly



Apprentissage linéaire pour données 2 projetées poly



c) Projection gaussiennes

```
In [203... # projection gaussienne
base_size = 100
index_b = np.random.choice(np.arange(1, datax.shape[0]), base_size)
base = datax[index_b]
for sigma in [0.5, 1, 1.5, 2] :
```

```

print("Sigma = ", sigma)
# apprentissage data 0
gauss0 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch
gauss0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())

plot_frontiere(x0, gauss0.predict)
plot_data(x0, y0)
plt.show()
display_error(gauss0.train_errors, gauss0.test_errors)

# apprentissage data 1
gauss1 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch
gauss1.fit(x1_train, y1_train.ravel(), x1_test, y1_test.ravel())

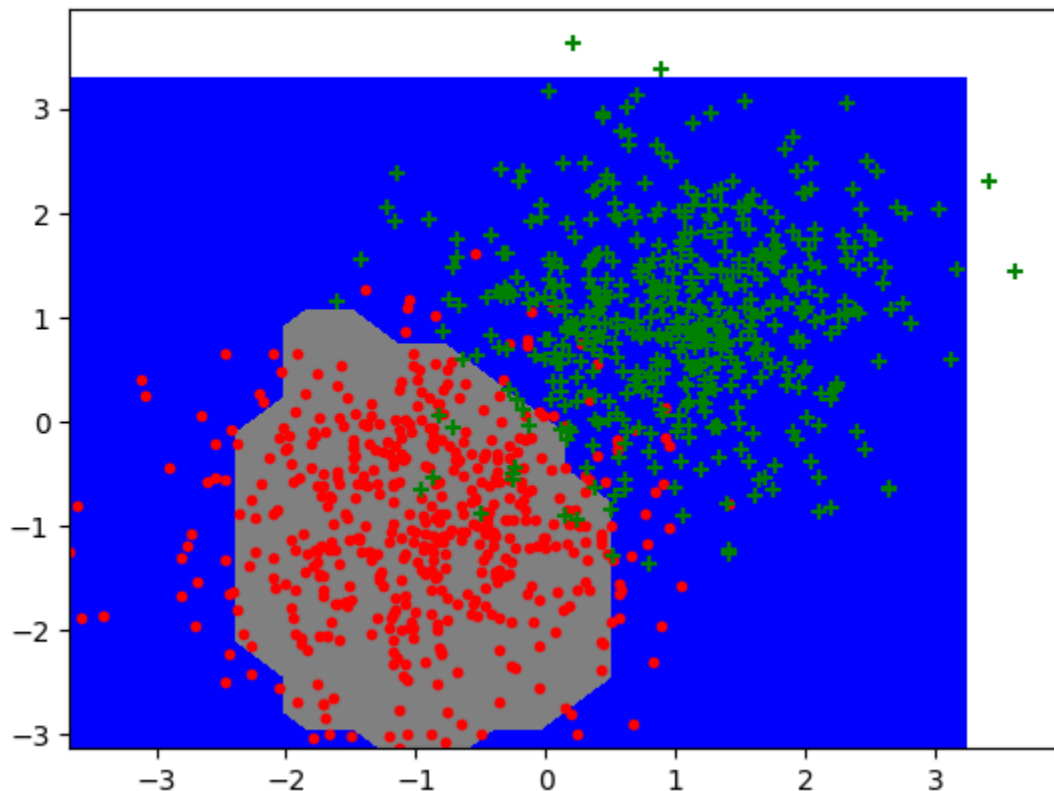
plot_frontiere(x1, gauss1.predict)
plot_data(x1, y1)
plt.show()
display_error(gauss1.train_errors, gauss1.test_errors)

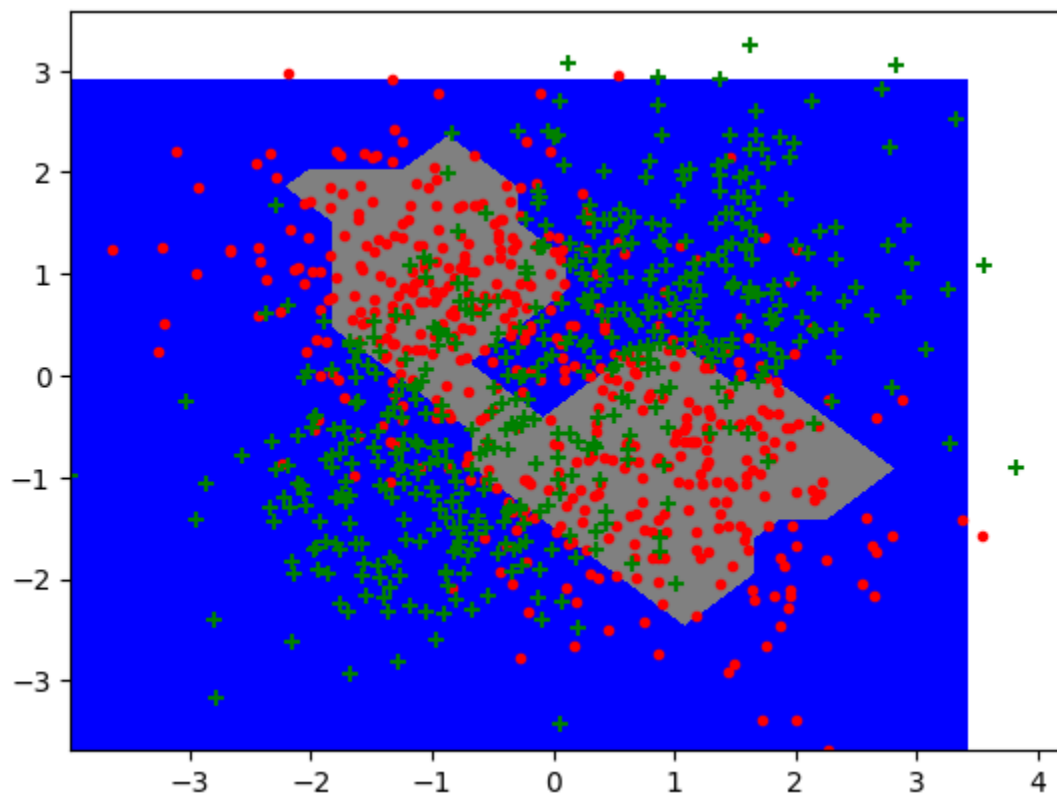
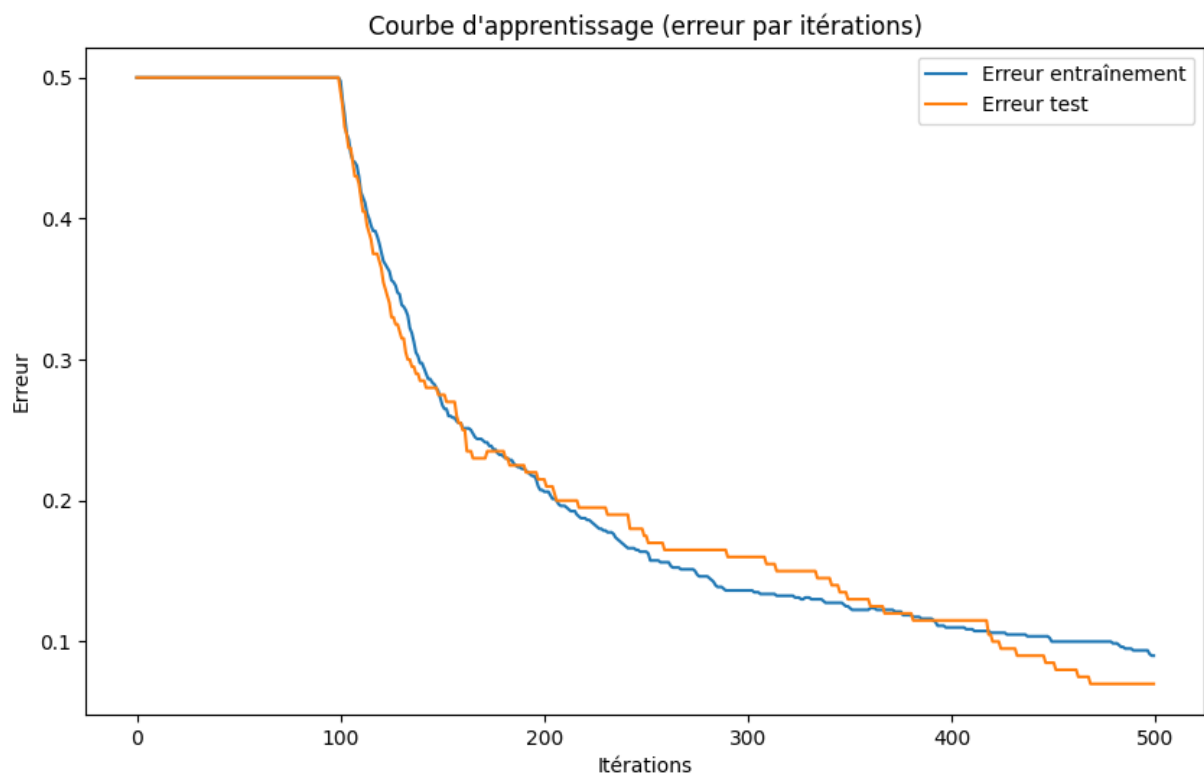
# apprentissage data 2
gauss2 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch
gauss2.fit(x2_train, y2_train.ravel(), x2_test, y2_test.ravel())

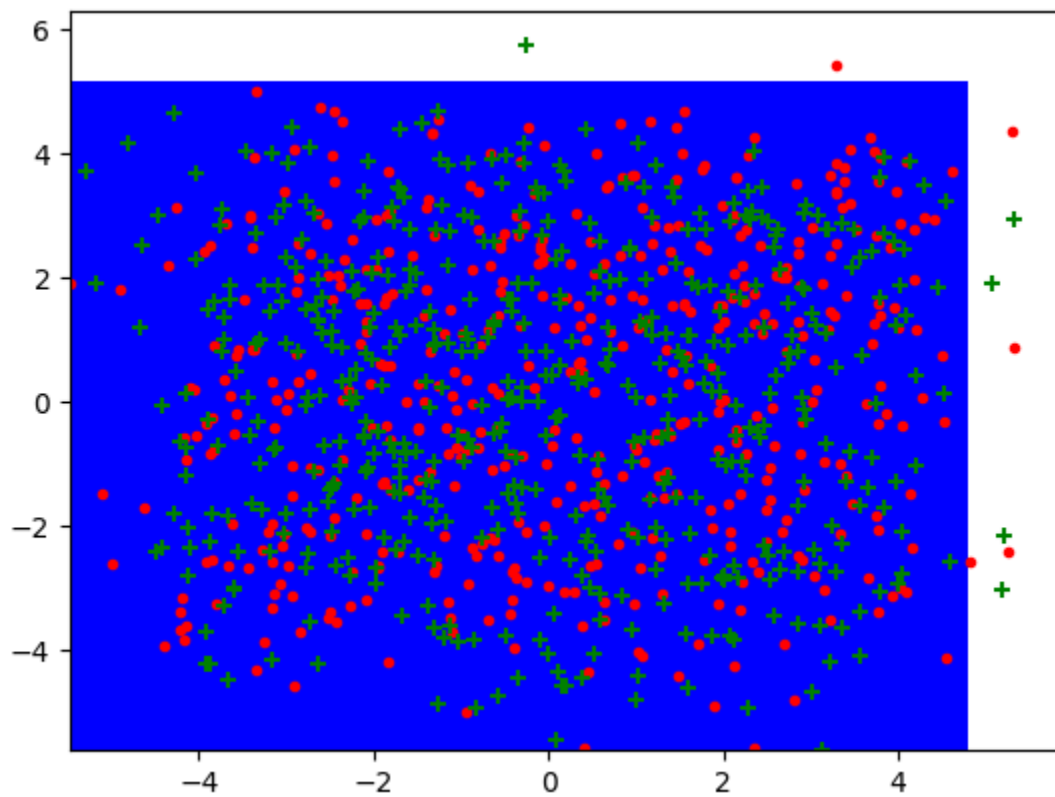
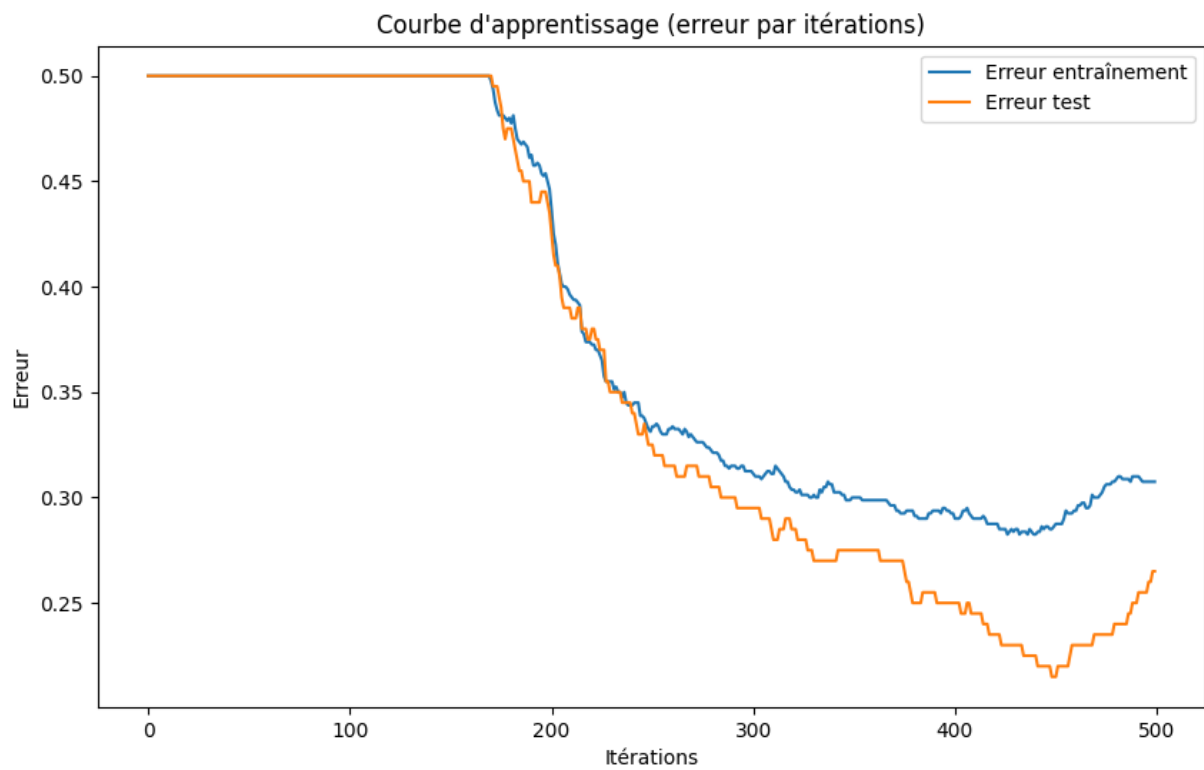
plot_frontiere(x2, gauss2.predict)
plot_data(x2, y2)
plt.show()
display_error(gauss2.train_errors, gauss2.test_errors)

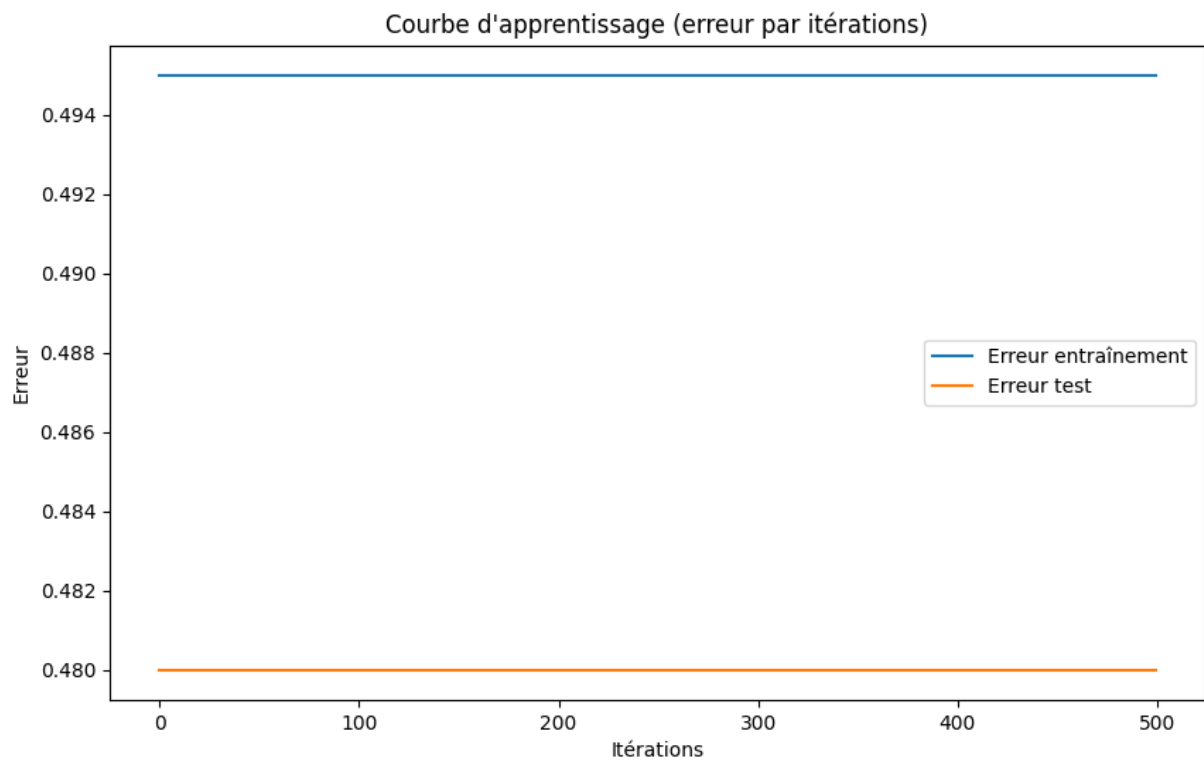
```

Sigma = 0.5

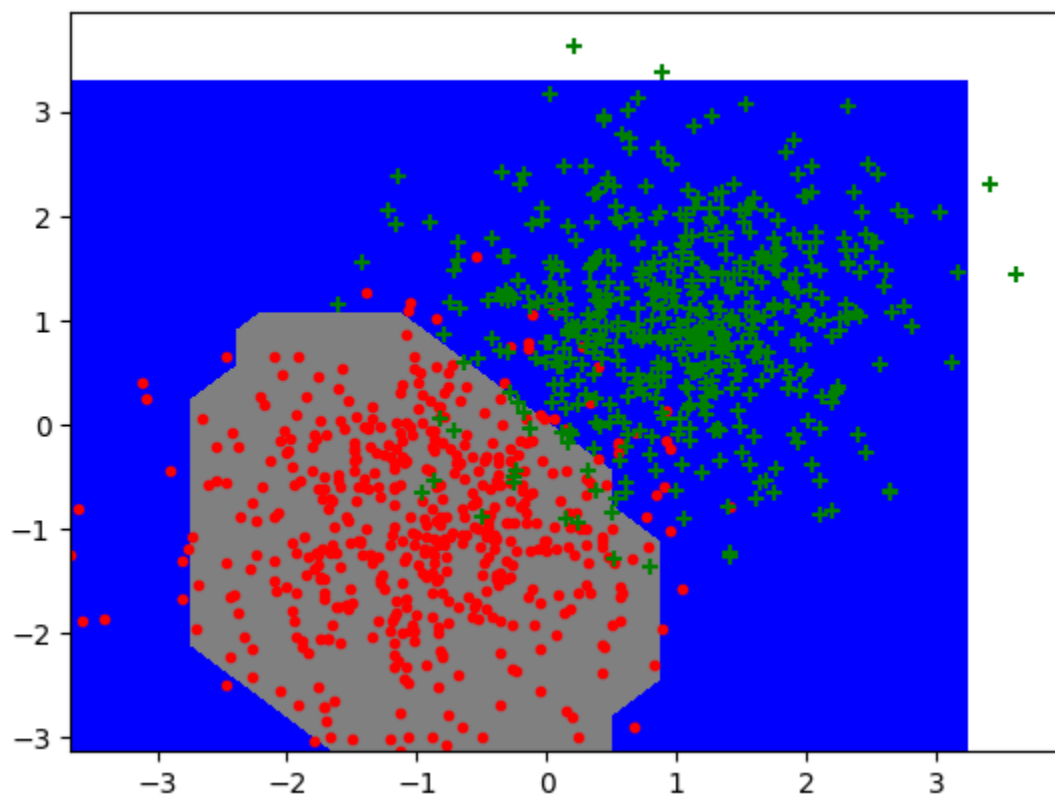


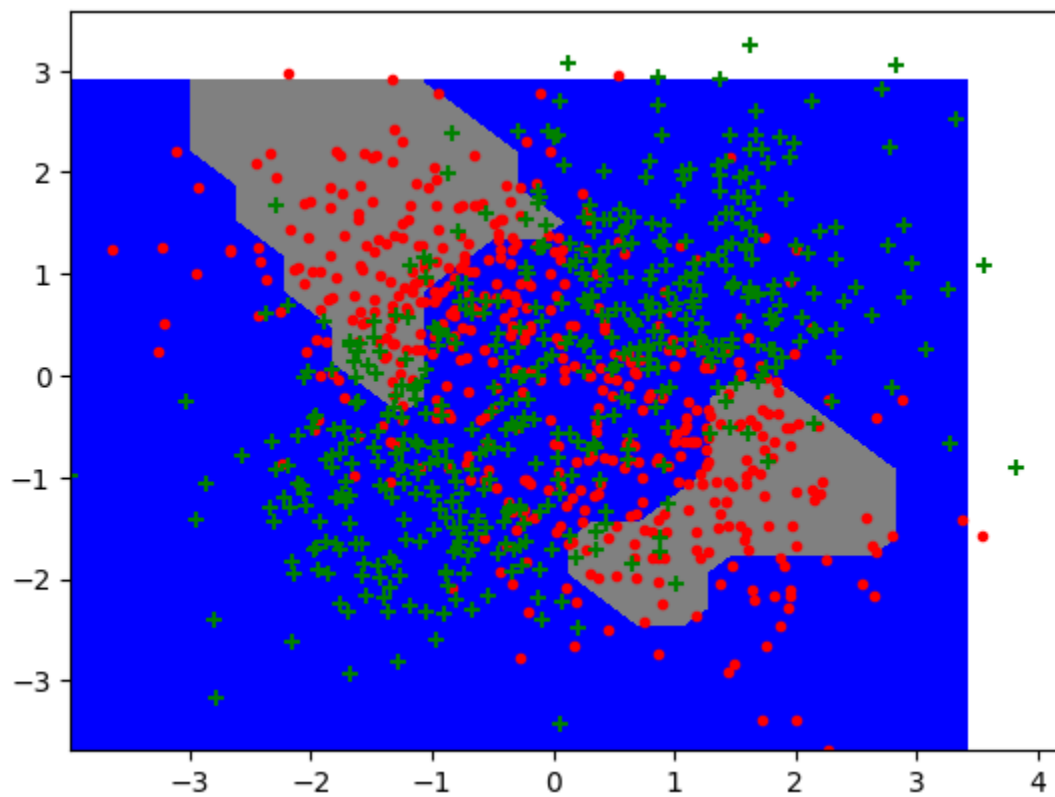
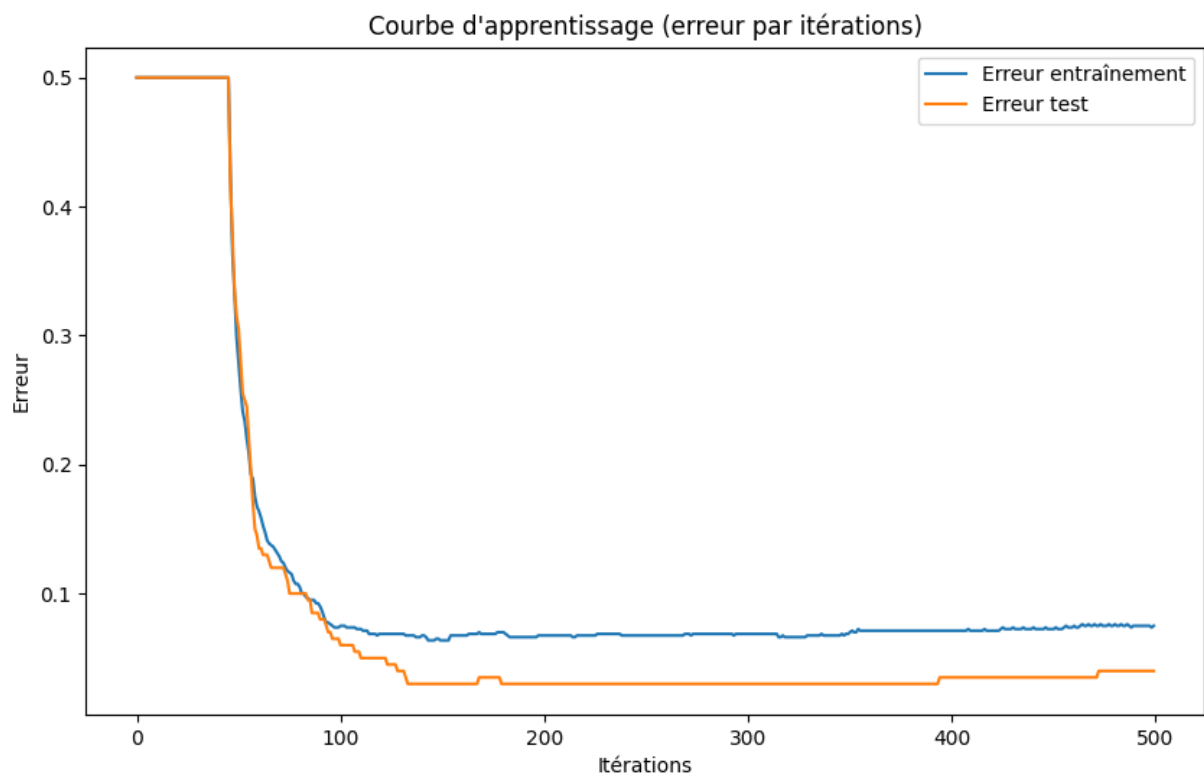


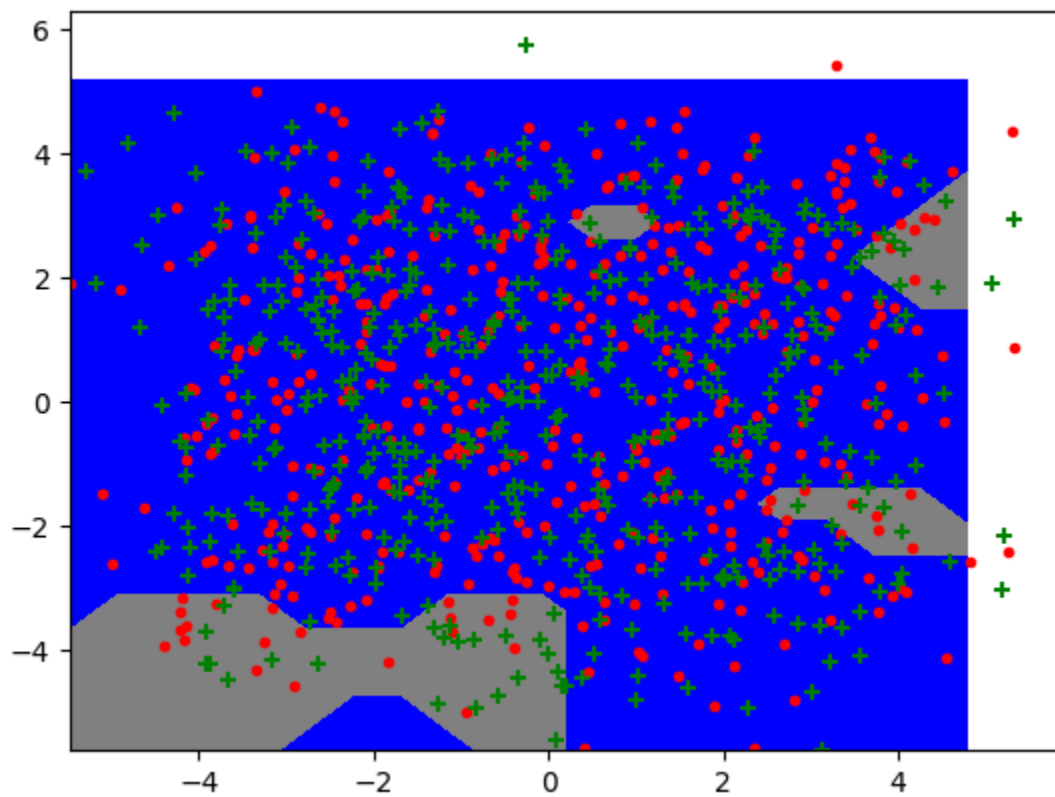
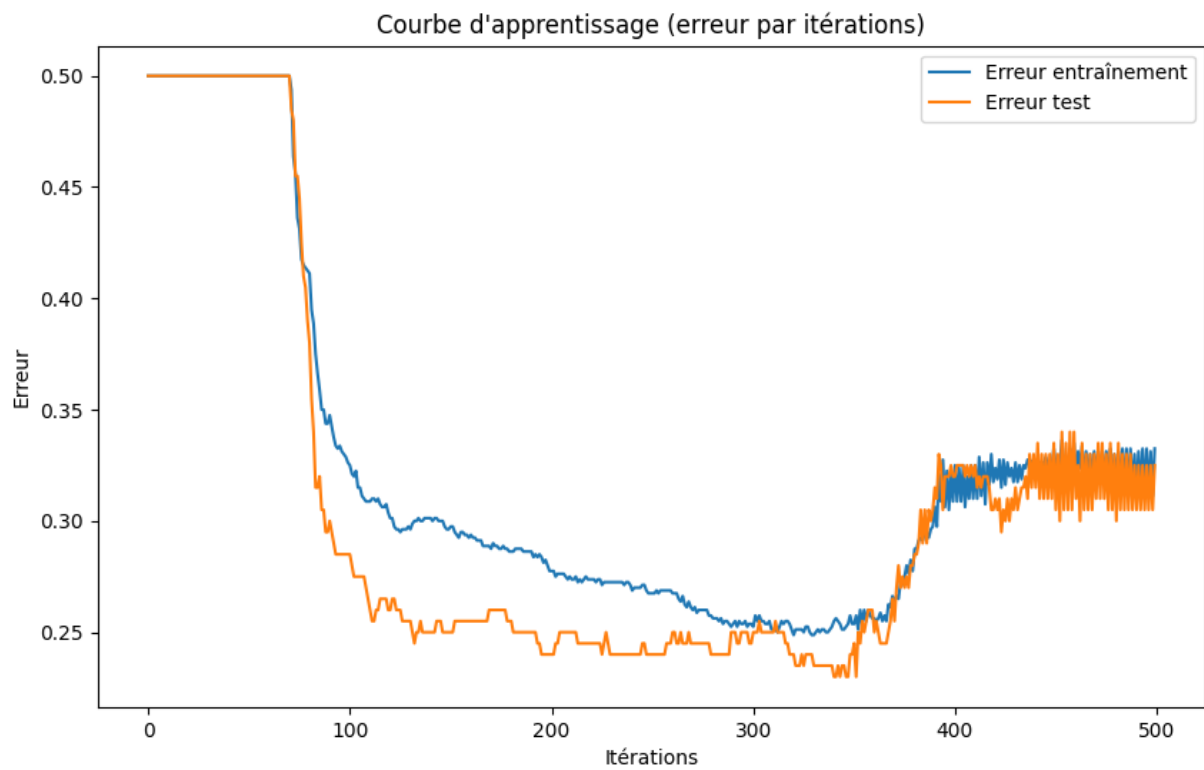


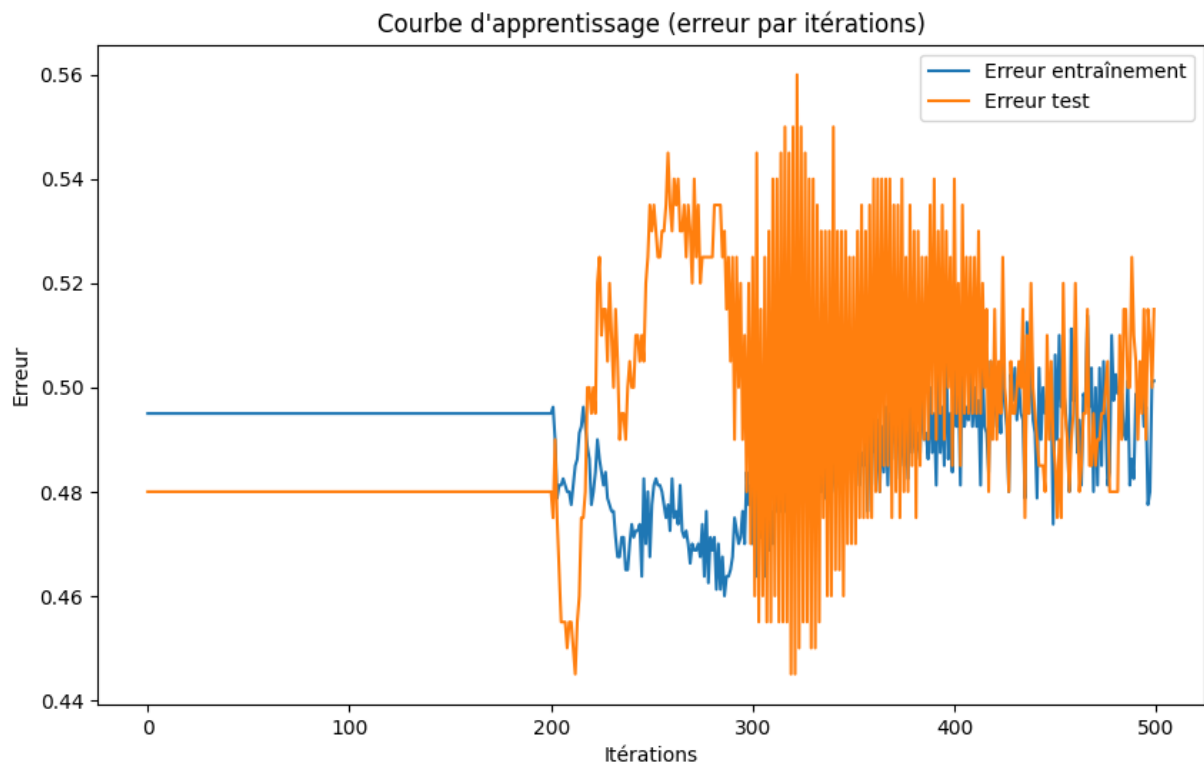


Sigma = 1

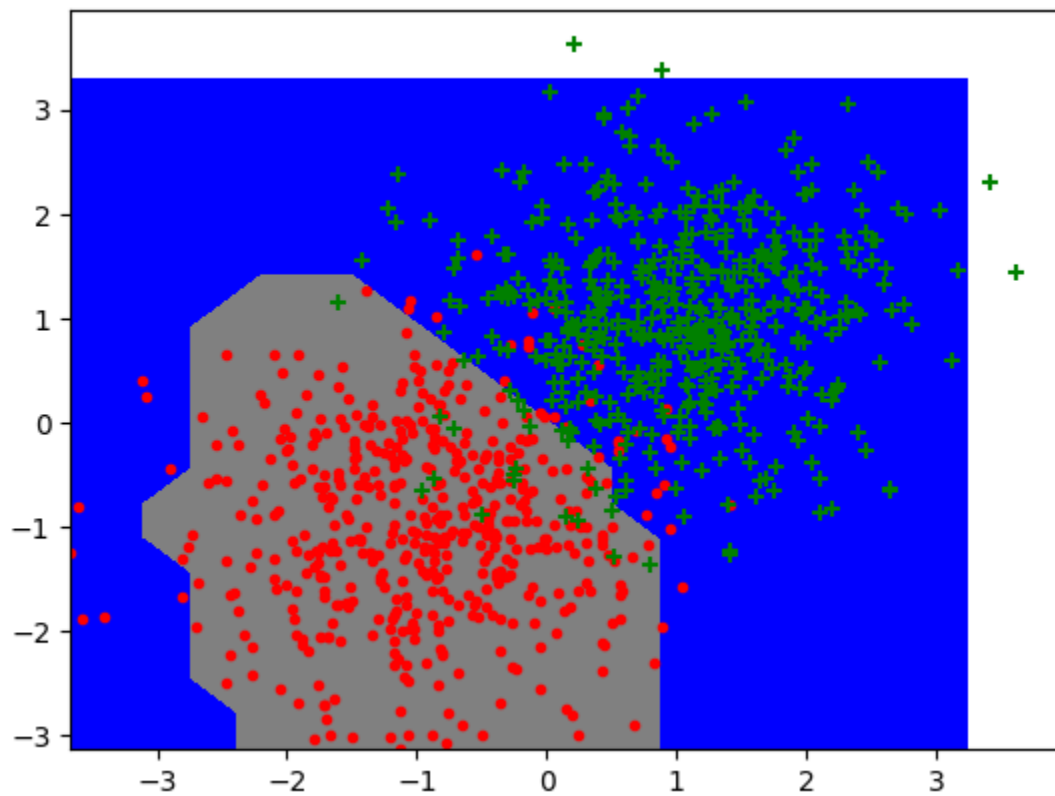


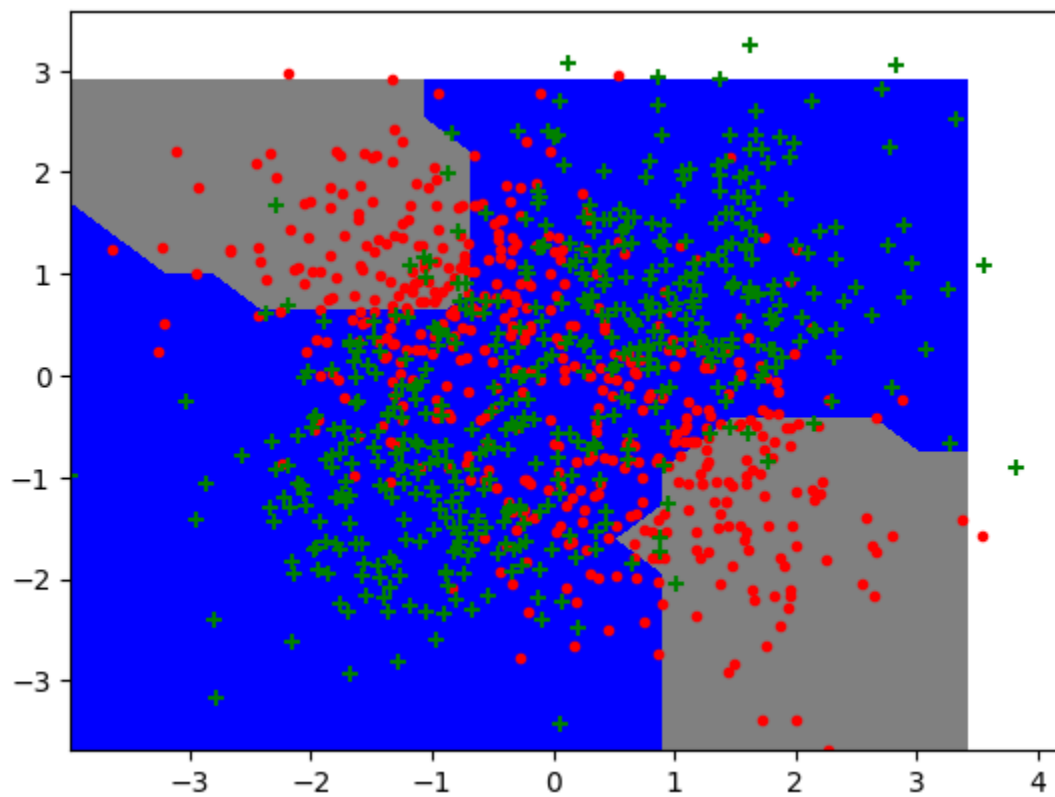
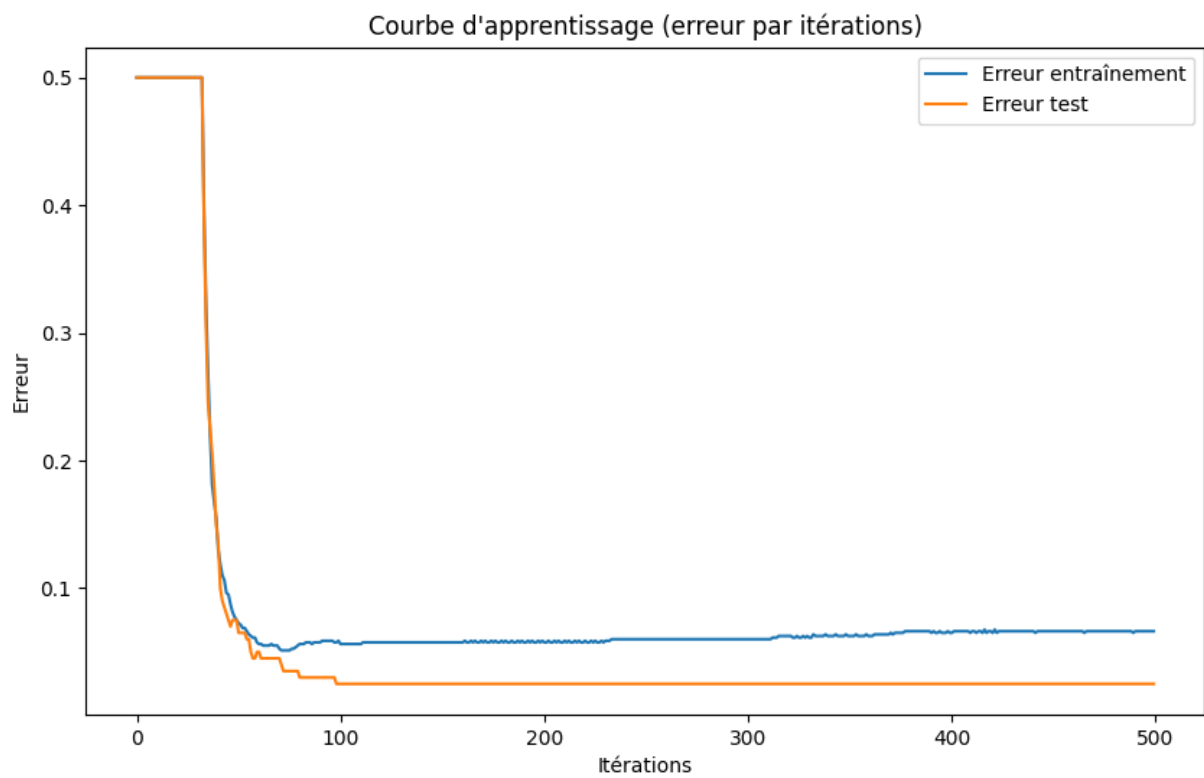


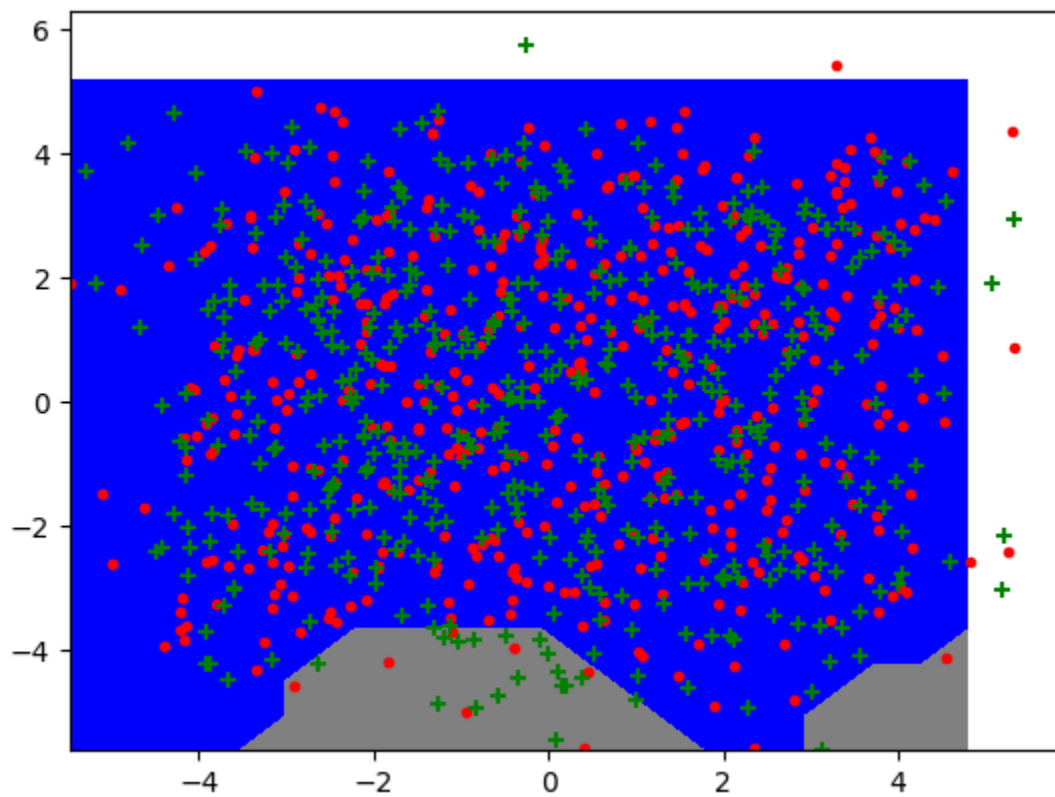
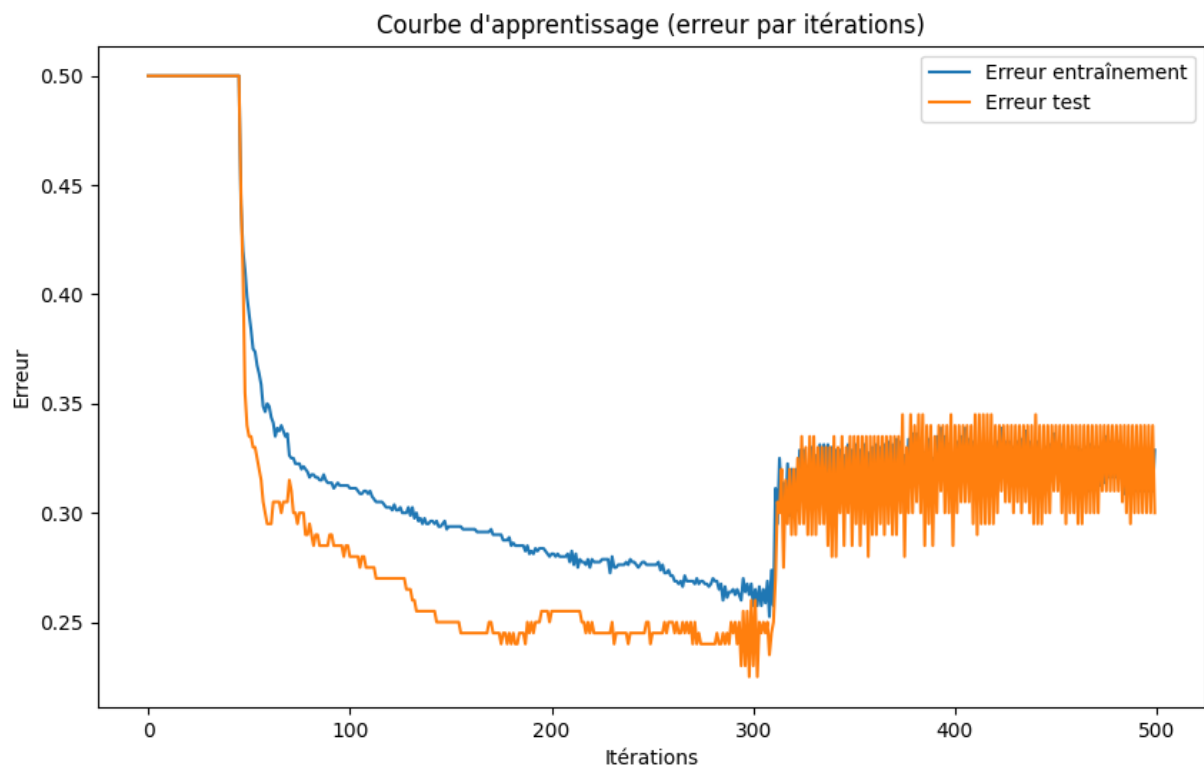


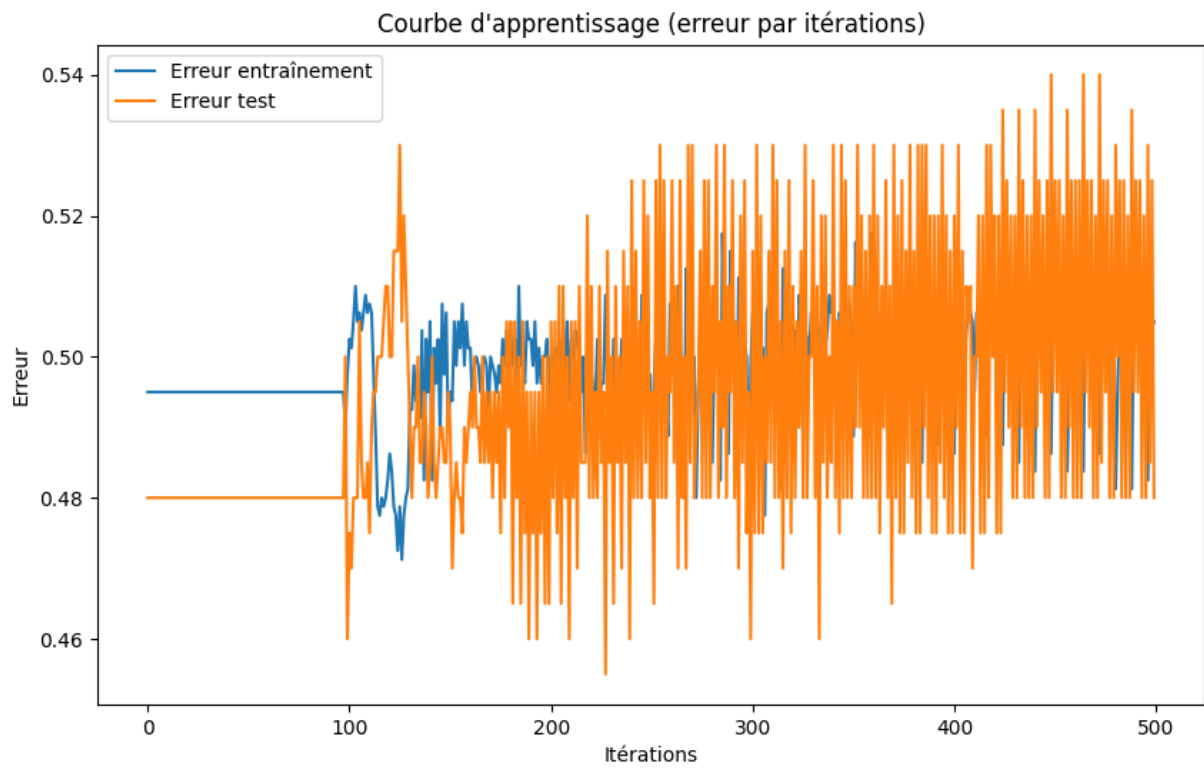


Sigma = 1.5

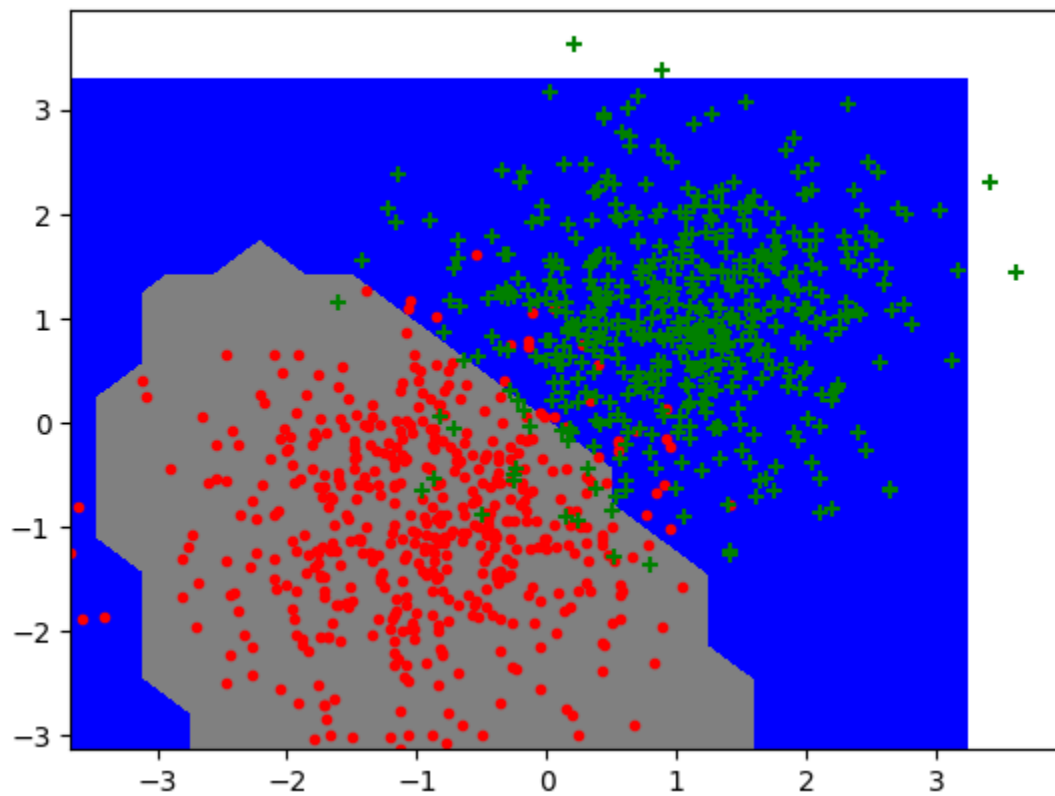


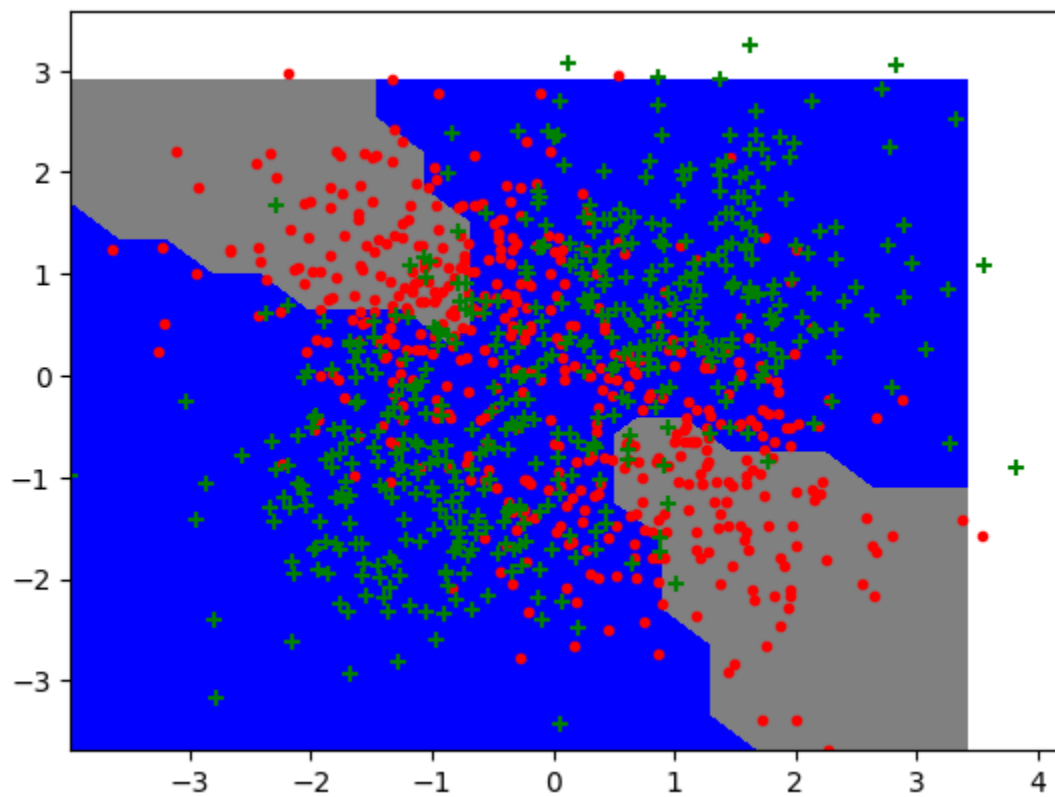
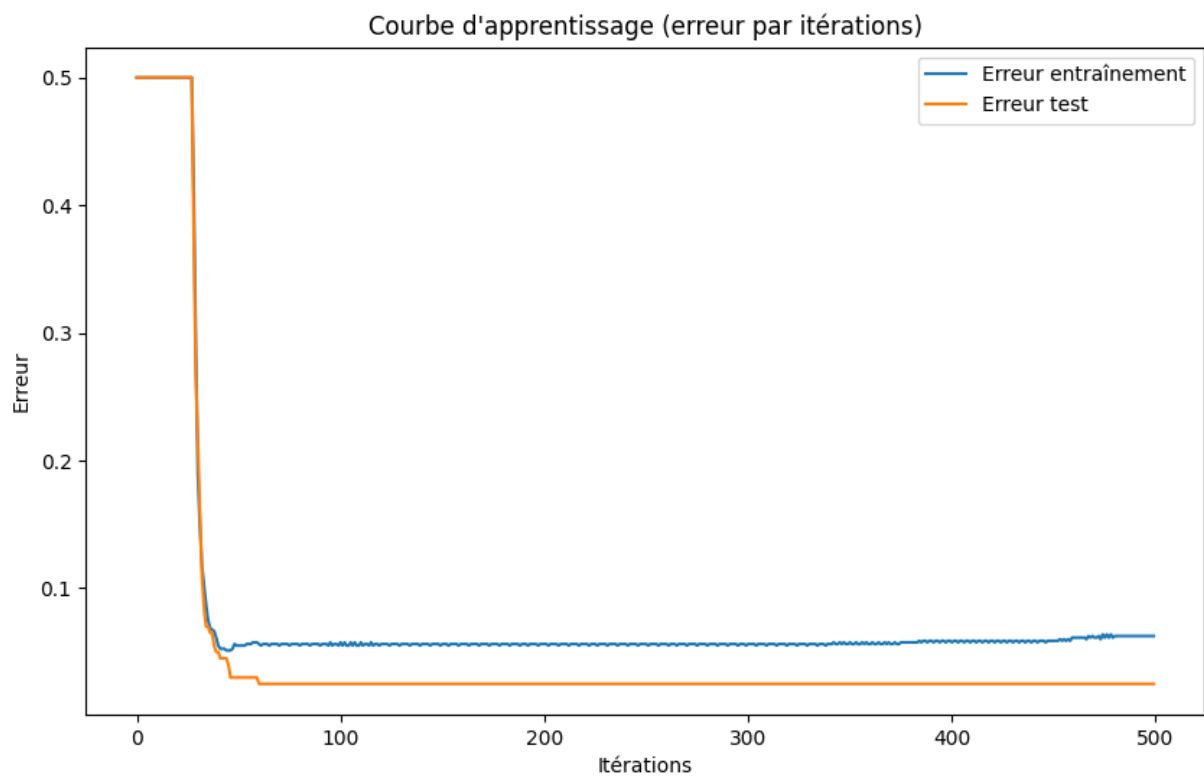


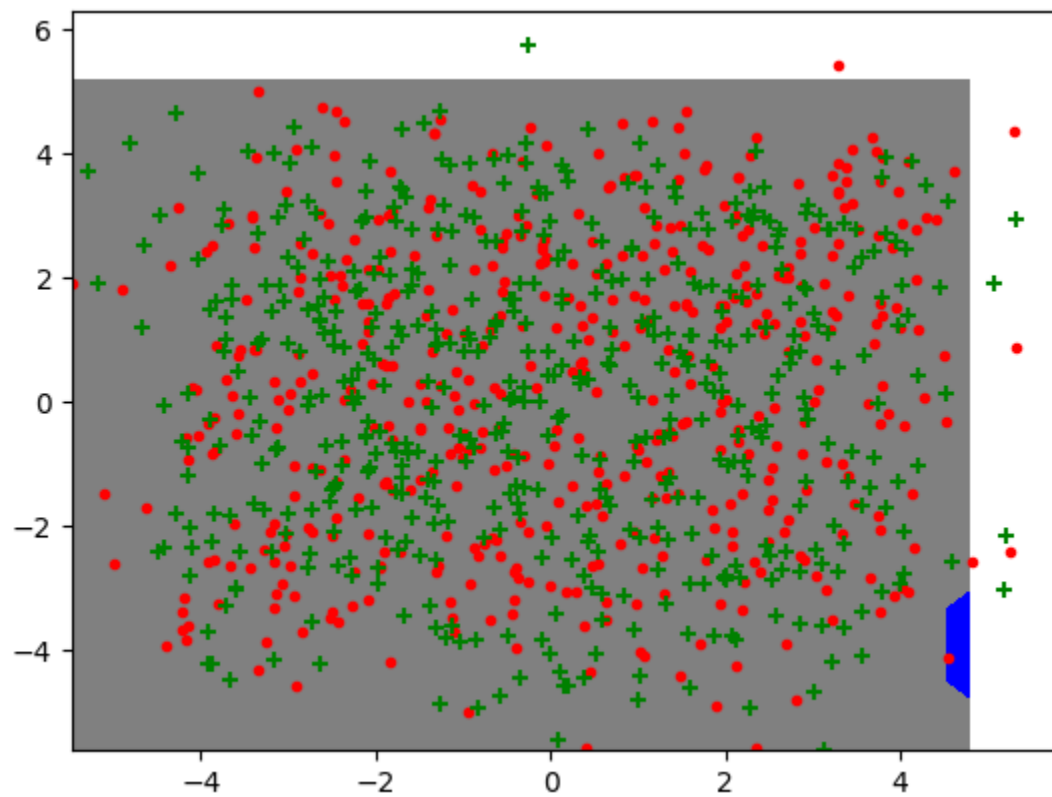
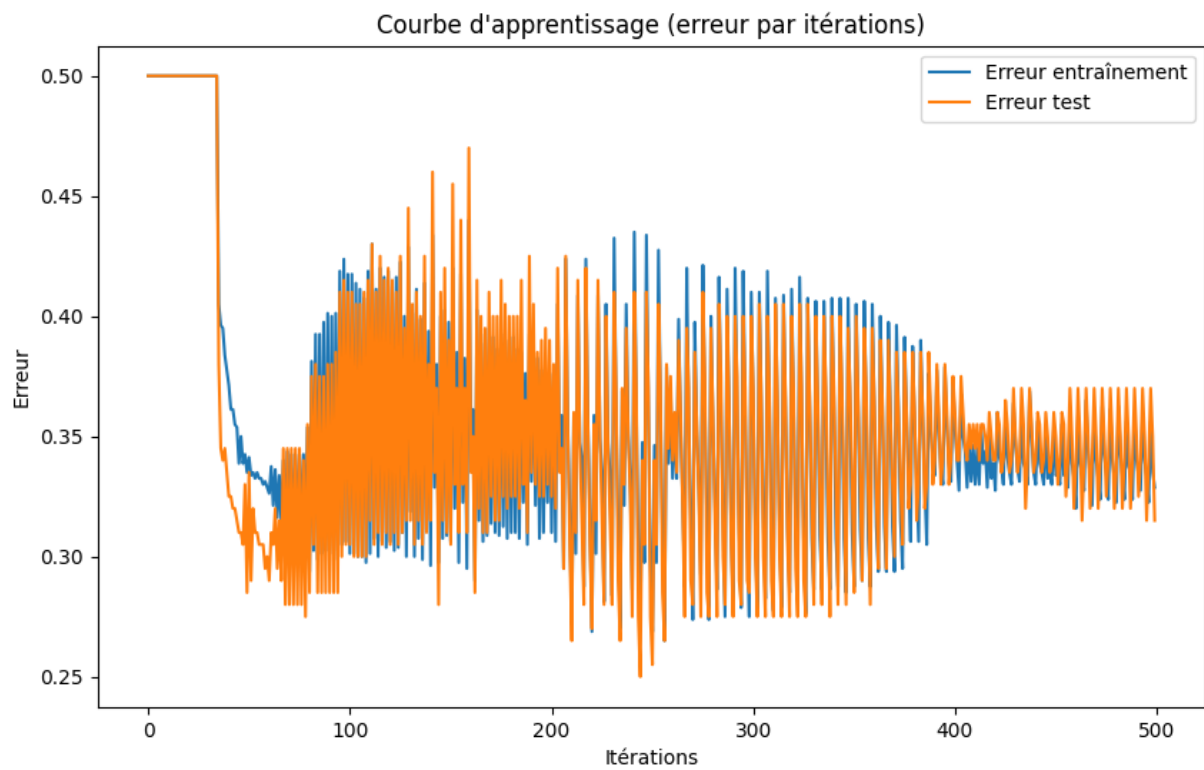


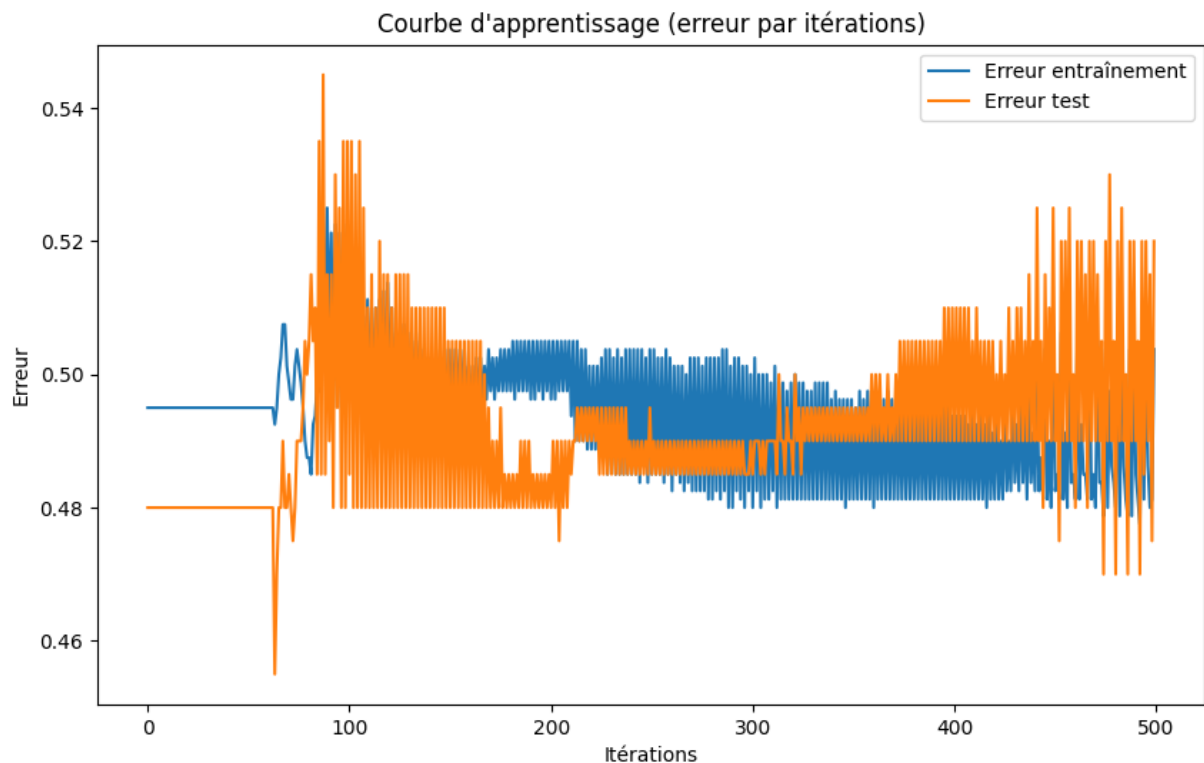


Sigma = 2









Interprétation

La taille de la base de projection ainsi que la taille de sigma sont deux facteurs importants pour la classification.

- Une taille de la base de projection petite est rapide à calculer mais peut ne pas être suffisante pour bien séparer les données. Une grande taille de base de projection capture les informations plus complexes mais plus la taille est grande plus il y a de risque d'overfitting et donc des mauvais résultats de classification. Ici on a testé avec une taille de 10, 100 et 200. La base de taille 10 ne capture pas les informations, avec la base de taille 100, le réseau capture beaucoup mieux les informations avec une taille 200, le réseau commence à faire de l'overfitting.
- La taille de sigma dans une projection gaussienne est un paramètre qui influence la résolution de chaque base gaussienne. Une petite valeur de sigma rend la gaussienne très précise alors qu'une grande valeur de sigma est plus large, donnant un rendu plus lissé. Le sigma optimal semble être celui d'une valeur de 1.

c) Hinge Loss

```
In [ ]: def hinge_loss(w, X, y, alpha=1.0, lambd=0.1):
    margins = alpha - y * (X @ w)
    hinge = np.maximum(0, margins)
    loss = np.sum(hinge) + lambd * np.linalg.norm(w) ** 2
    return loss
```

```
In [190]: def hinge_loss_grad(w, X, y, alpha=1.0, lambd=0.1):
```

```

margins = alpha - y * (X @ w)
indicator = (margins > 0).astype(float)
grad = -np.sum((indicator * y)[:, np.newaxis] * X, axis=0)
grad += 2 * lambda * w
return grad

```

```

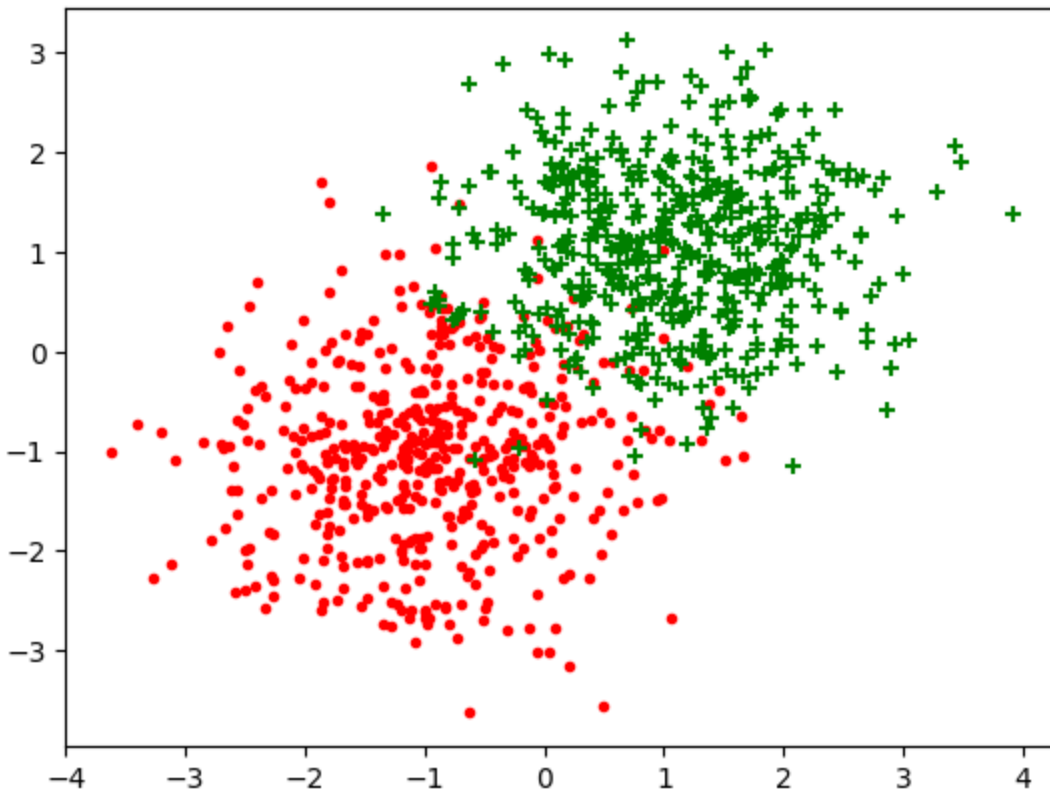
In [194... sigma = 1
base_size = 100
index_b = np.random.choice(np.arange(1, datax.shape[0]), base_size)
base = datax[index_b]

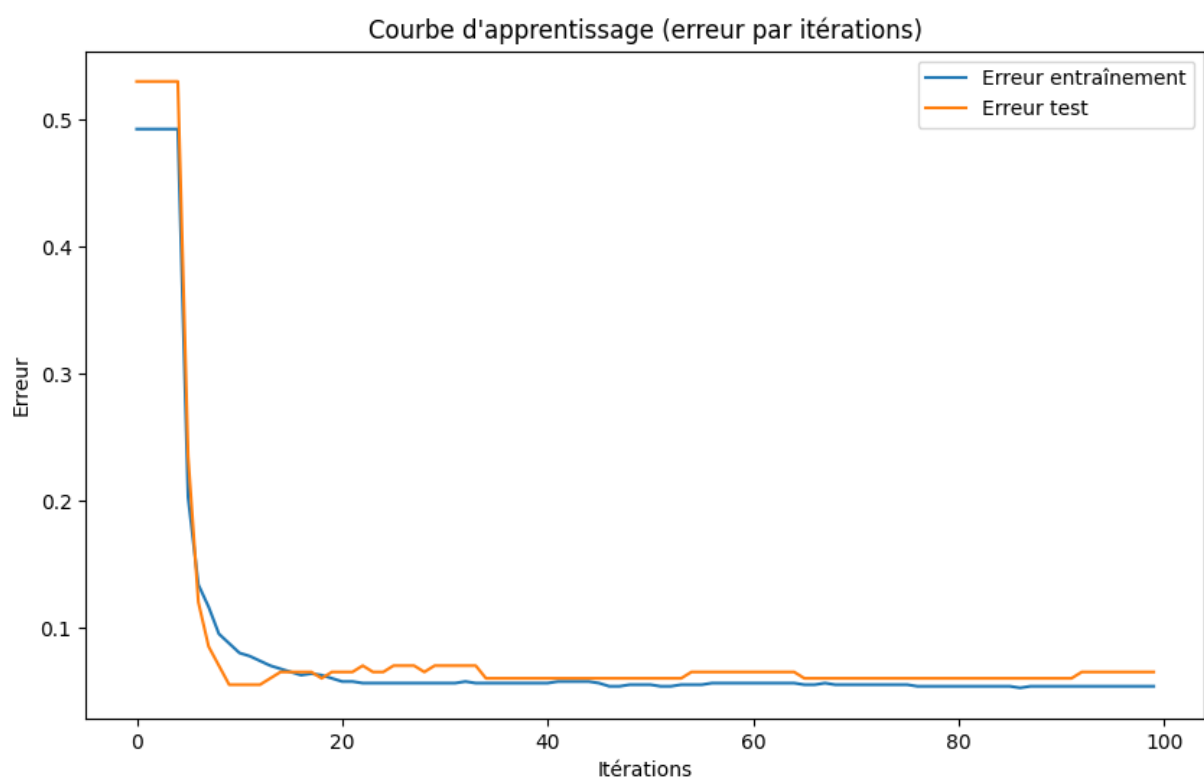
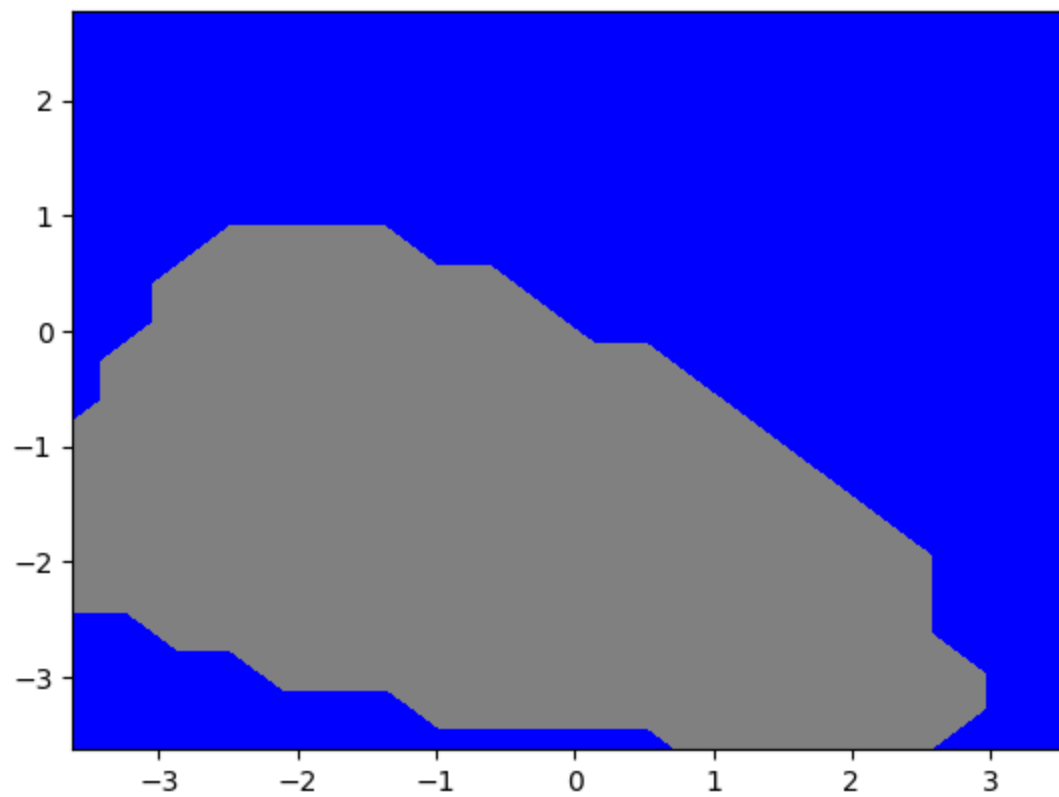
# apprentissage data 0
for alpha in [0.5, 1, 1.5] : #taille de la marge
    for lambda in [1e-2, 0.1, 1] :
        print(f"alpha = {alpha}, lambda = {lambda}")
        gauss0 = Lineaire3(loss=partial(hinge_loss, alpha=alpha, lambda=lambda)
        gauss0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())

        plot_data(x0, y0)
        plt.show()
        plot_frontiere(x0, gauss0.predict)
        display_error(gauss0.train_errors, gauss0.test_errors)

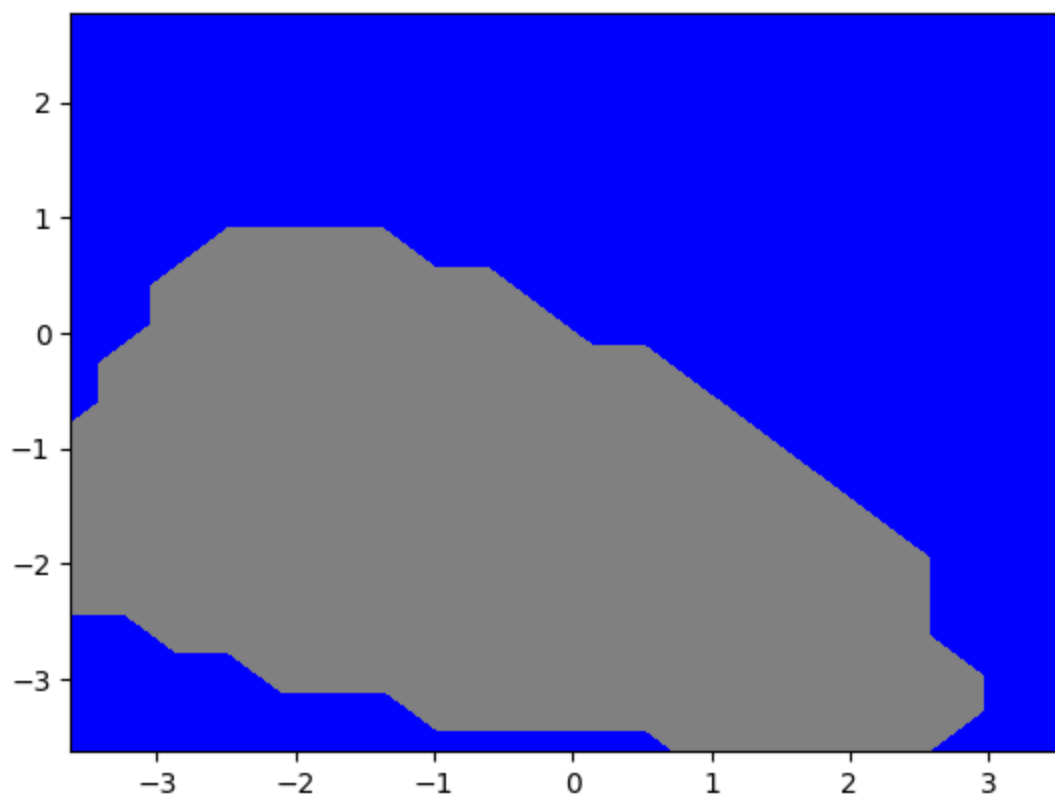
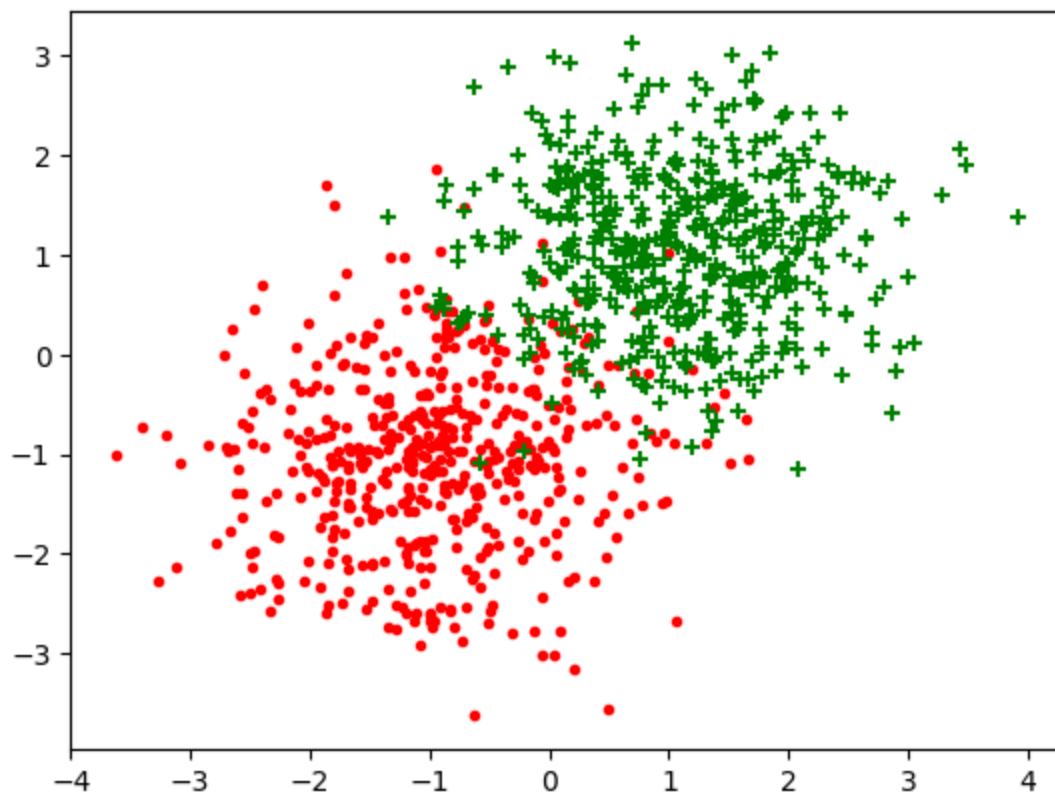
```

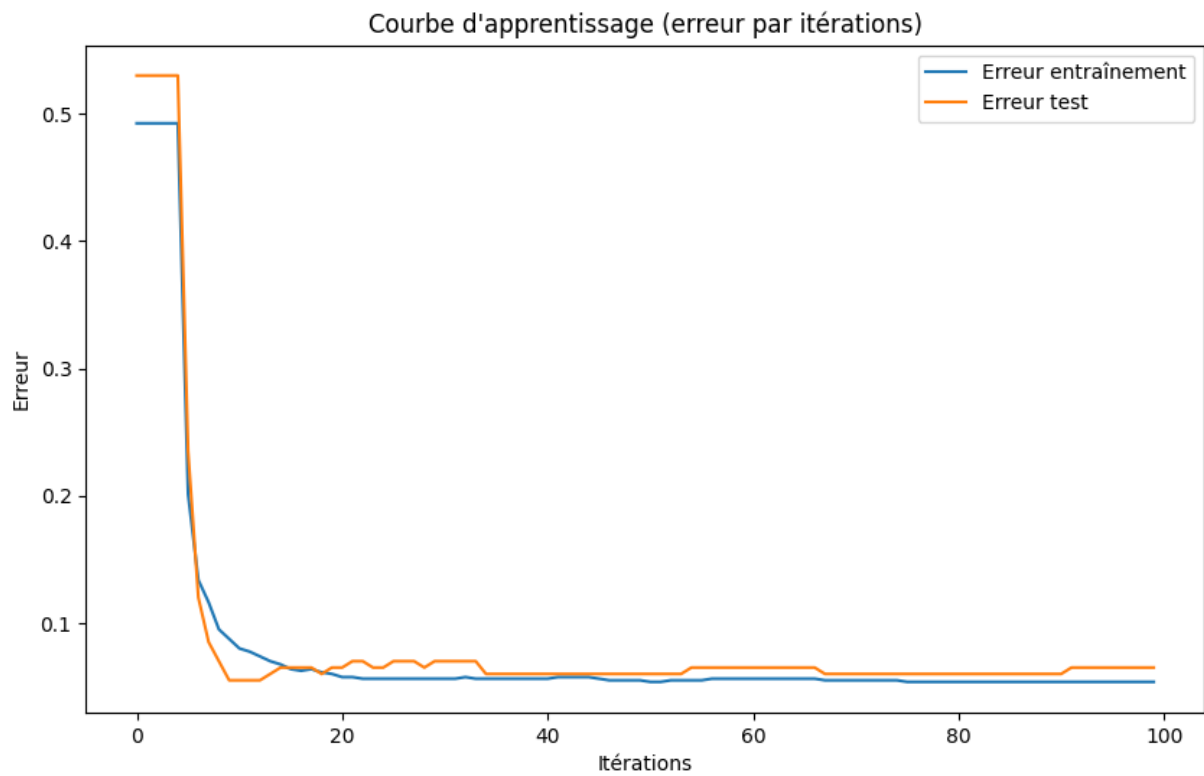
alpha = 0.5, lambda = 0.01



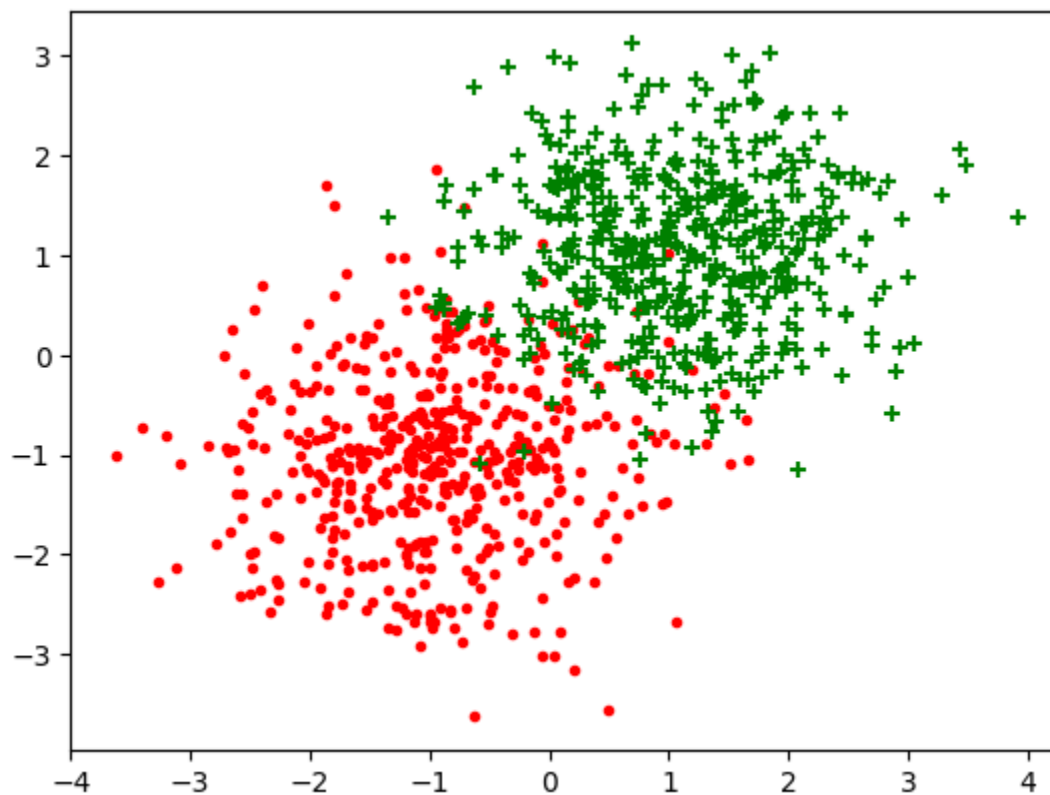


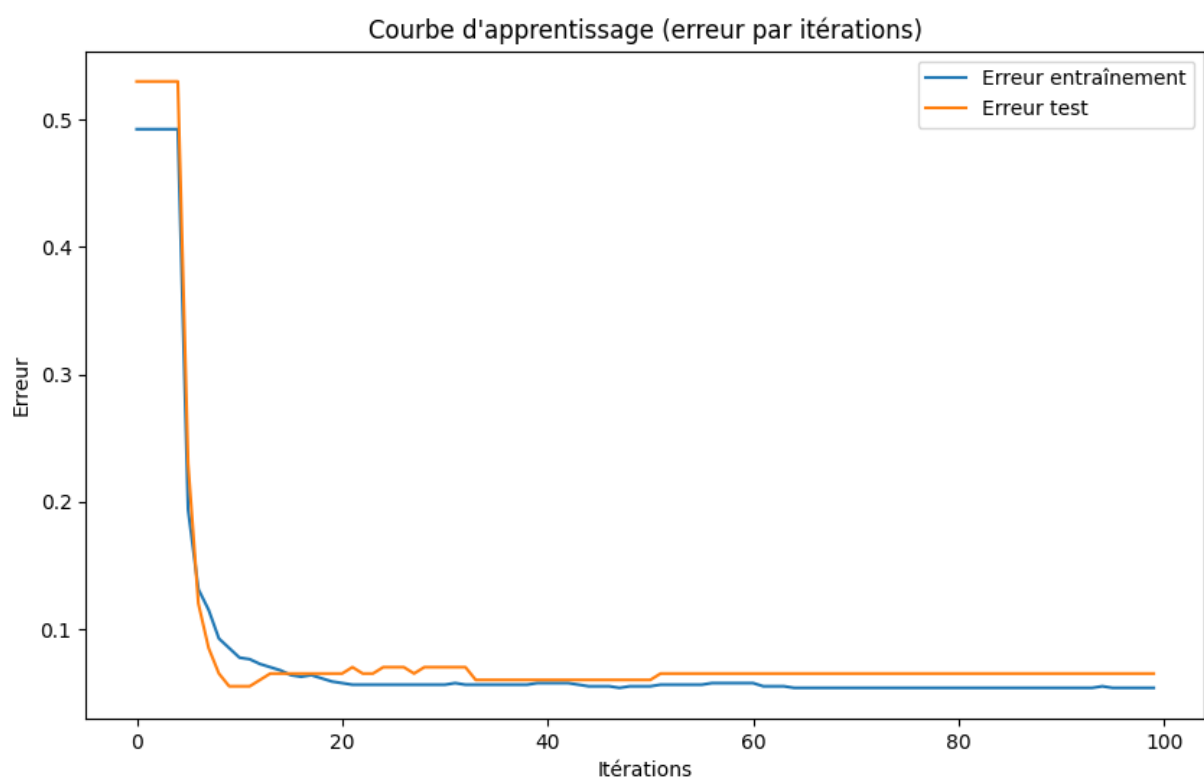
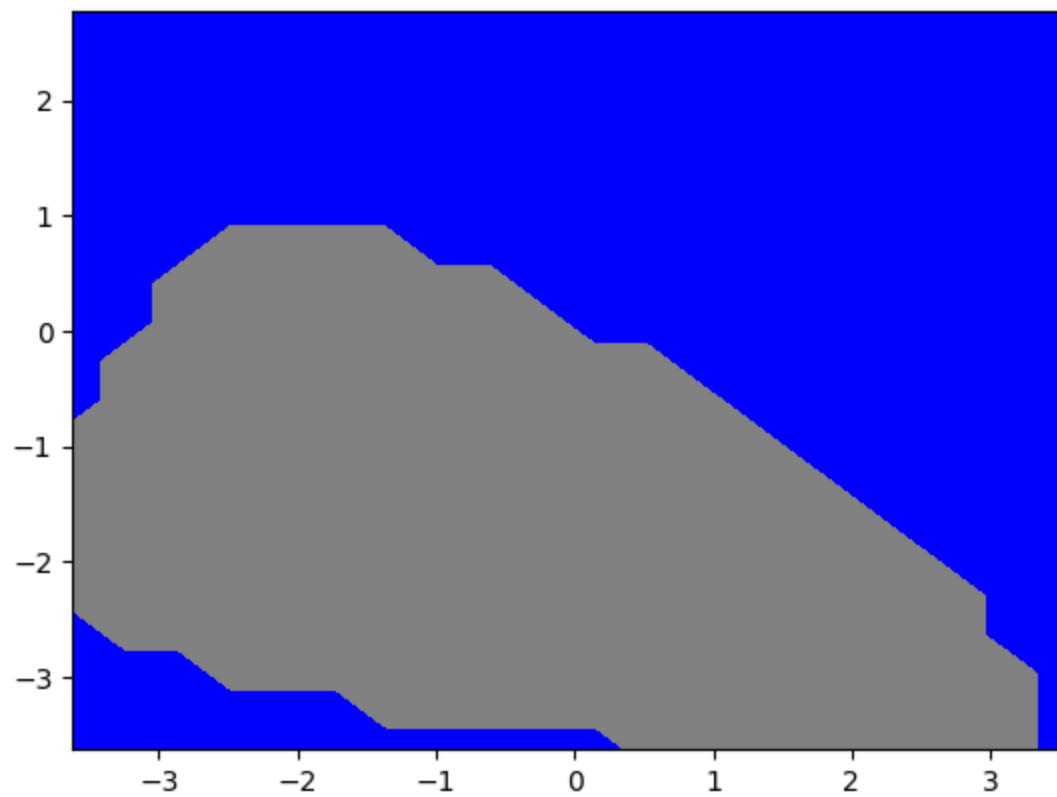
$\alpha = 0.5$, $\lambda = 0.1$



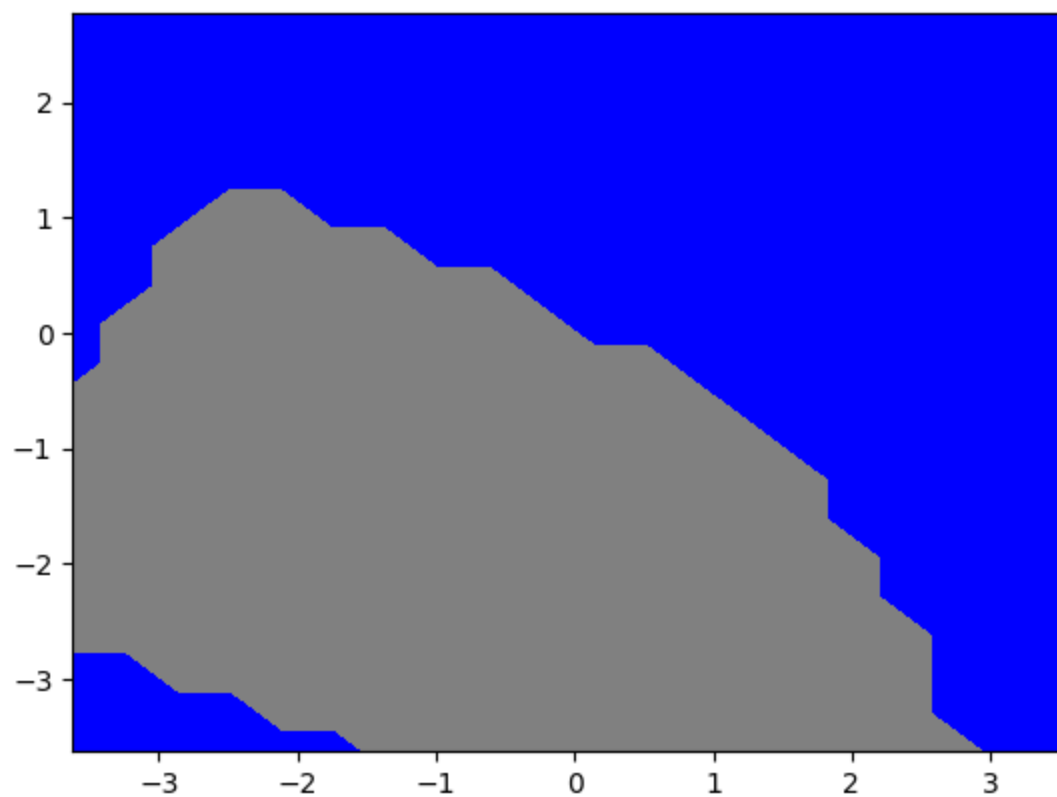
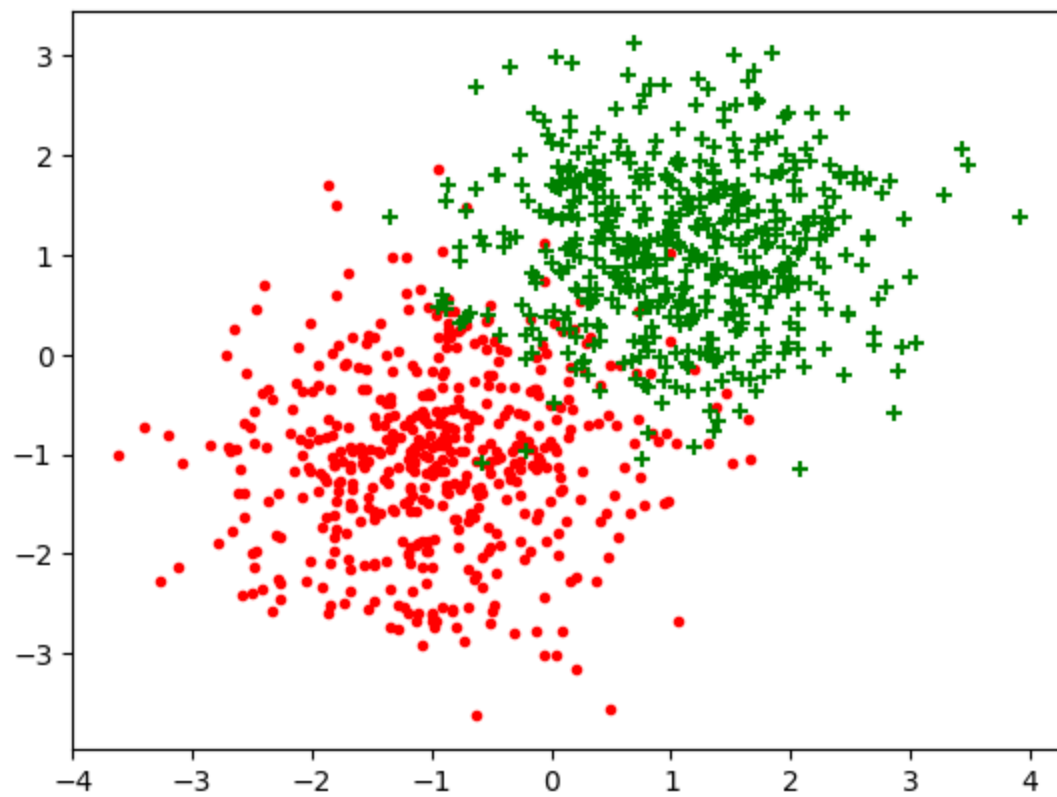


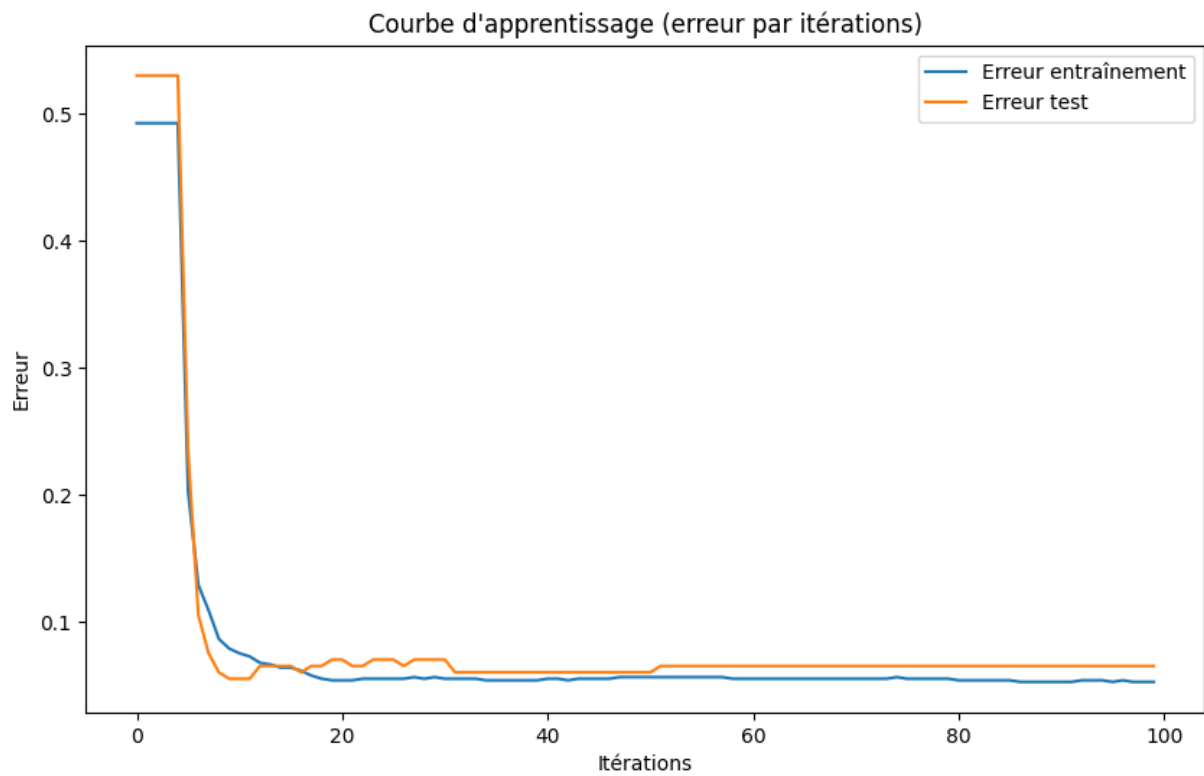
$\alpha = 0.5$, $\lambda = 1$



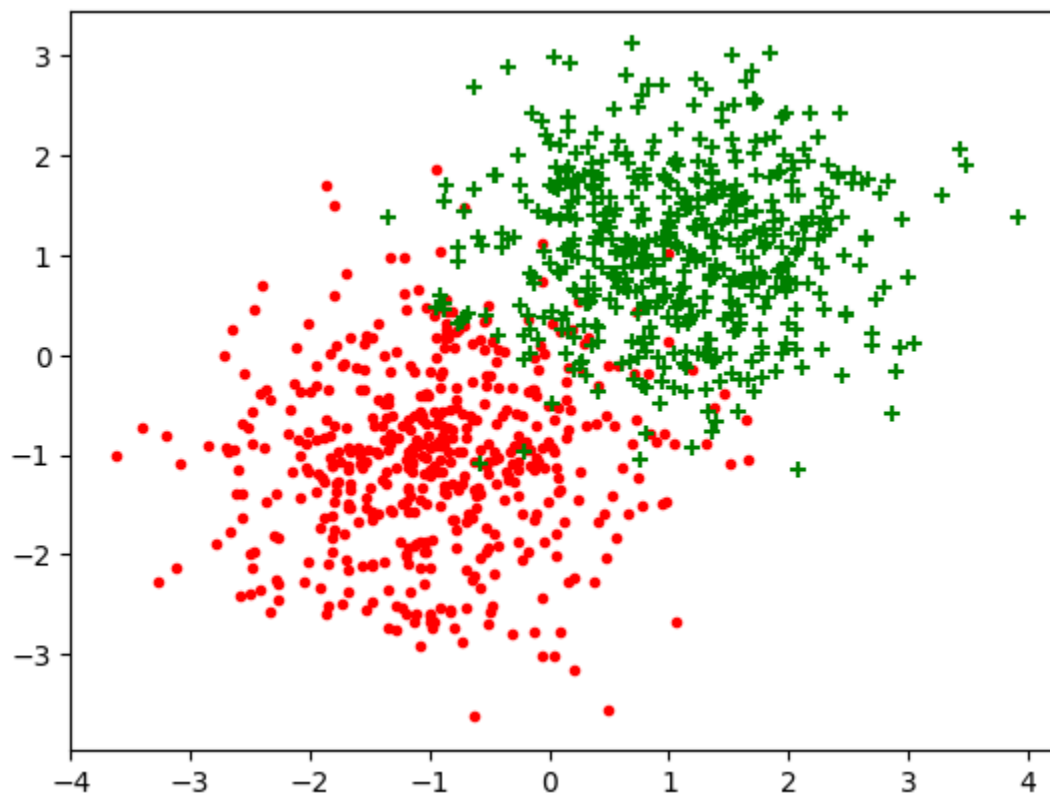


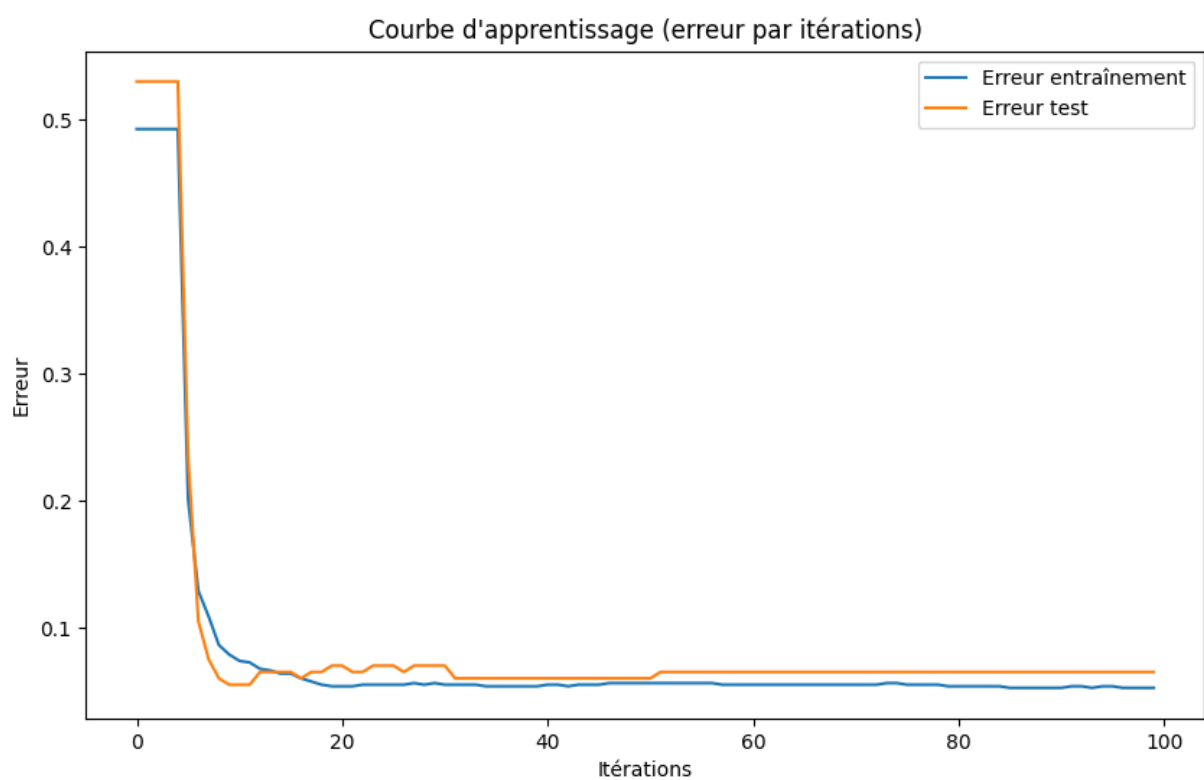
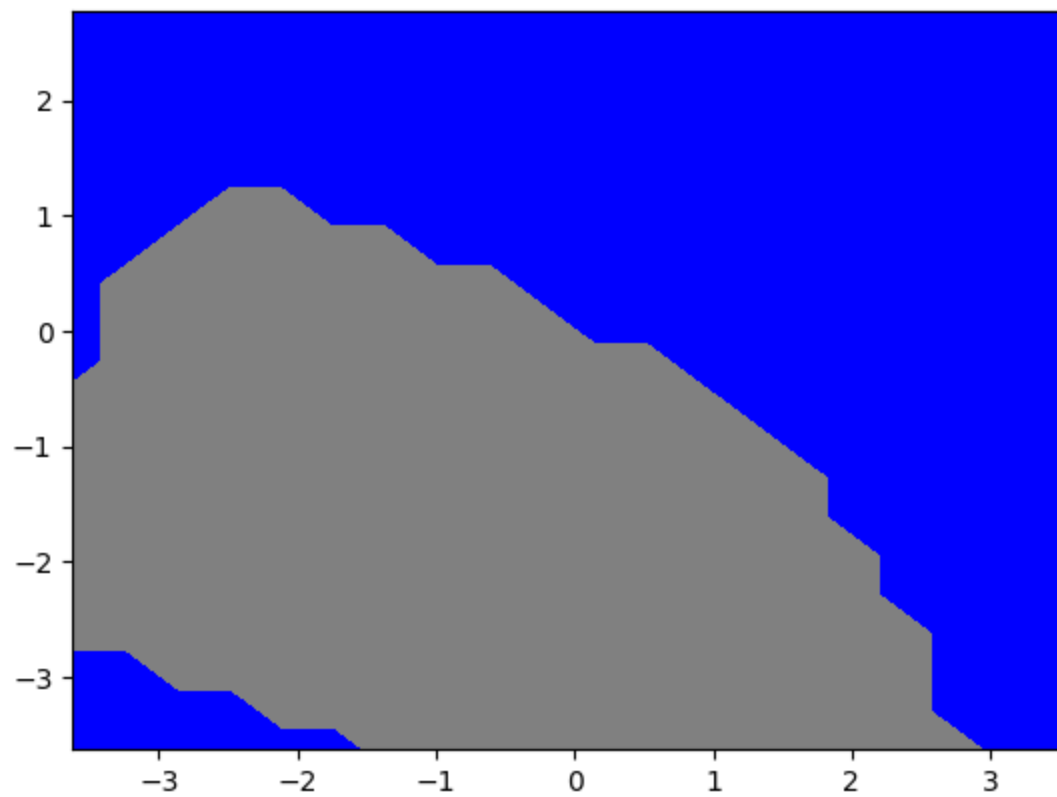
$\alpha = 1$, $\lambda = 0.01$



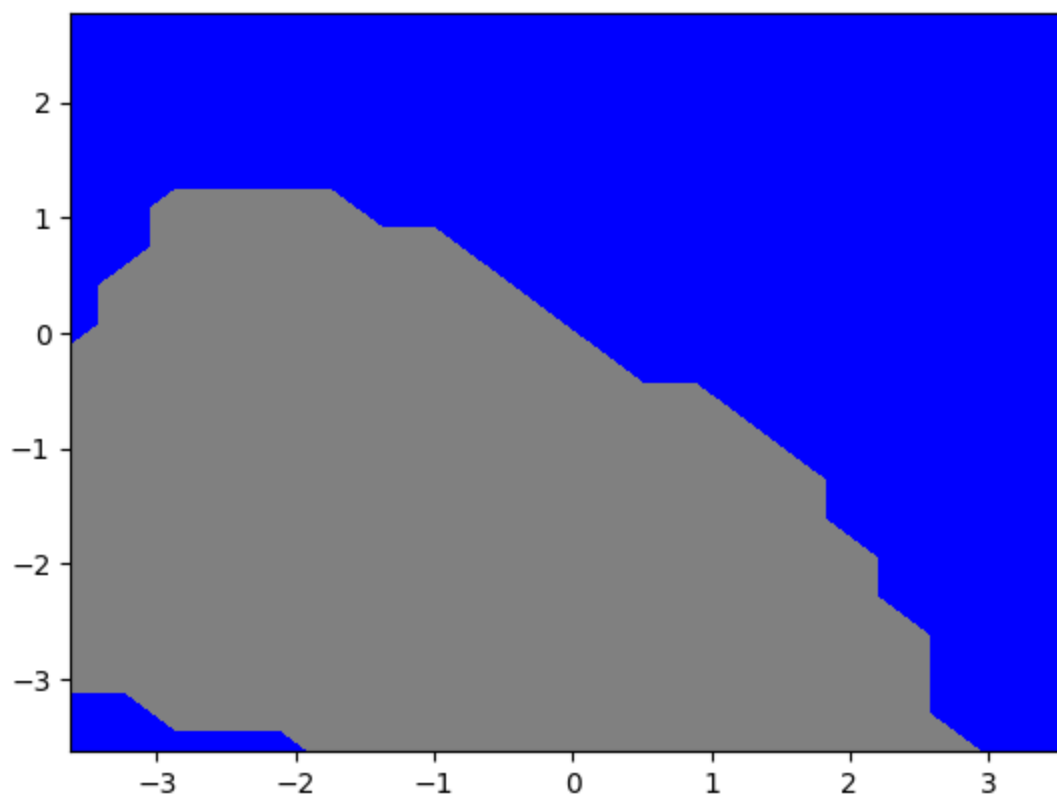
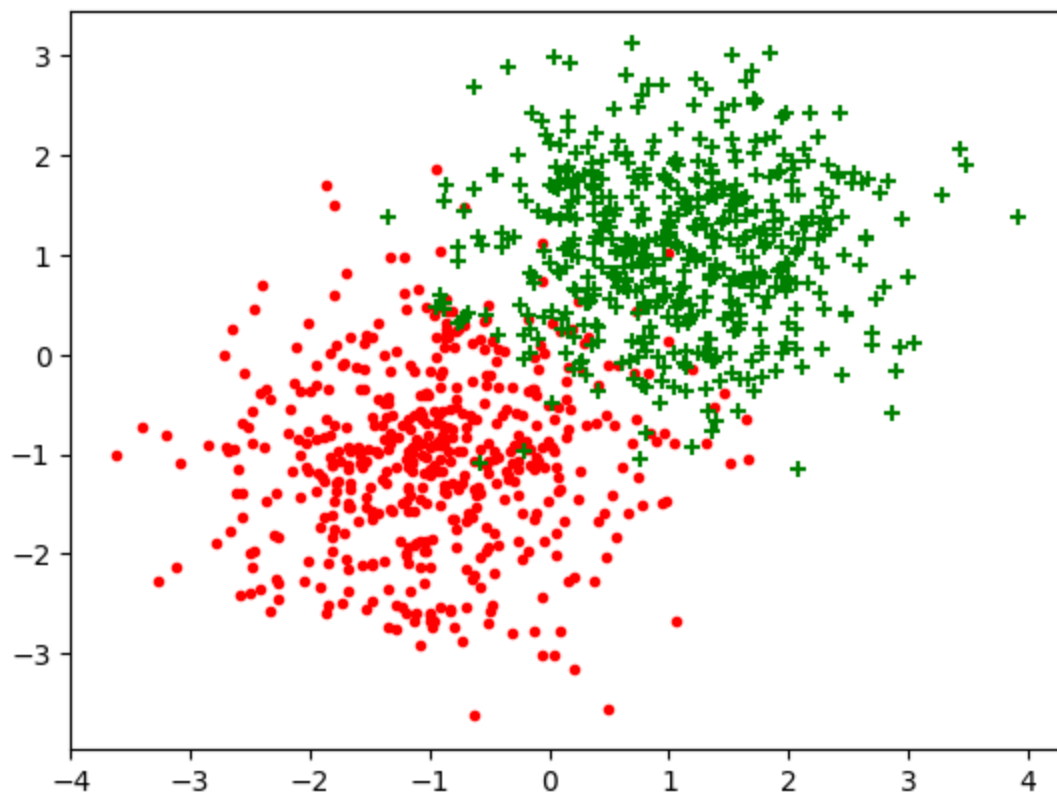


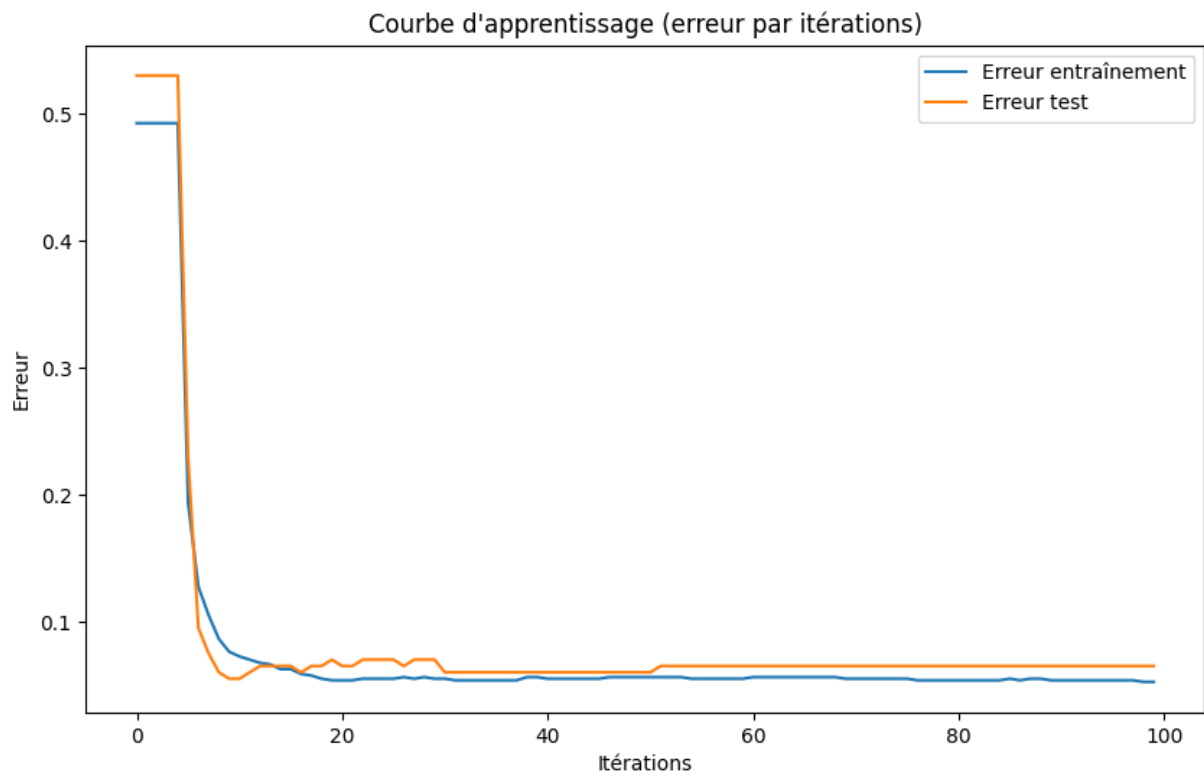
$\alpha = 1$, $\lambda = 0.1$



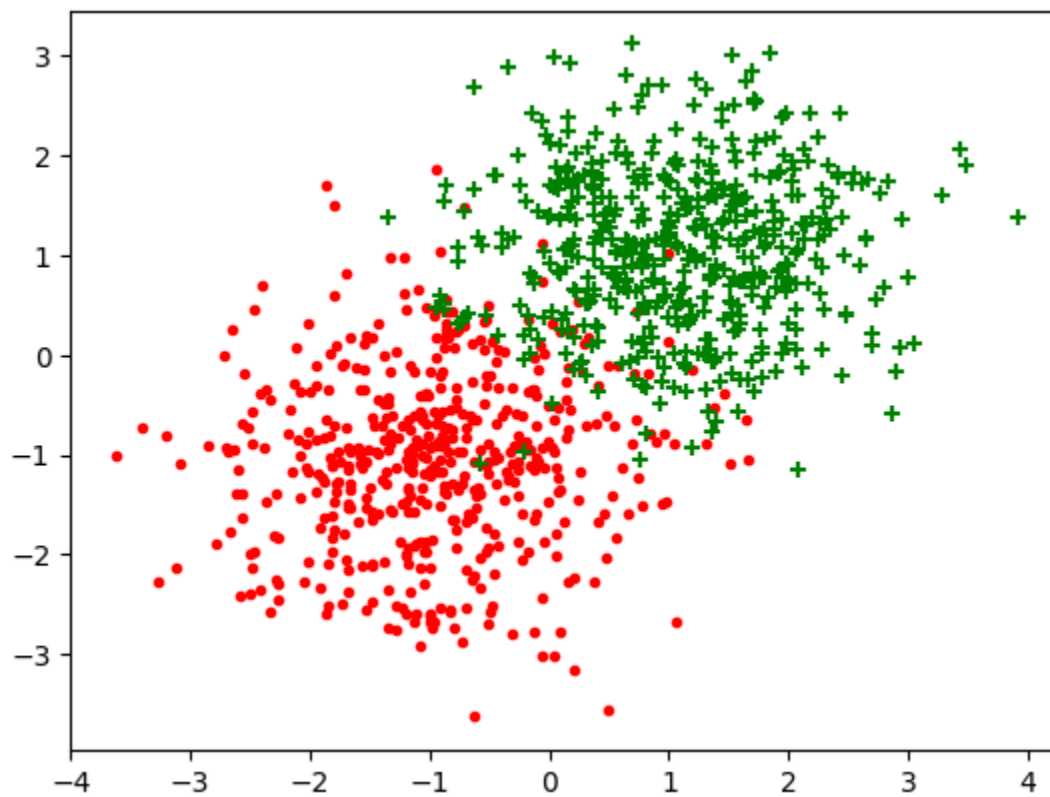


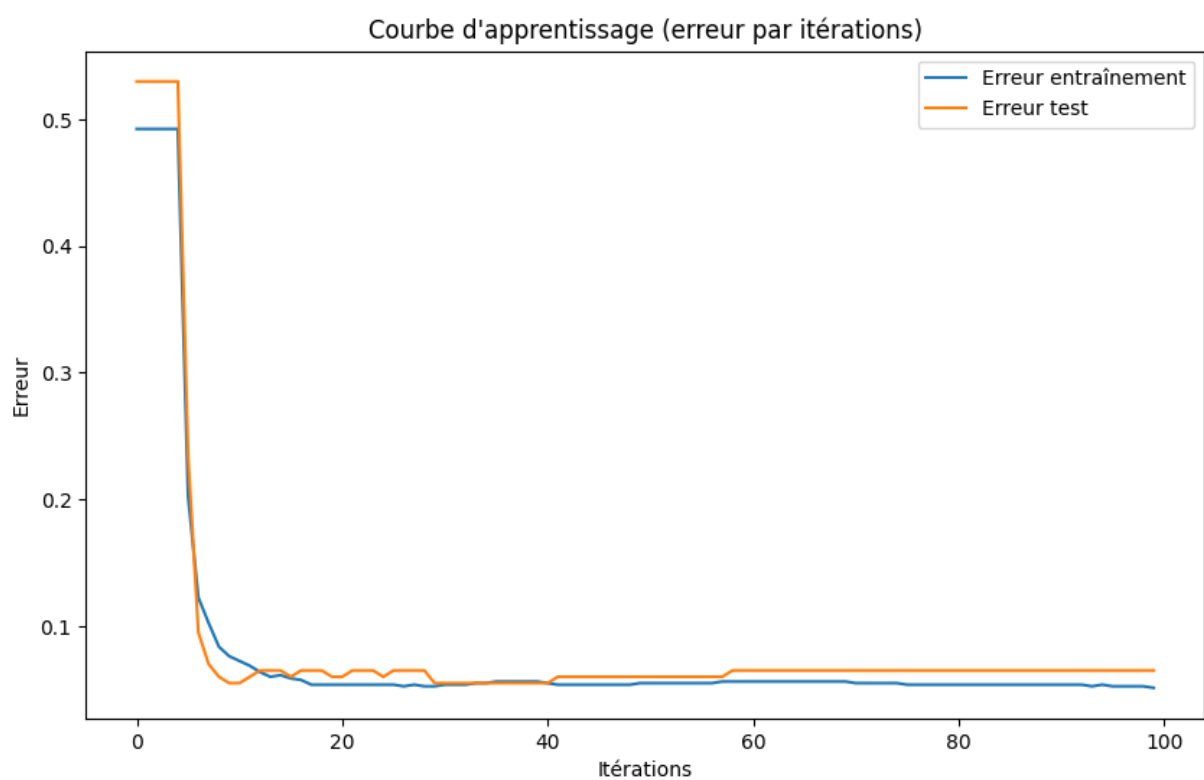
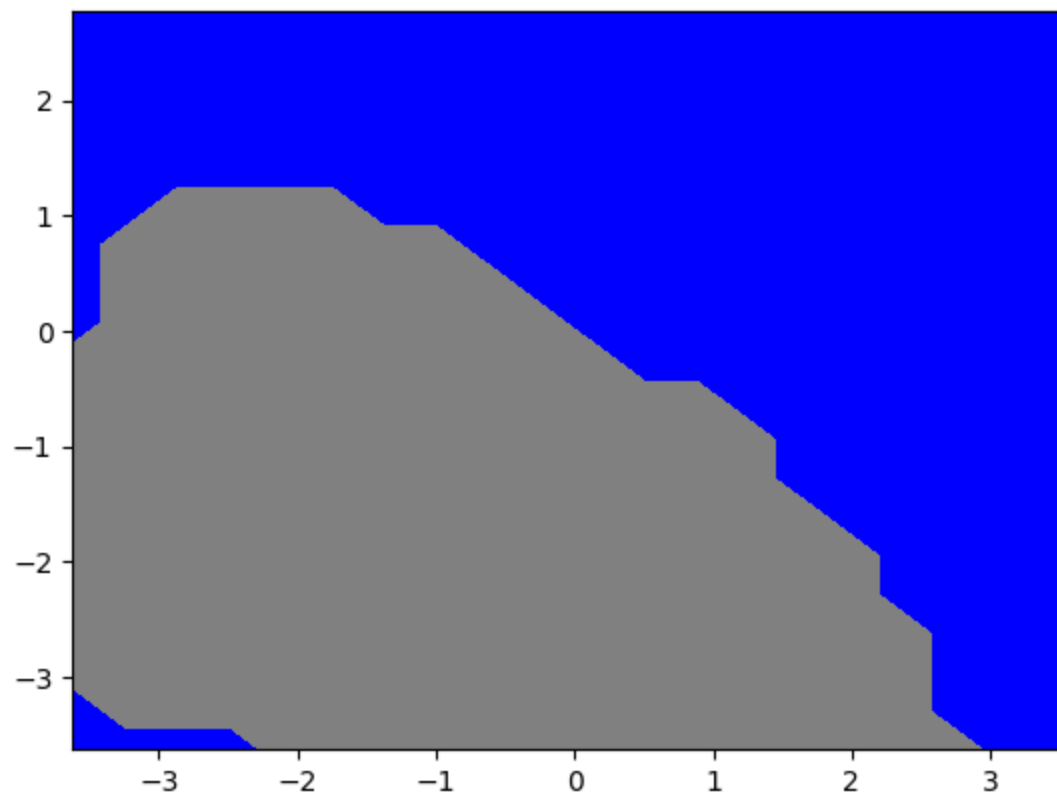
$\alpha = 1$, $\lambda = 1$



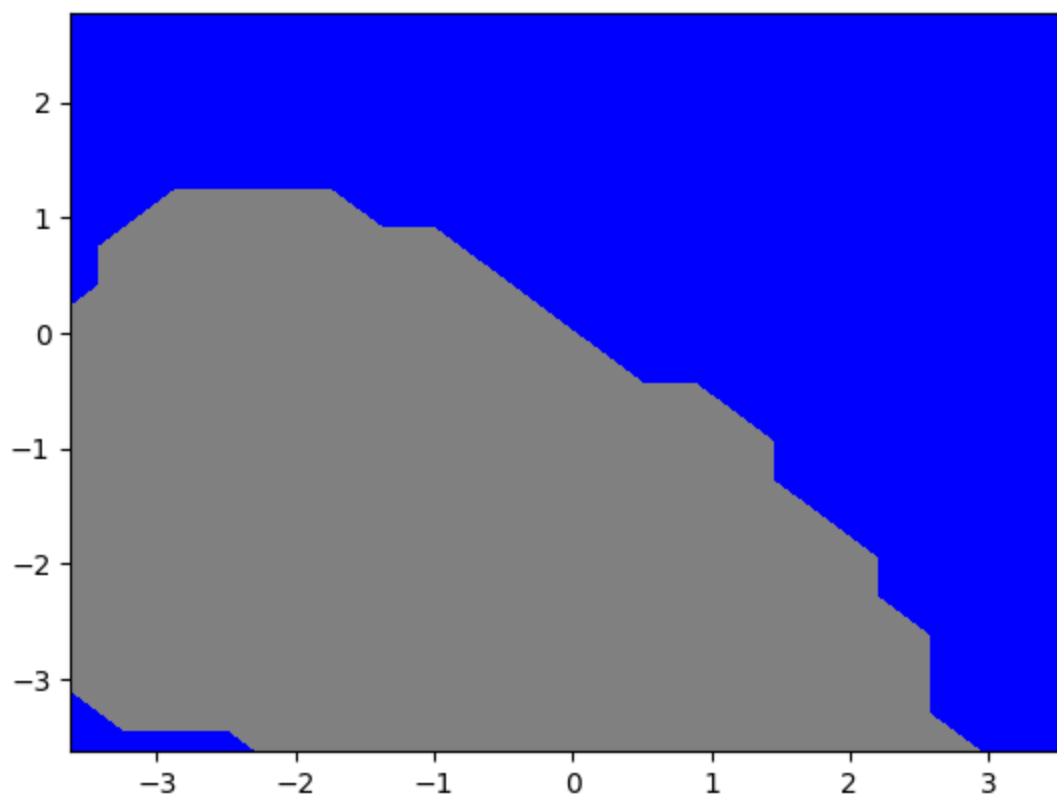
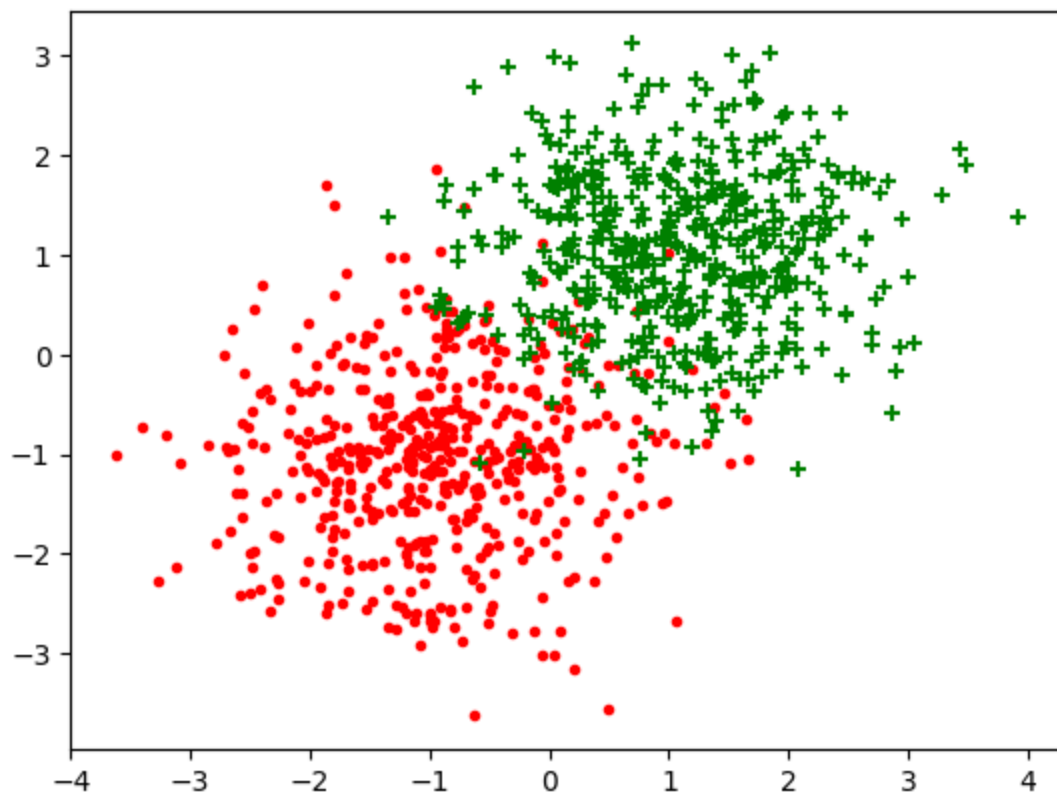


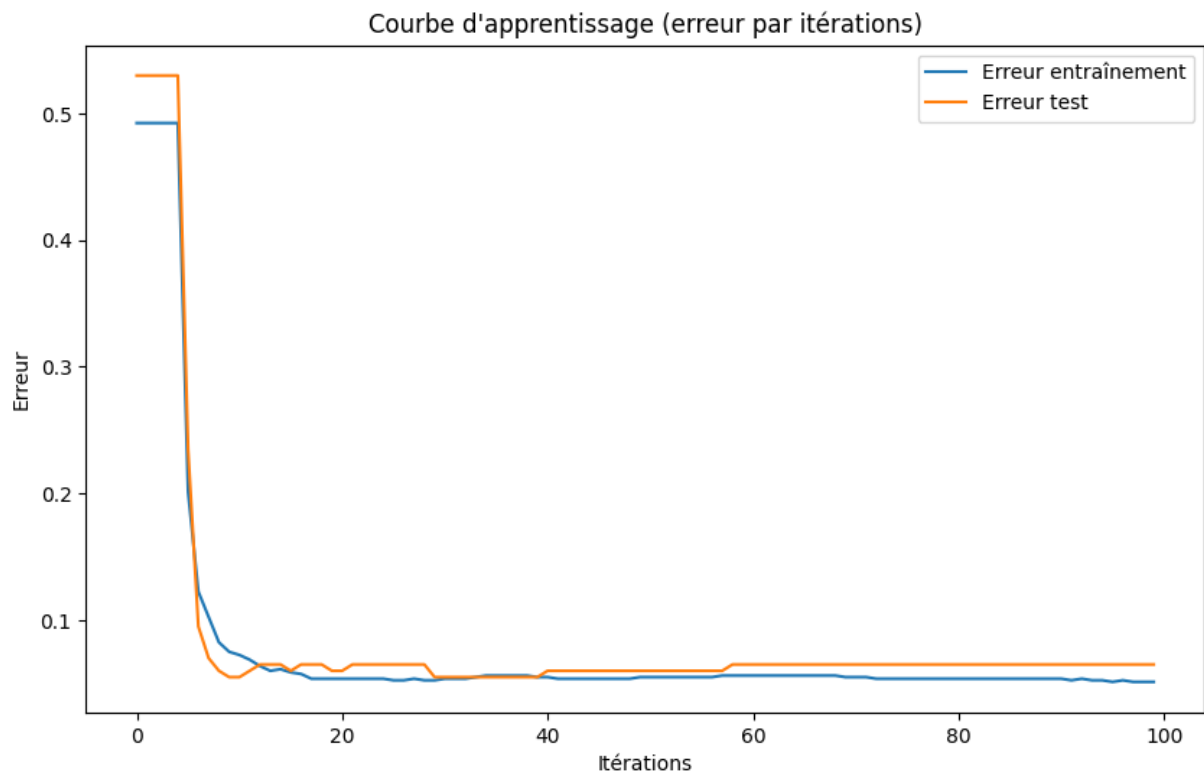
$\alpha = 1.5$, $\lambda = 0.01$



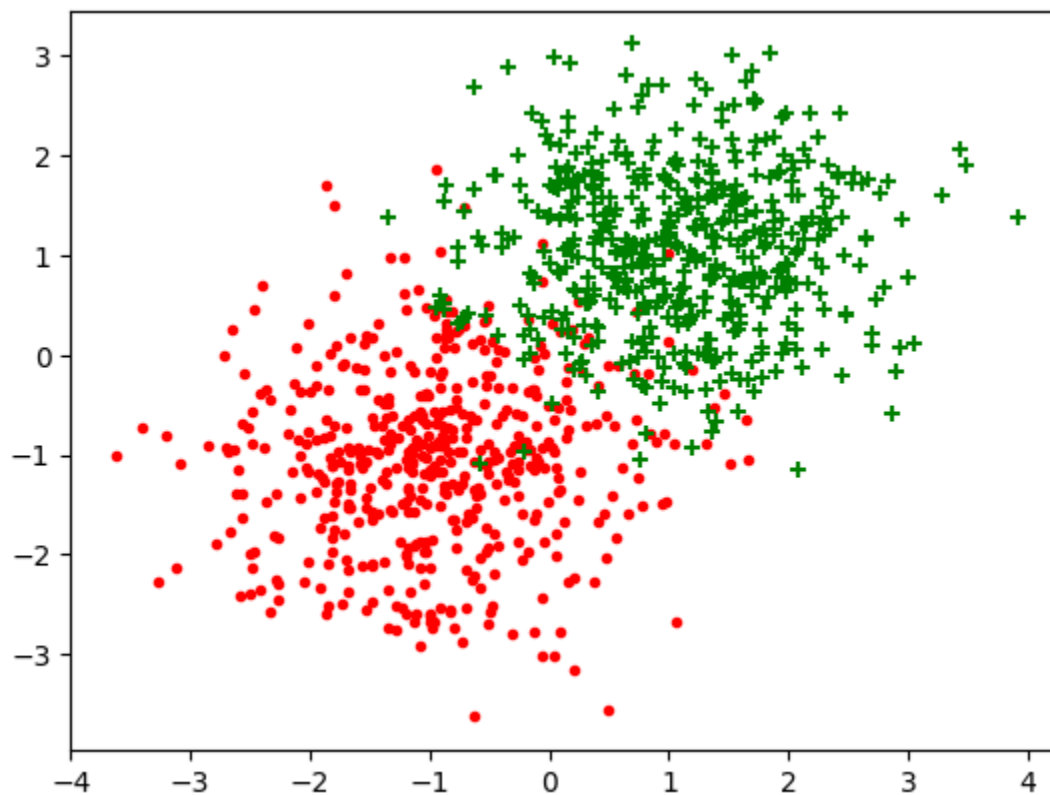


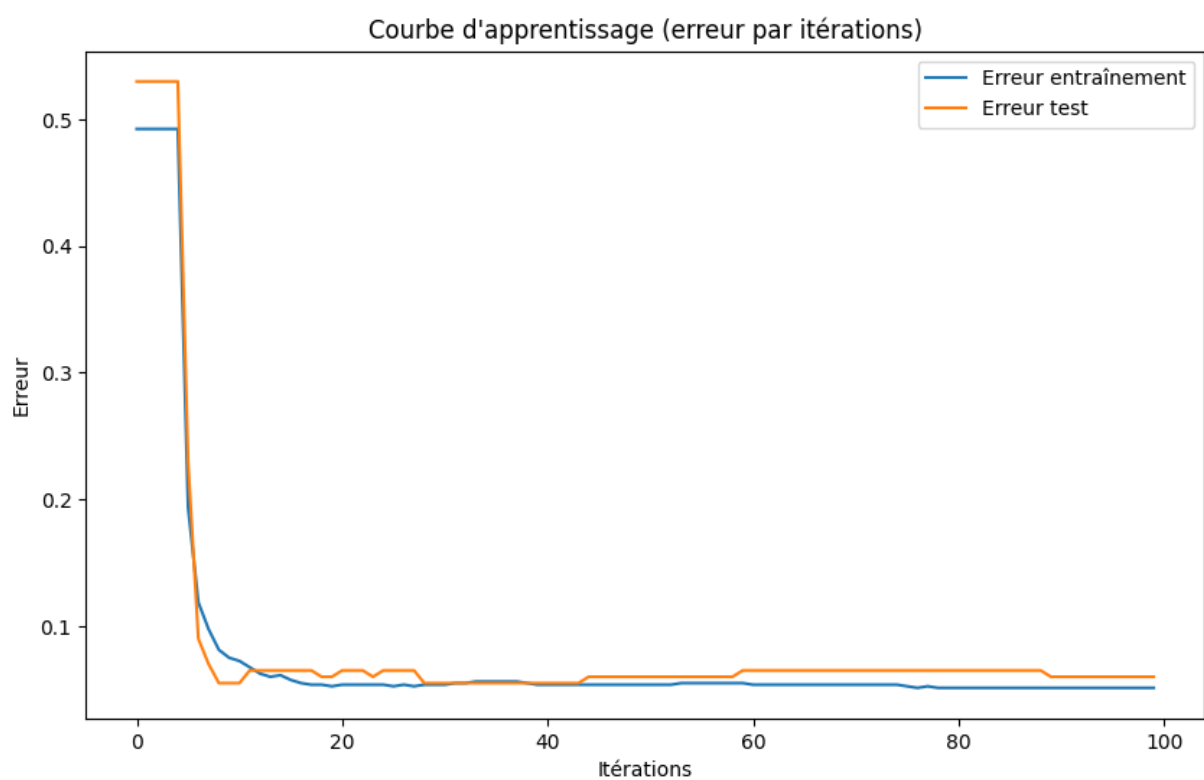
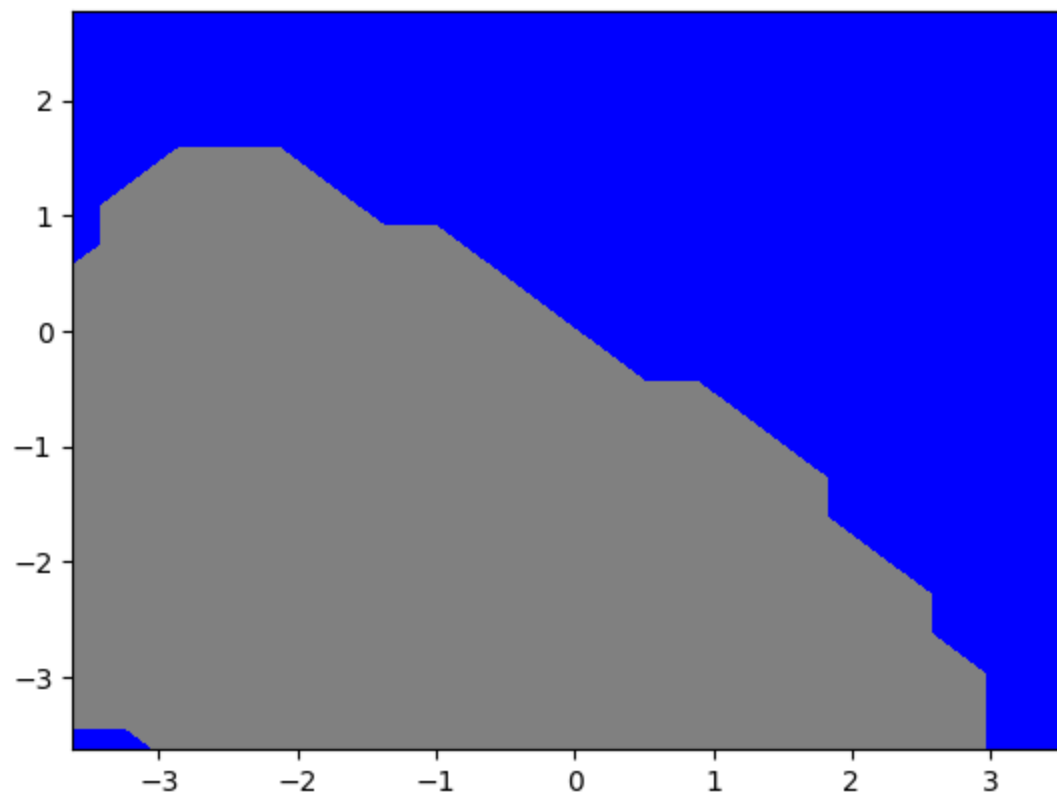
$\alpha = 1.5$, $\lambda = 0.1$





$\alpha = 1.5$, $\lambda = 1$





Interprétation

- Plus les valeurs de α sont grandes, plus la marge est grande et moins le système est permissif. Ainsi on obtient une classification plus strictes avec un α plus grand.
- Plus λ est grand plus le modèle est régularisé donc moins le modèle est complexe

(limite l'overfitting). A l'inverse si λ est trop faible alors le modèle est plus complexe mais avec un plus fort risque d'overfitting.

- Le lien avec SVM réside dans la mise en place d'une marge entre les données et la frontière de décision avec un certain coefficient de permisivité. Par ailleurs, SVM utilise la fonction de coût hinge loss et son gradient associé.

IV - SVM et grid search

```
In [2]: from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import Perceptron
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
```

IV.1) Comparaison sklearn et les modèles mis en place dans les tme précédents

a) Récupérer les données

```
In [16]: x, y = gen_arti(epsilon=0.8)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, ran
y_train, y_test = y_train.ravel(), y_test.ravel())
```

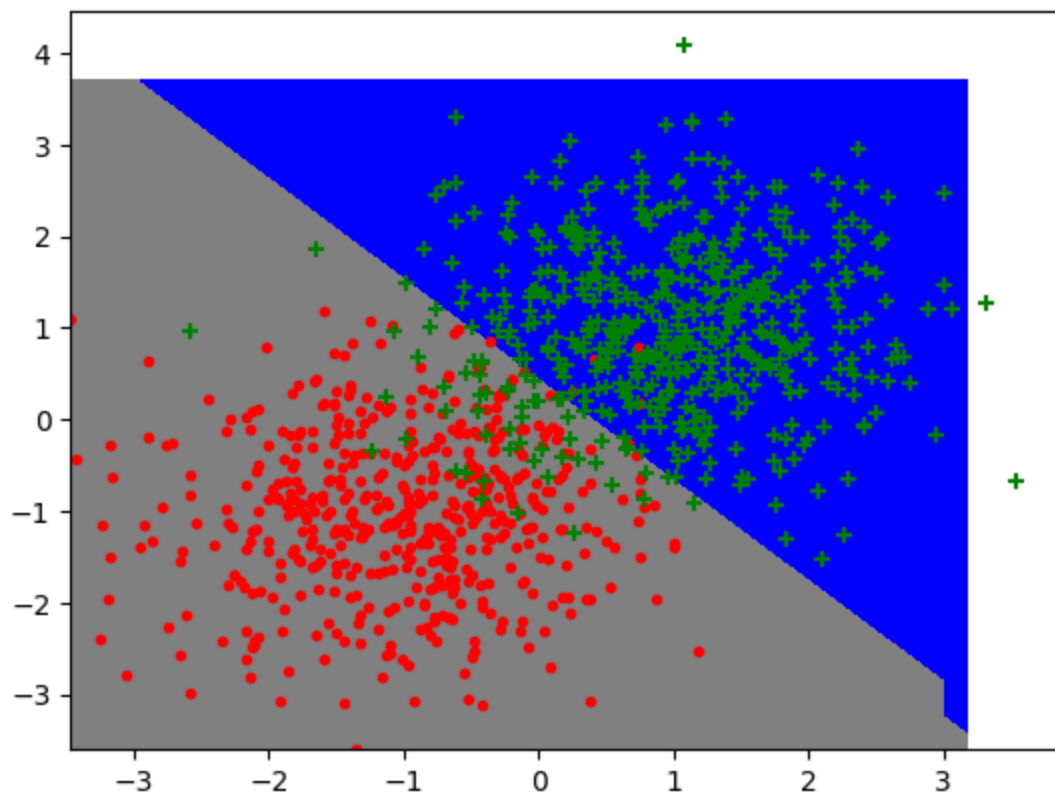
b) modèles sklearn

```
In [208... models = {
    "Perceptron": Perceptron(max_iter=1000, tol=1e-3),
    "k-NN": KNeighborsClassifier(n_neighbors=5),
    "Arbre de décision": DecisionTreeClassifier(max_depth=5),
    "SVM rbf": SVC(kernel='rbf', C=1.0), # Ajout du SVM linéaire
}

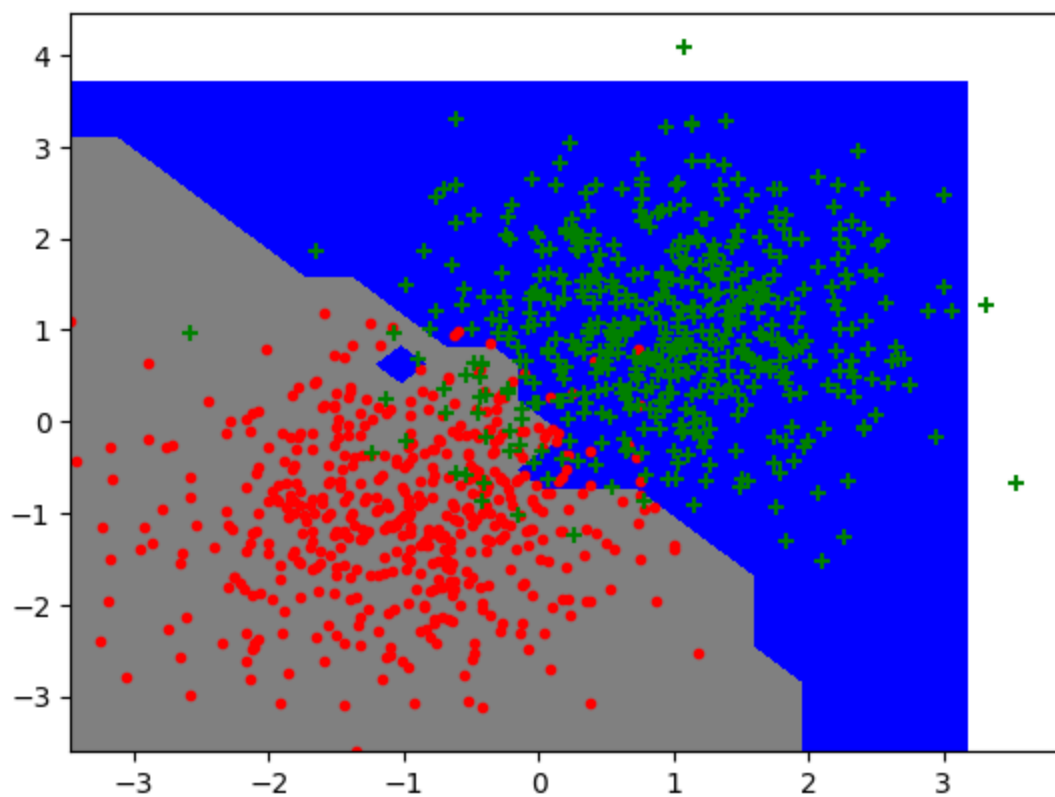
for name, model in models.items():
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_score:.2f}")

    plot_frontiere(x, model.predict)
    plot_data(x, y)
    plt.show()
```

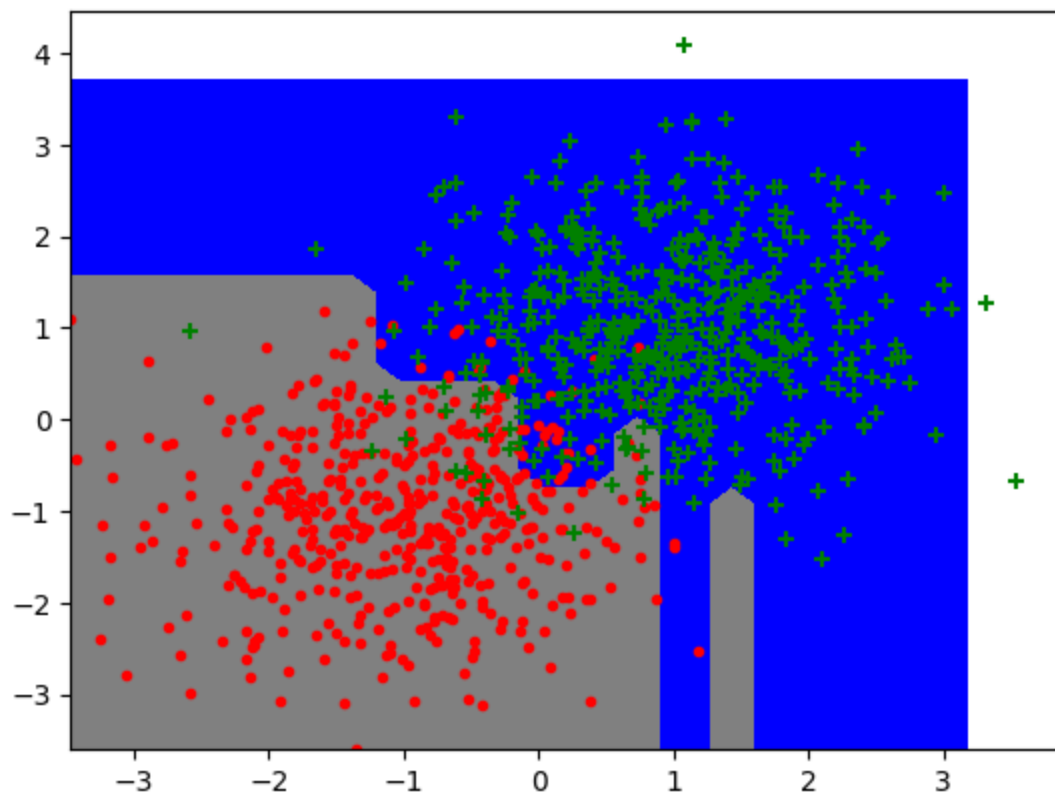
Perceptron - Train Acc: 0.94, Test Acc: 0.94



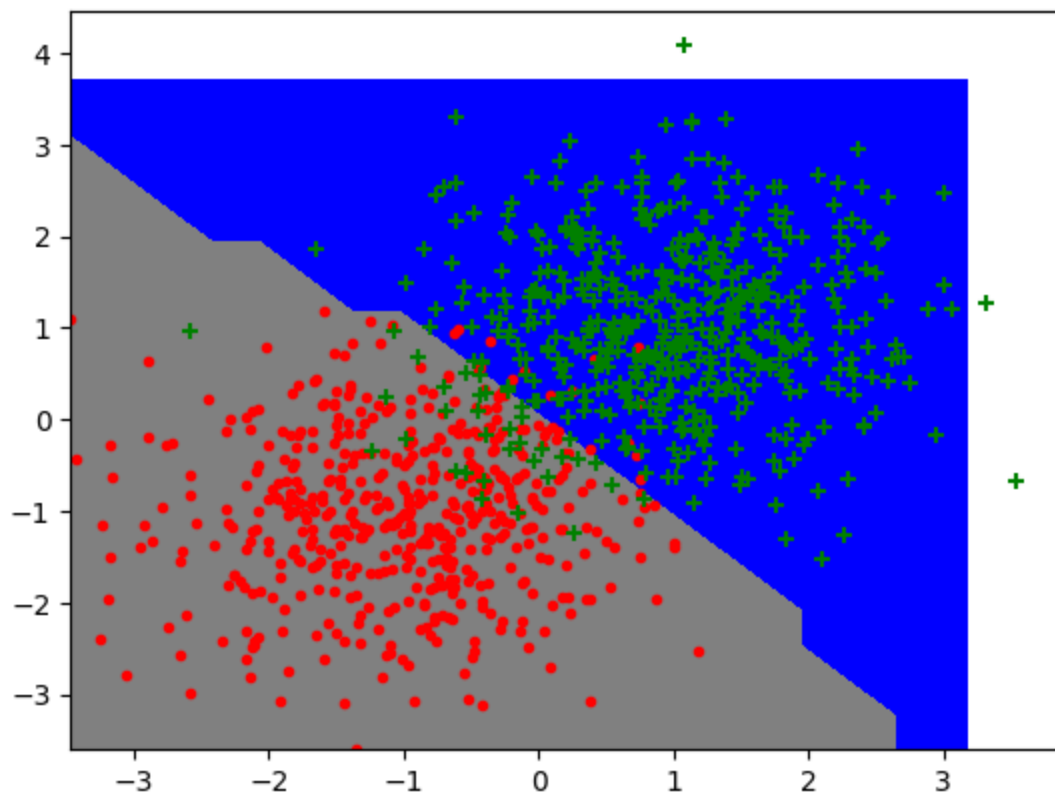
k-NN - Train Acc: 0.95, Test Acc: 0.94



Arbre de décision - Train Acc: 0.96, Test Acc: 0.93



SVM rbf - Train Acc: 0.95, Test Acc: 0.96

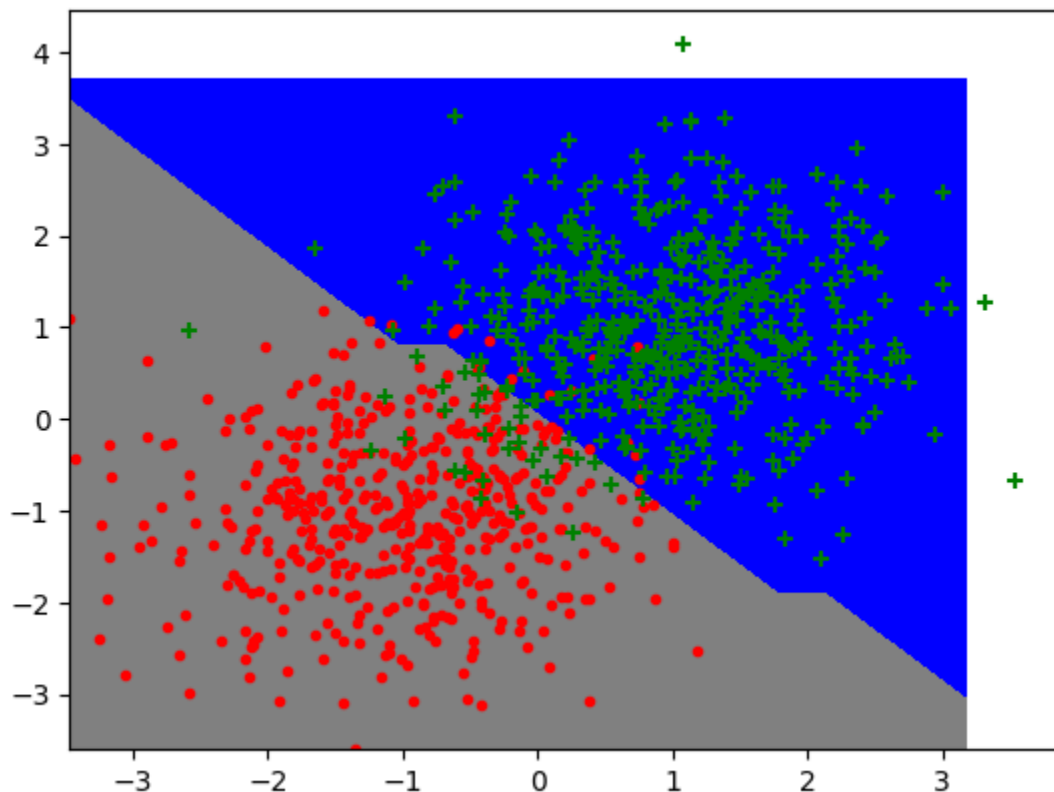


Tous les modèles sont performants (>90%). Le plus homogène semble être le k-NN avec des performances similaires en train et test.

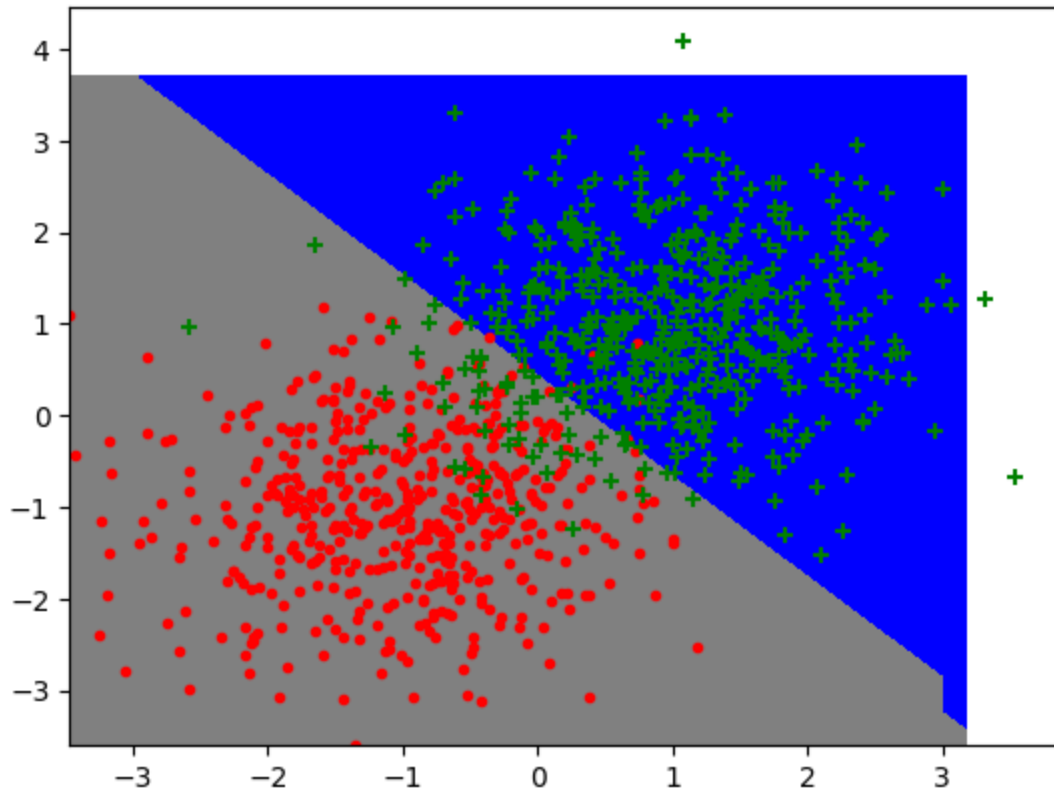
c) Comparaison perceptron DIY vs perceptron Sklearn

```
In [210... models = {  
    "DIY" :Lineaire(loss=perceptron_loss, loss_g=perceptron_grad, max_iter=3  
    "Perceptron": Perceptron(max_iter=300, tol=1e-3)  
}  
  
for name, model in models.items():  
    model.fit(x_train, y_train)  
    train_score = model.score(x_train, y_train)  
    test_score = model.score(x_test, y_test)  
    print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_score:.2f}  
  
    plot_frontiere(x, model.predict)  
    plot_data(x, y)  
    plt.show()
```

DIY - Train Acc: 0.94, Test Acc: 0.96



Perceptron - Train Acc: 0.94, Test Acc: 0.94



Le perceptron que nous avons produit semble avoir de meilleures performances que celui de sklearn.

IV.2) étude des différents svm (linéaires, rbf, poly)

```
In [ ]: def plot_frontiere_proba(data, f, step=20):
        """
        Plot the decision boundary based on the probability function.

        Params:
            data (np.ndarray): input features to define the grid
            f (callable): function that outputs probabilities or decision values
            step (int): step size for grid resolution (default is 20)

        Side Effects:
            Displays the contour plot of the decision boundary
        """

        grid, x, y = make_grid(data=data, step=step)
        plt.contourf(x, y, f(grid).reshape(x.shape), 255)
```

```
In [ ]: def plot_data_with_support_vectors(x, y, support_vectors):
        """
        Plot data points and highlight the support vectors.

        Params:
            x (np.ndarray): input features
            y (np.ndarray): labels (+1 or -1)
            support_vectors (np.ndarray): coordinates of the support vectors
```

Side Effects:

Displays a scatter plot of data points with support vectors highlighted

```
plt.scatter(x[:, 0], x[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k', l
plt.scatter(support_vectors[:, 0], support_vectors[:, 1], facecolors='no
```

```
In [239... for s in ["linear", "rbf", "poly"]:
    for C in [0.1, 1, 10] :
        for gamma in [0.01, 0.1, 1] :
            print(f"Modèle de SVM : {s}, C = {C}, gamma = {gamma}")
            model = SVC(kernel=s, C=C, gamma=gamma, probability=True)
            model.fit(x_train, y_train)

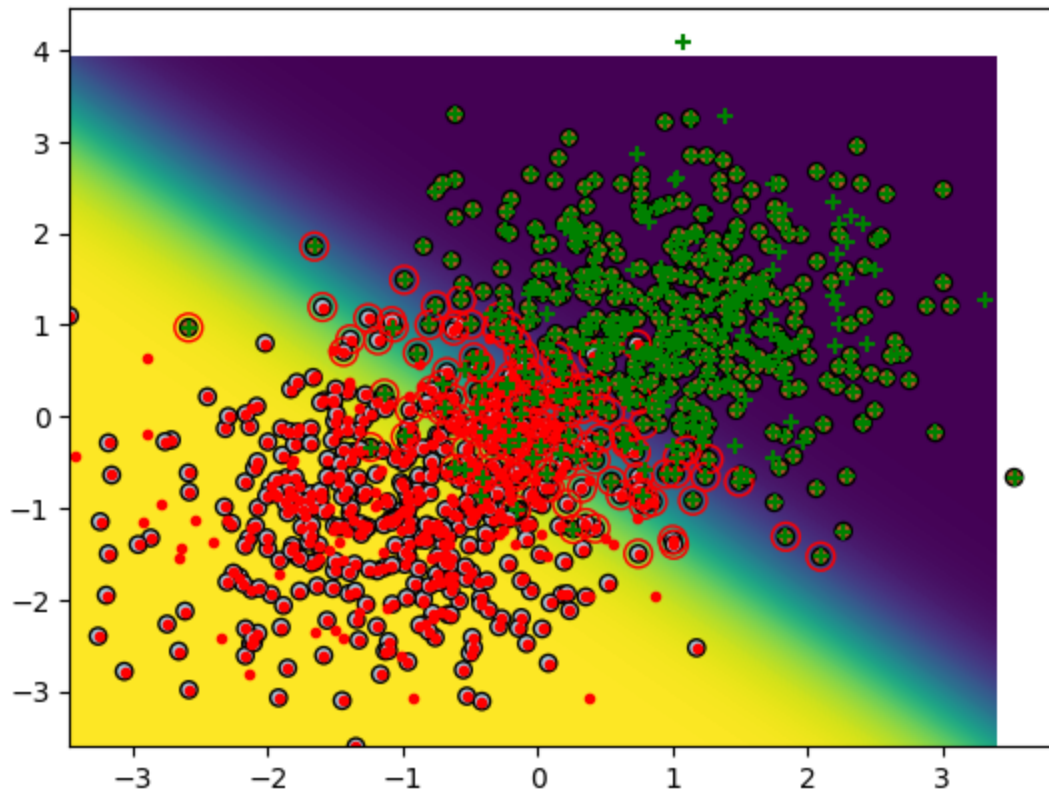
            support_vectors = model.support_vectors_

            train_score = model.score(x_train, y_train)
            test_score = model.score(x_test, y_test)
            print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_s

            plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0],
            plot_data_with_support_vectors(x_train, y_train, model.support_v
            plot_data(x, y)
            plt.show()
```

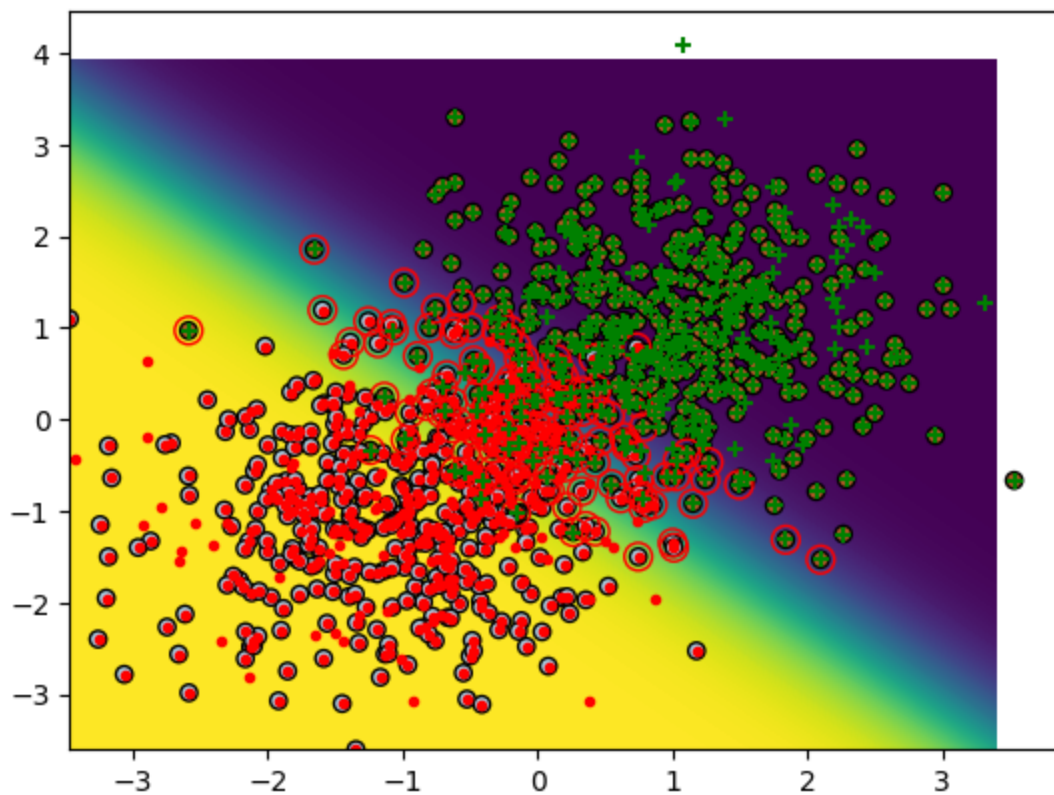
Modèle de SVM : linear, C = 0.1, gamma = 0.01

Perceptron - Train Acc: 0.94, Test Acc: 0.96

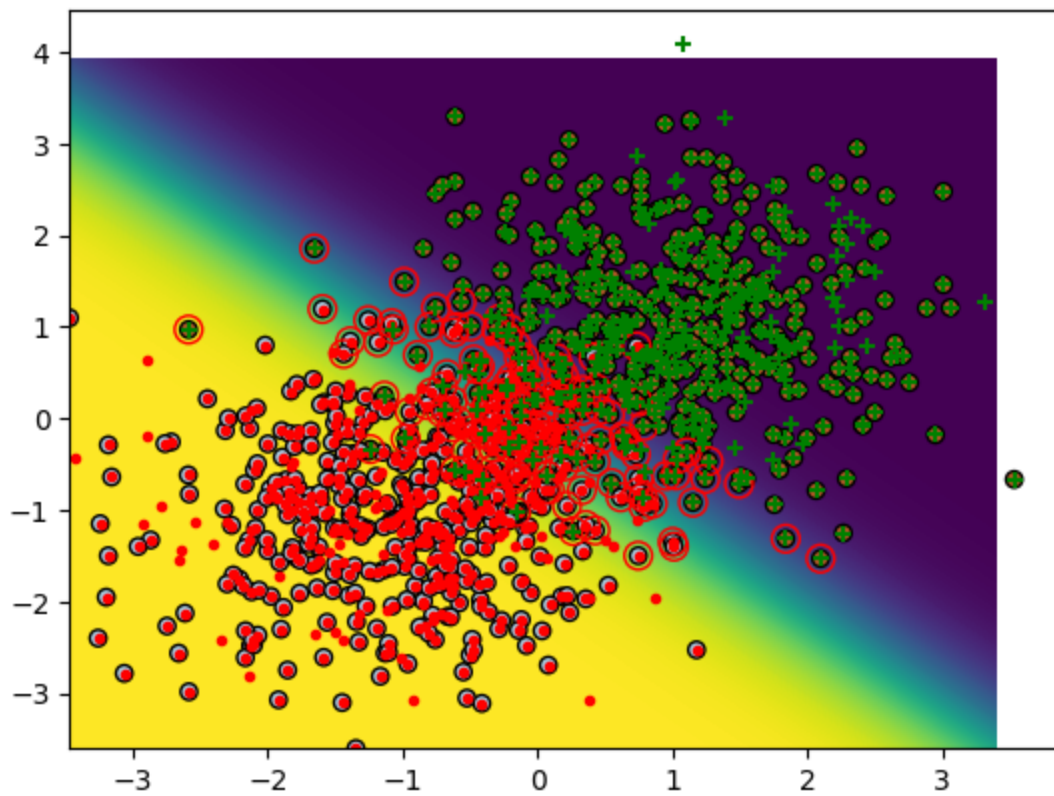


Modèle de SVM : linear, C = 0.1, gamma = 0.1

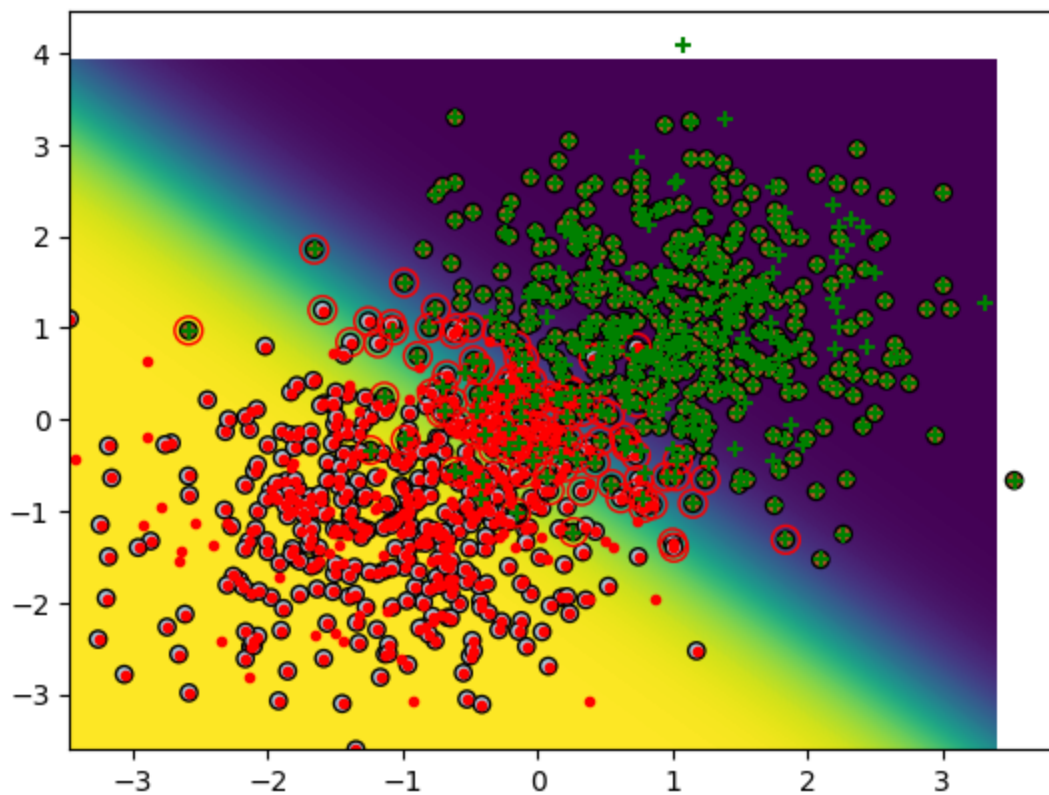
Perceptron - Train Acc: 0.94, Test Acc: 0.96



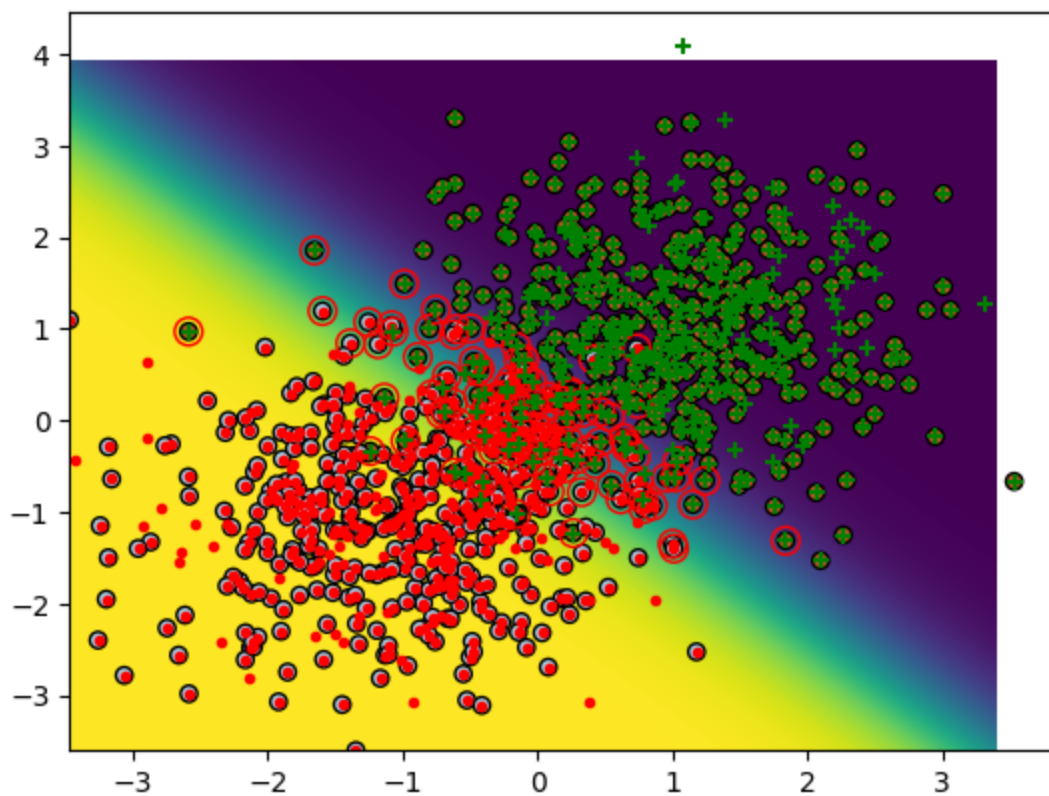
Modèle de SVM : linear, $C = 0.1$, $\gamma = 1$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



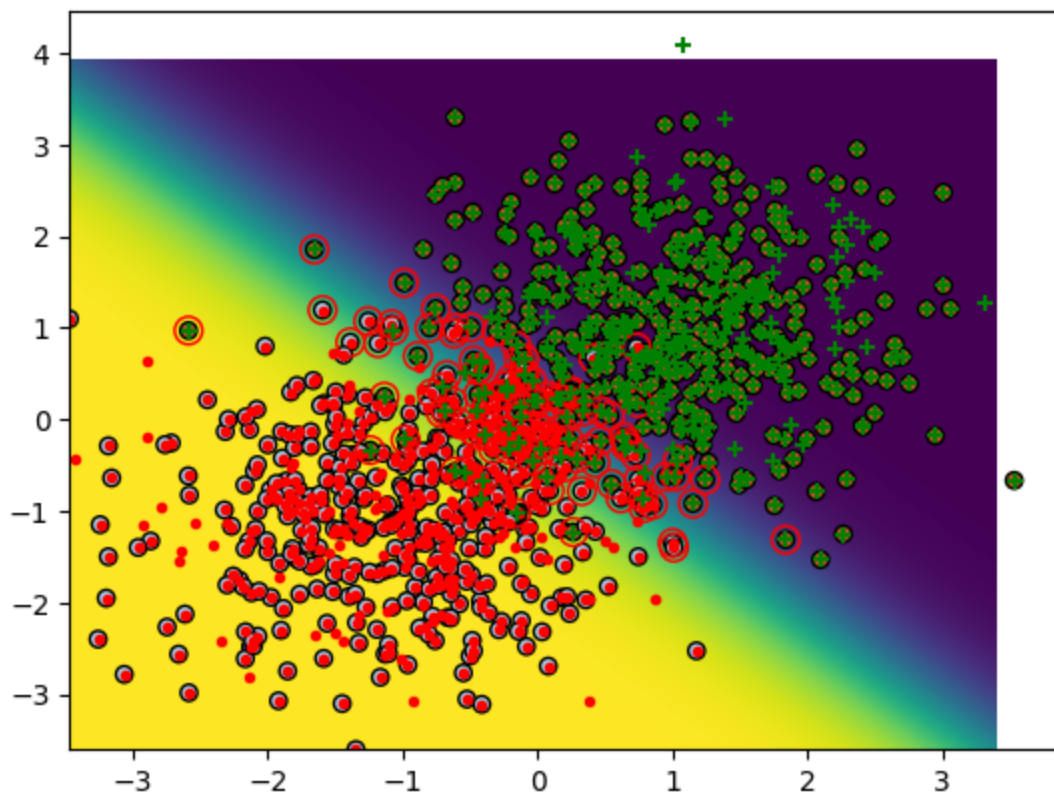
Modèle de SVM : linear, $C = 1$, $\gamma = 0.01$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



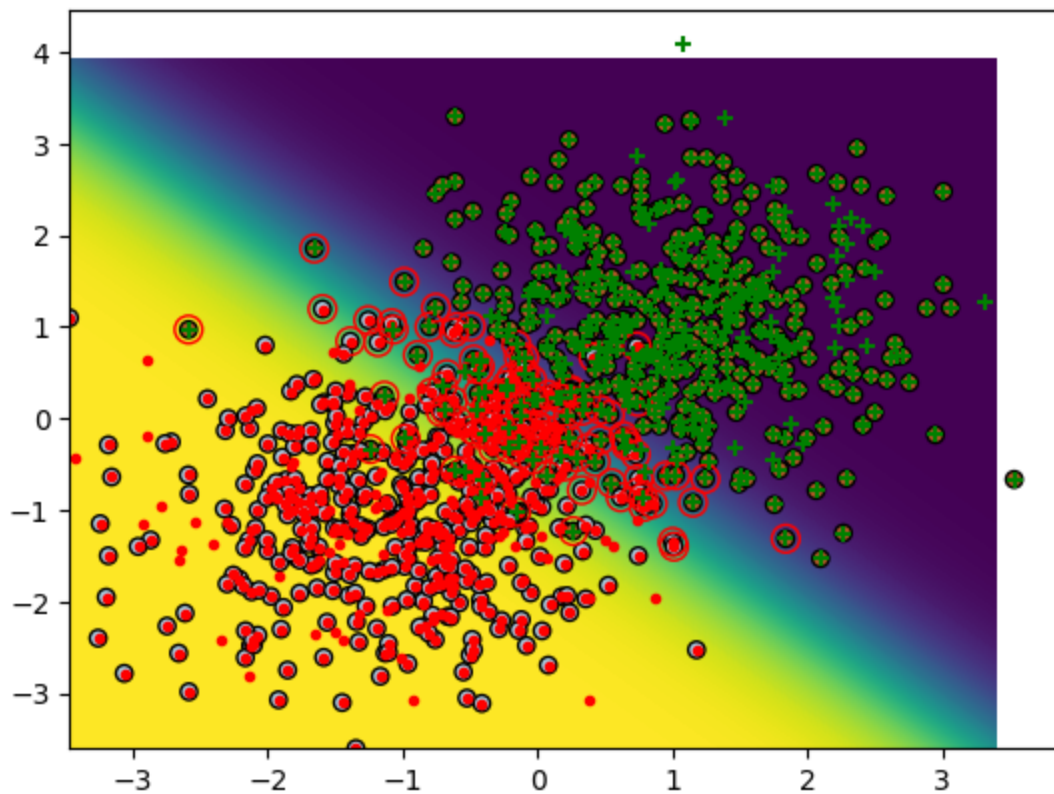
Modèle de SVM : linear, $C = 1$, $\gamma = 0.1$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



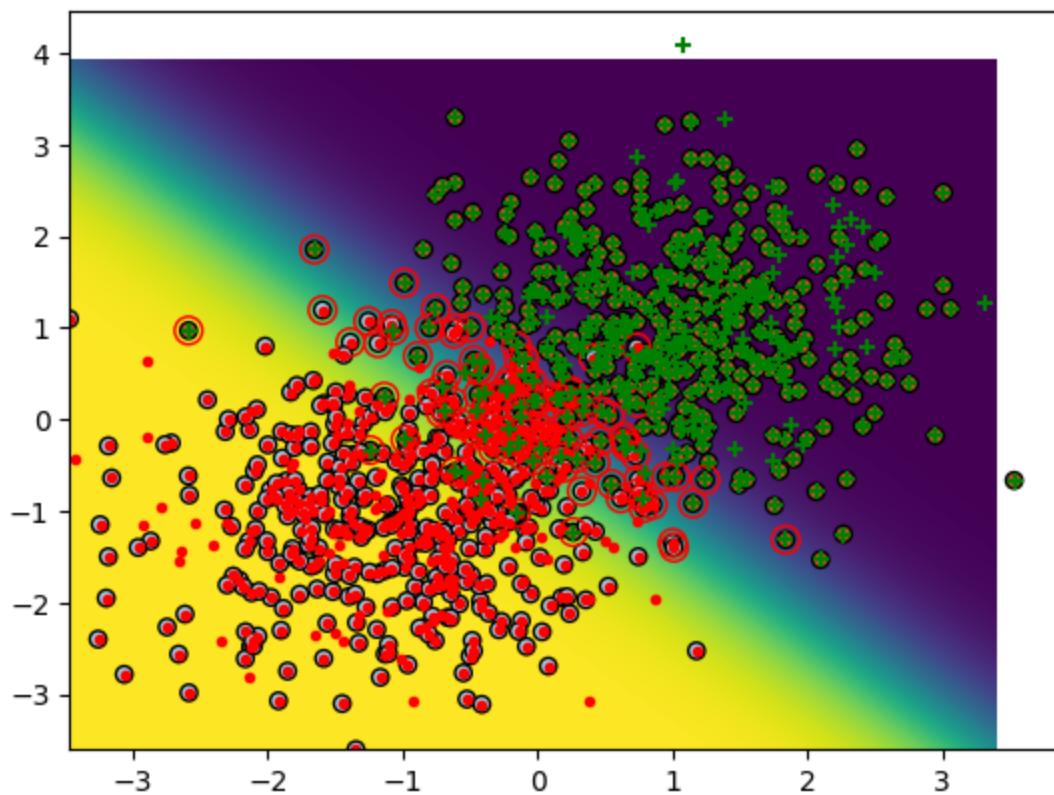
Modèle de SVM : linear, $C = 1$, $\gamma = 1$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



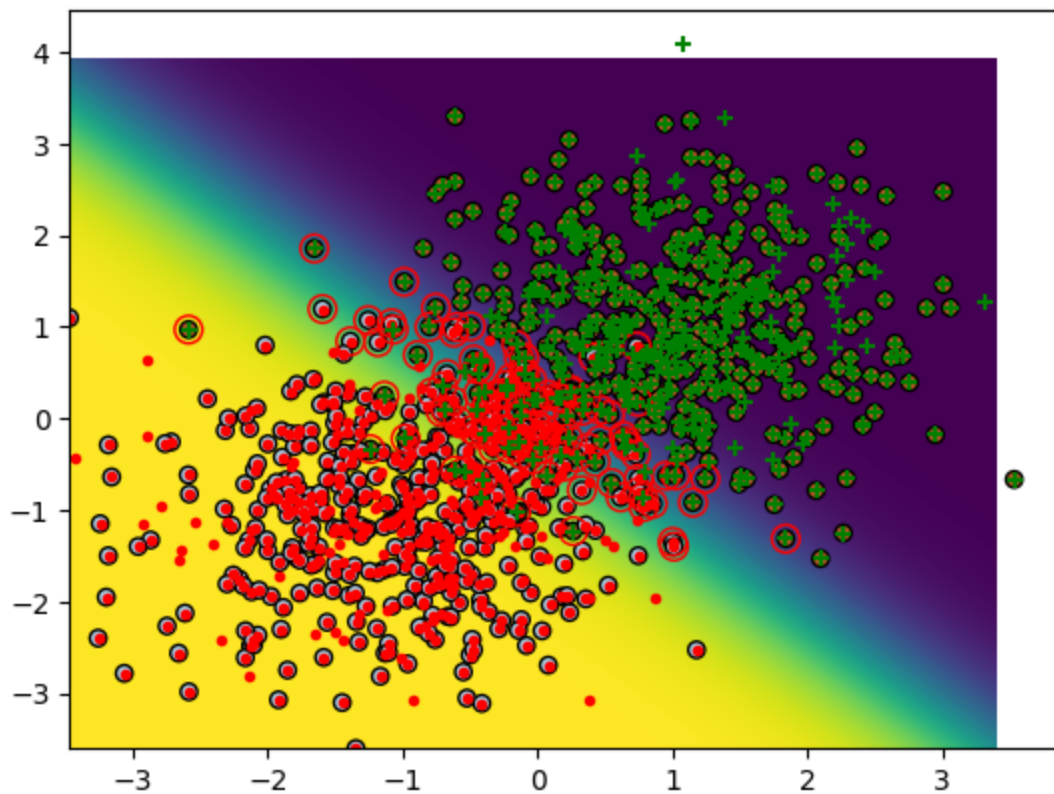
Modèle de SVM : linear, $C = 10$, $\gamma = 0.01$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



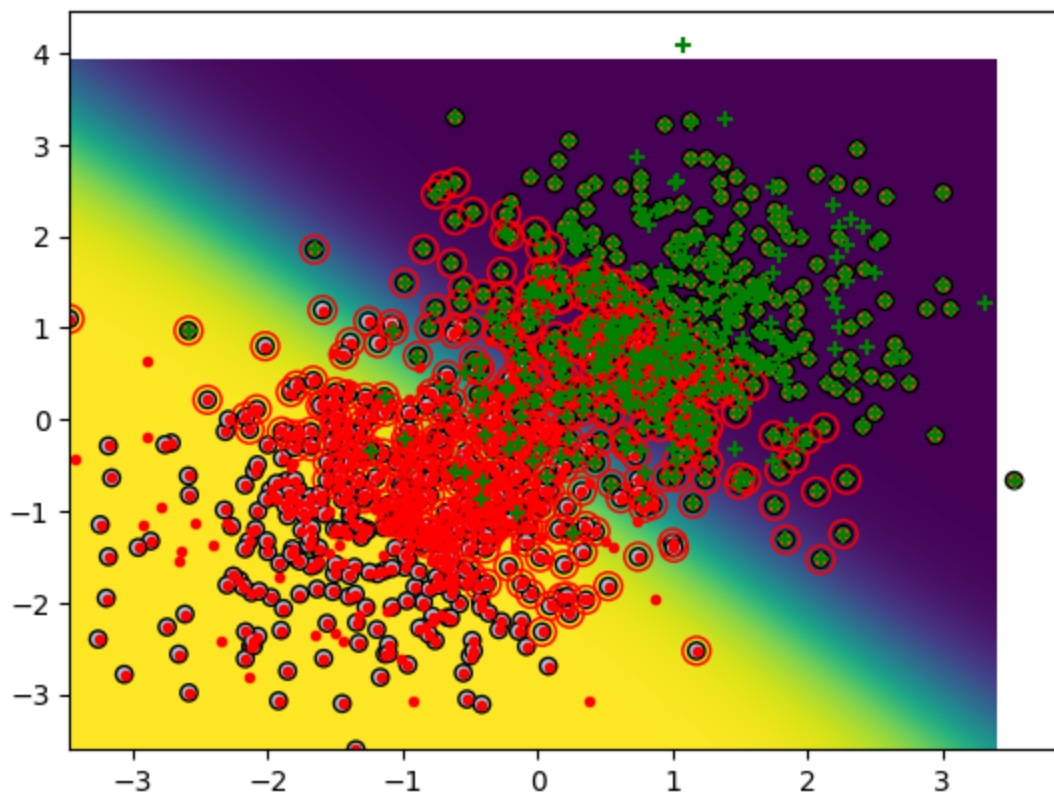
Modèle de SVM : linear, $C = 10$, $\gamma = 0.1$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



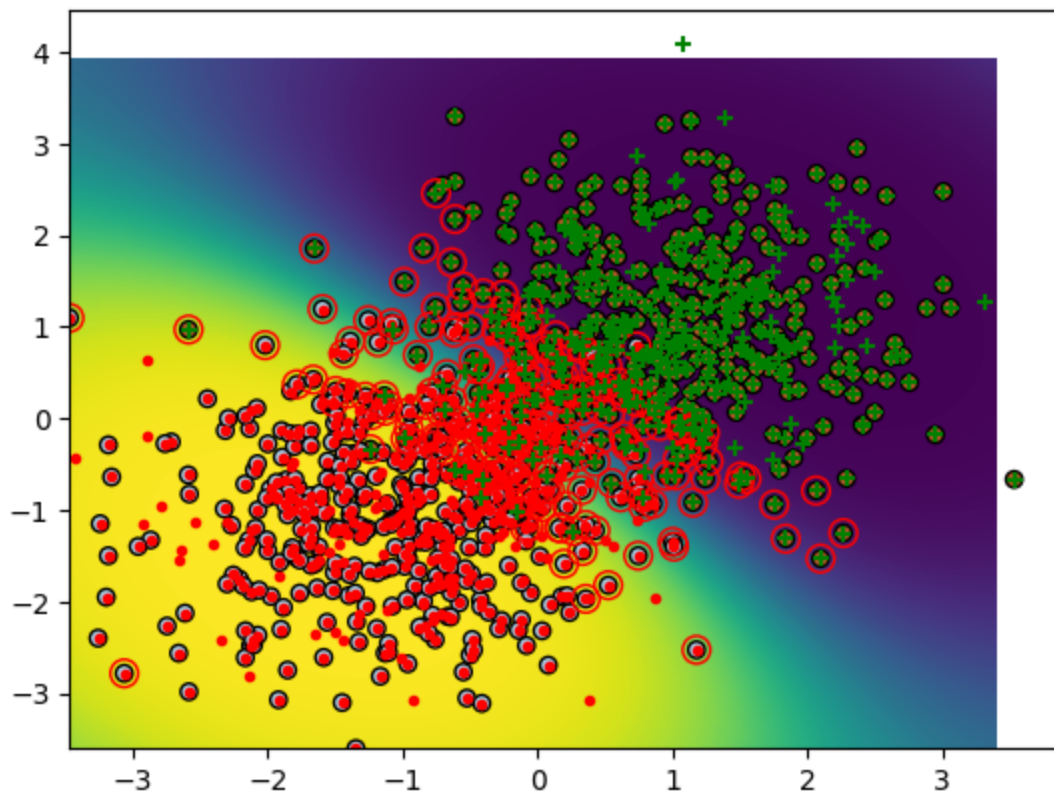
Modèle de SVM : linear, $C = 10$, $\gamma = 1$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



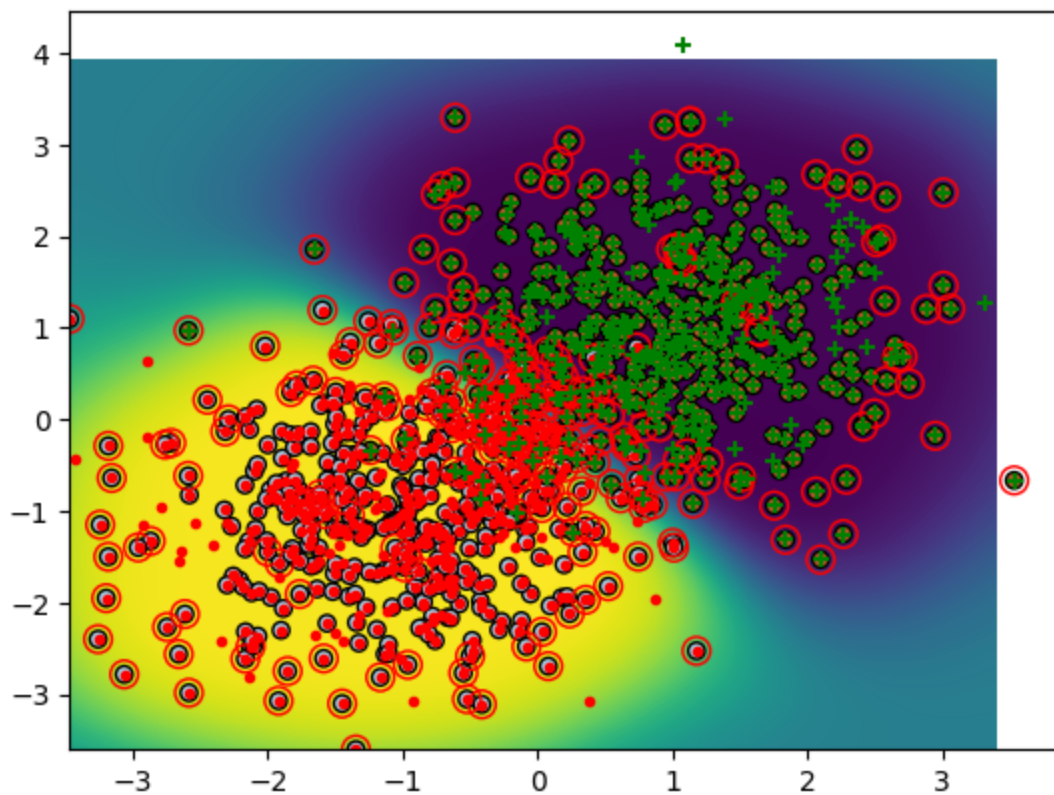
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 0.01$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



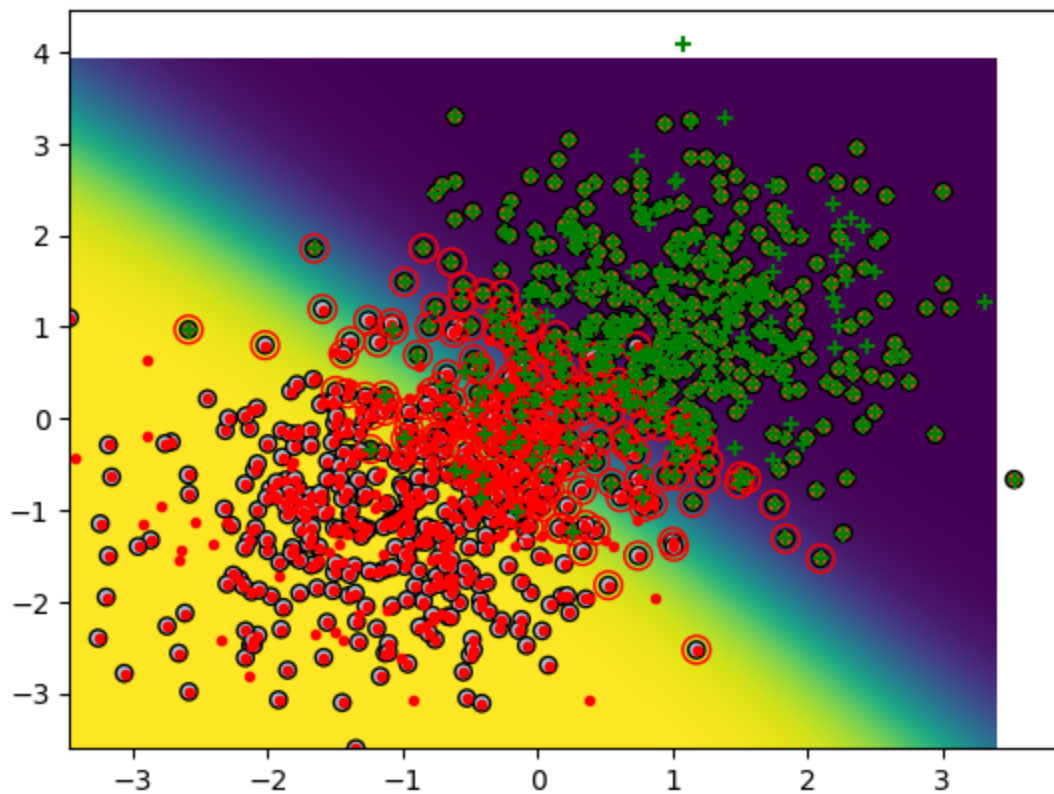
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 0.1$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



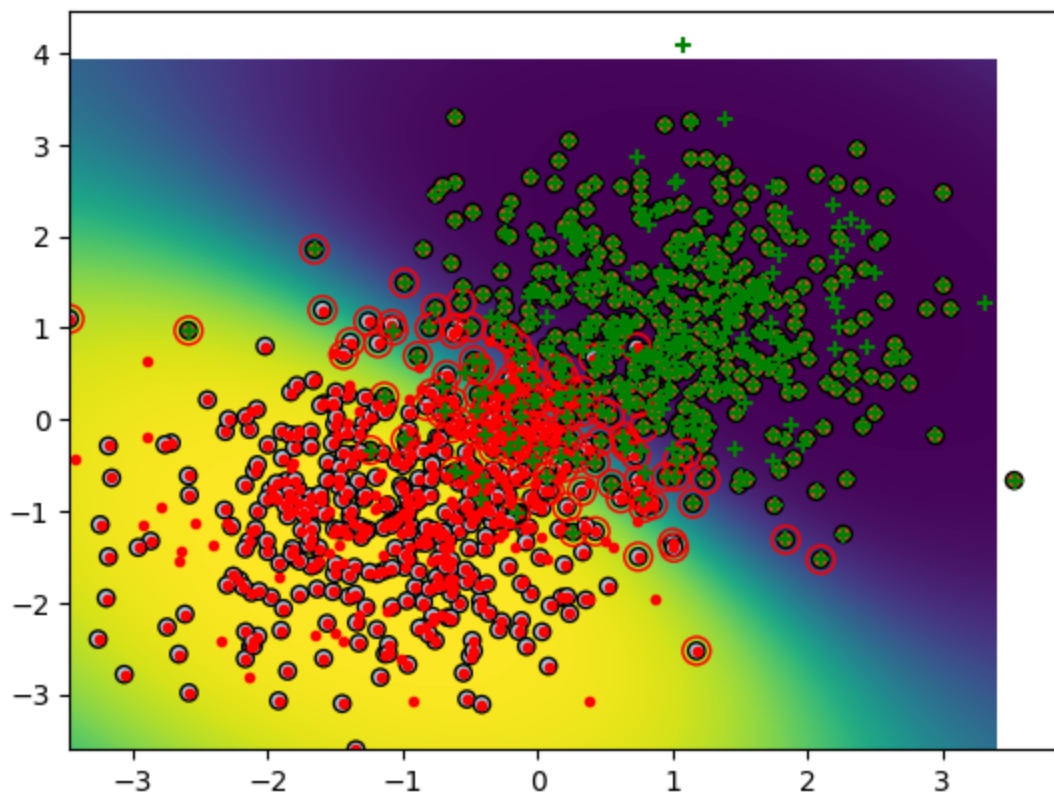
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 1$
Perceptron - Train Acc: 0.94, Test Acc: 0.97



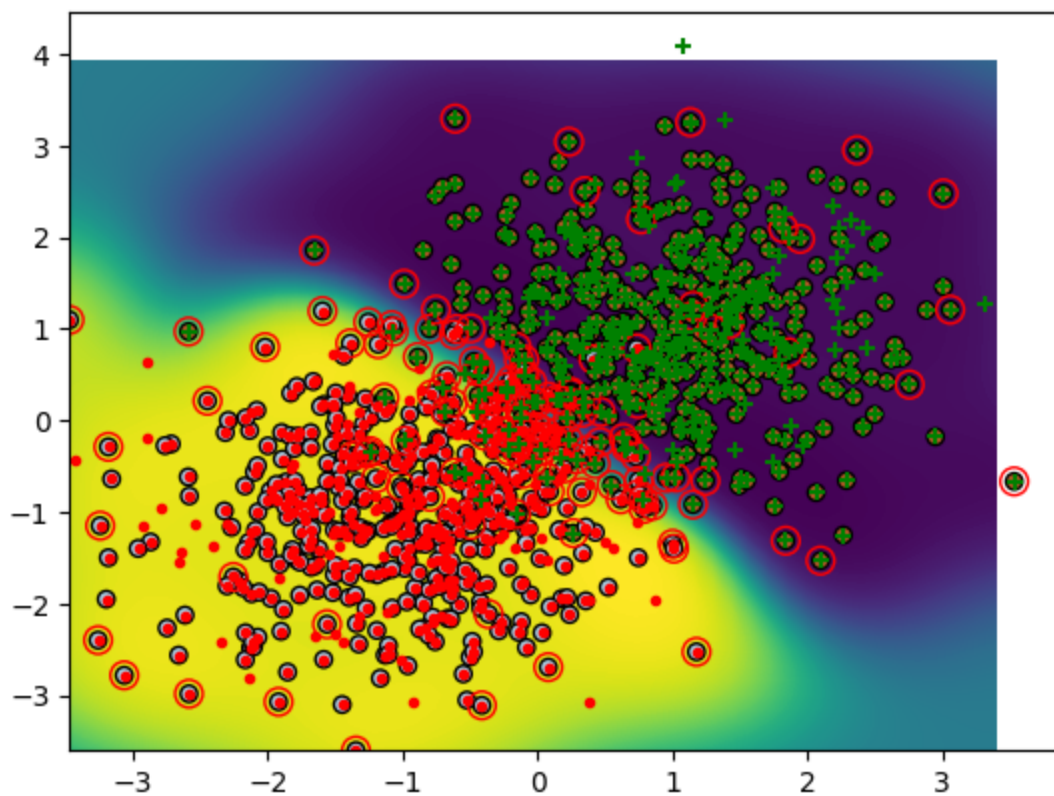
Modèle de SVM : rbf, $C = 1$, $\gamma = 0.01$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



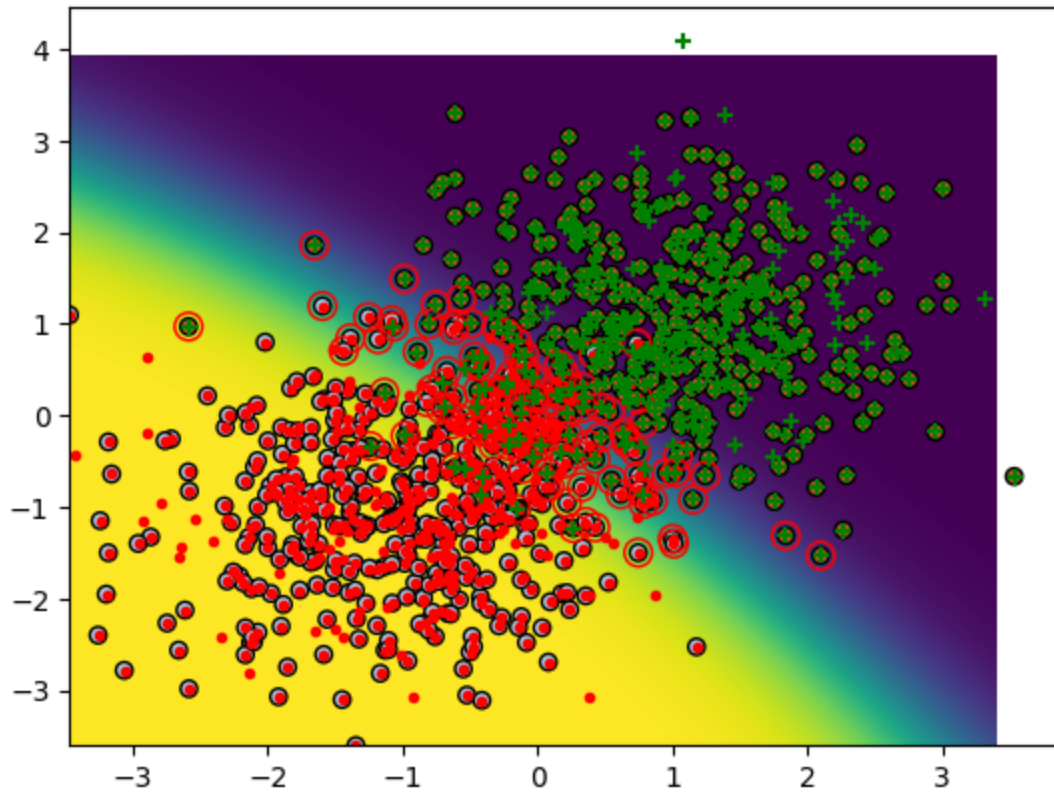
Modèle de SVM : rbf, $C = 1$, $\gamma = 0.1$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



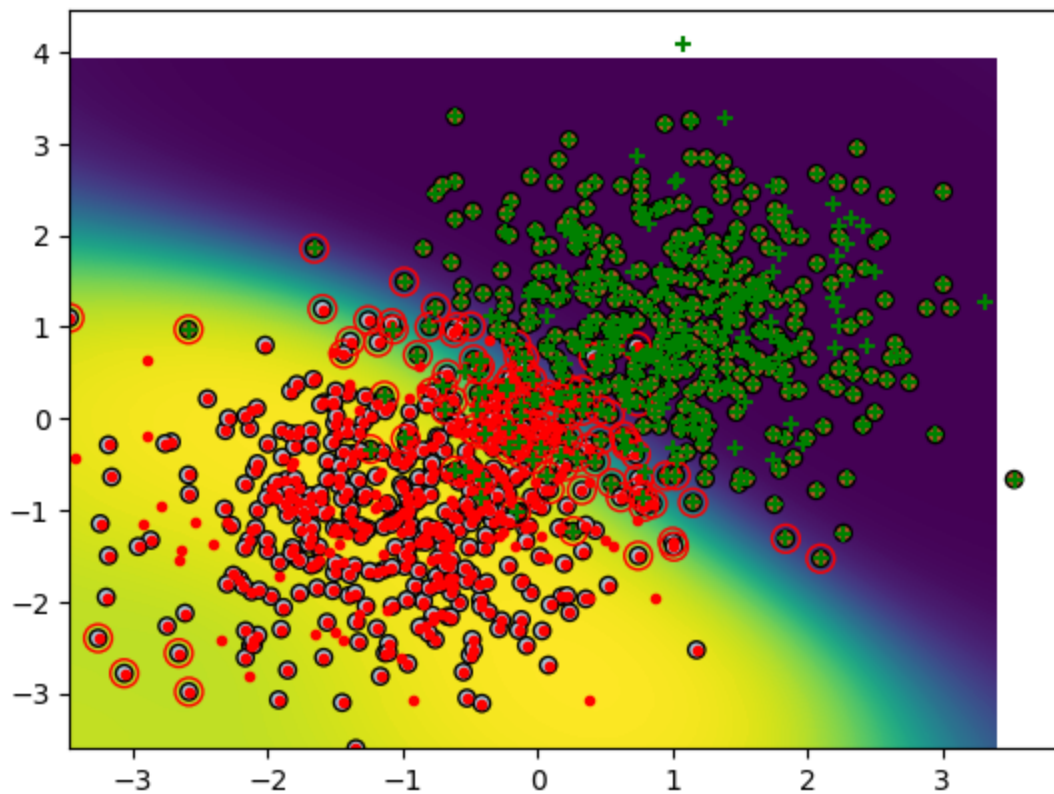
Modèle de SVM : rbf, $C = 1$, $\gamma = 1$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



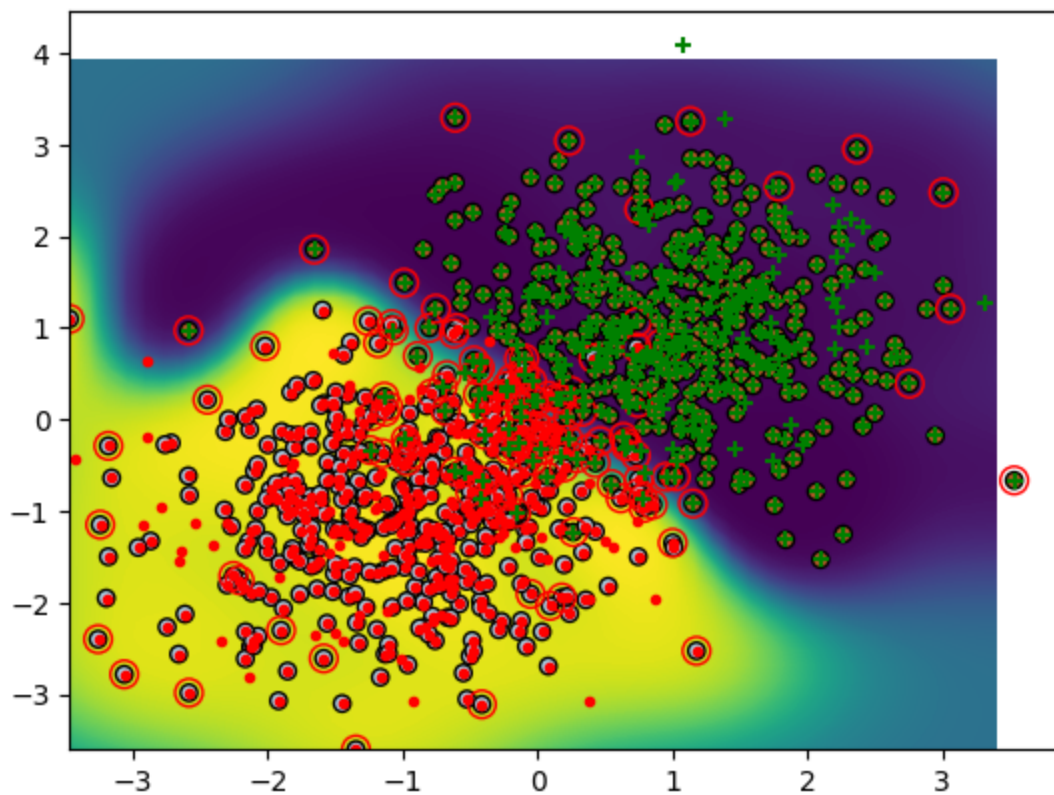
Modèle de SVM : rbf, $C = 10$, $\gamma = 0.01$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



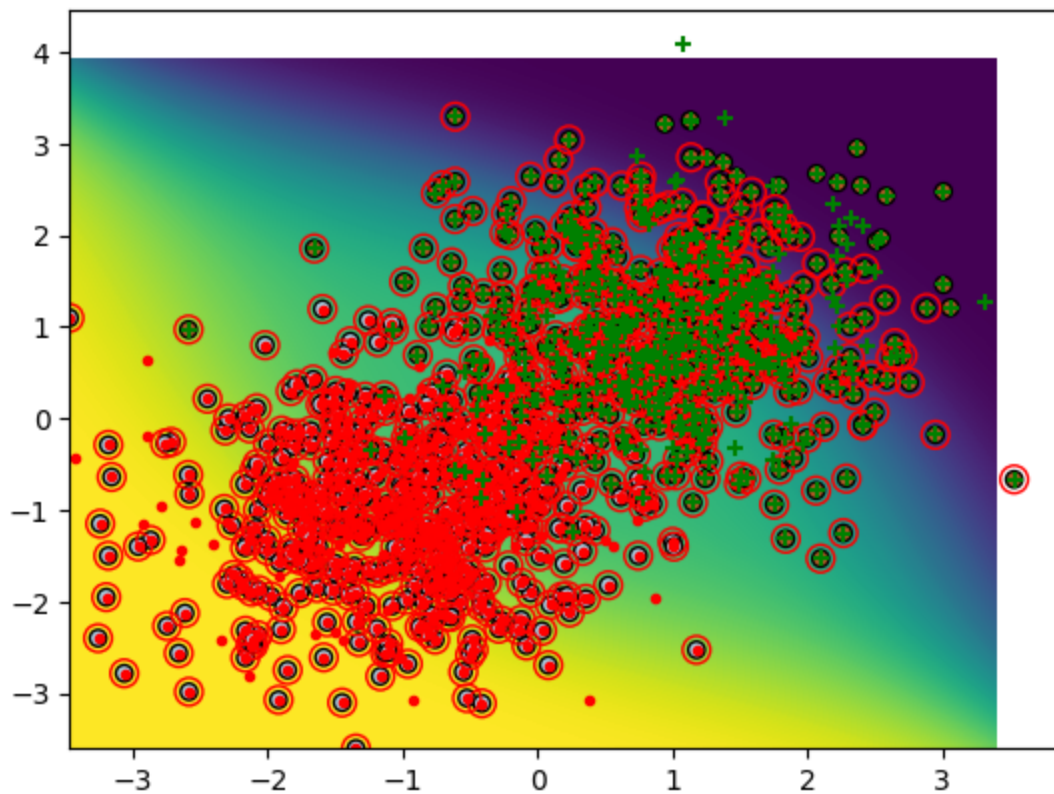
Modèle de SVM : rbf, $C = 10$, $\gamma = 0.1$
Perceptron - Train Acc: 0.95, Test Acc: 0.96



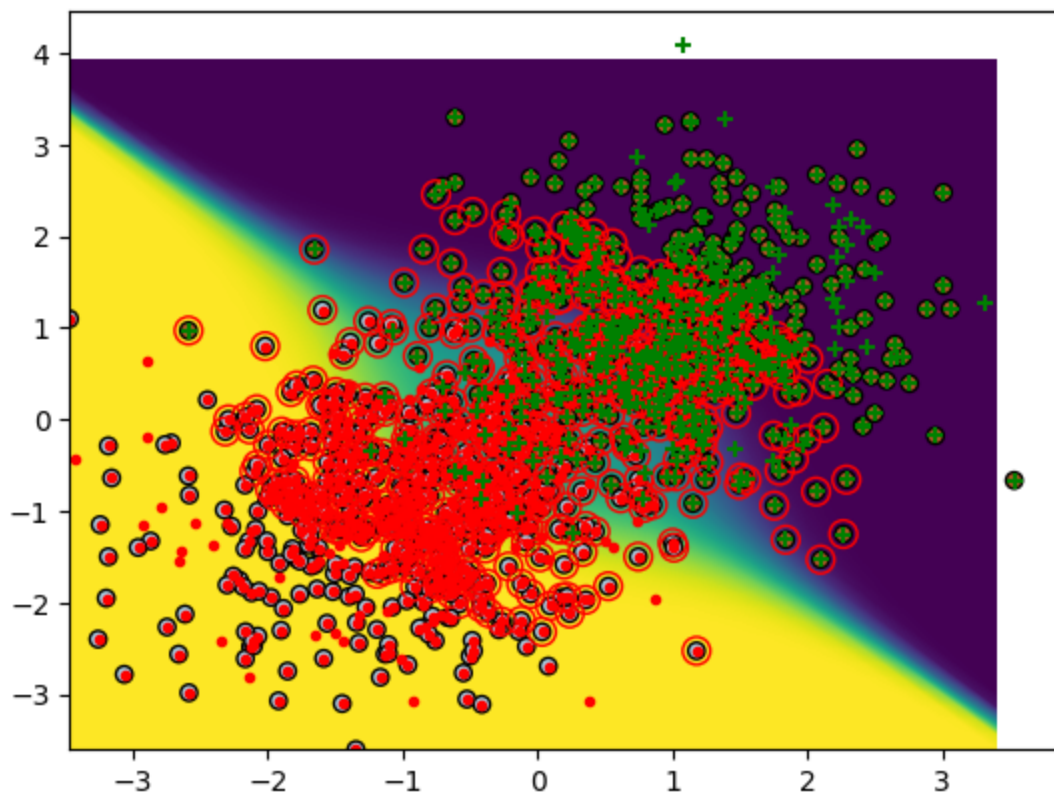
Modèle de SVM : rbf, $C = 10$, $\gamma = 1$
Perceptron - Train Acc: 0.95, Test Acc: 0.95



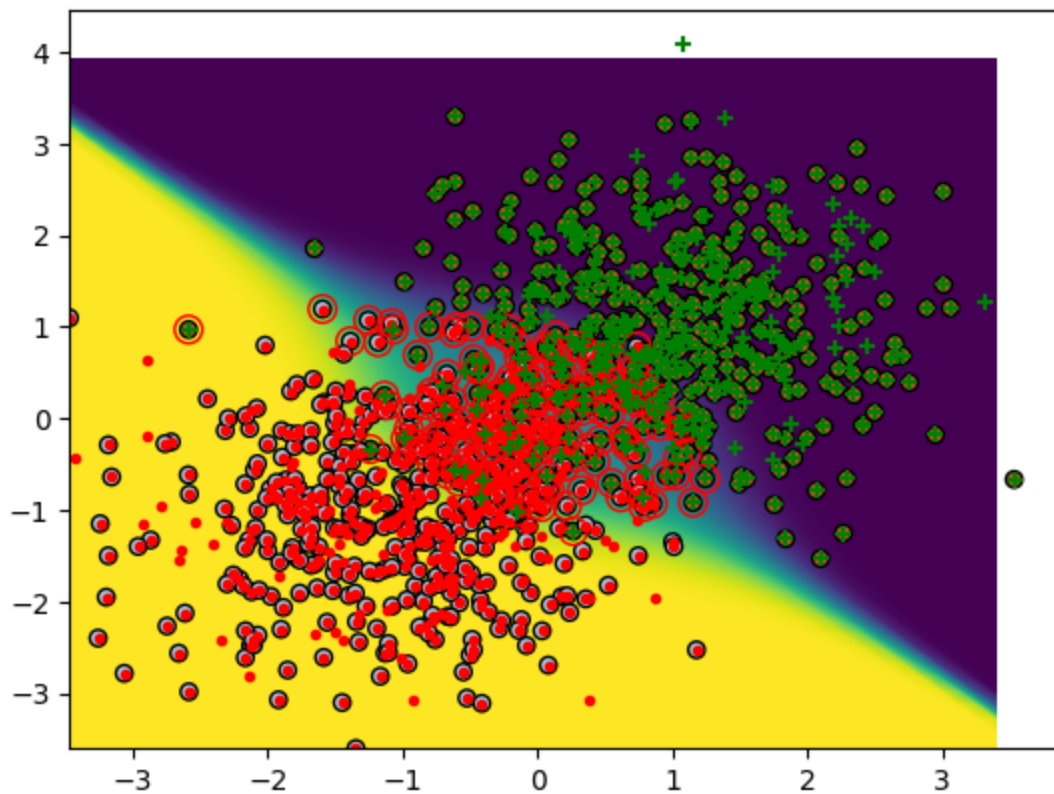
Modèle de SVM : poly, $C = 0.1$, $\gamma = 0.01$
Perceptron - Train Acc: 0.51, Test Acc: 0.46



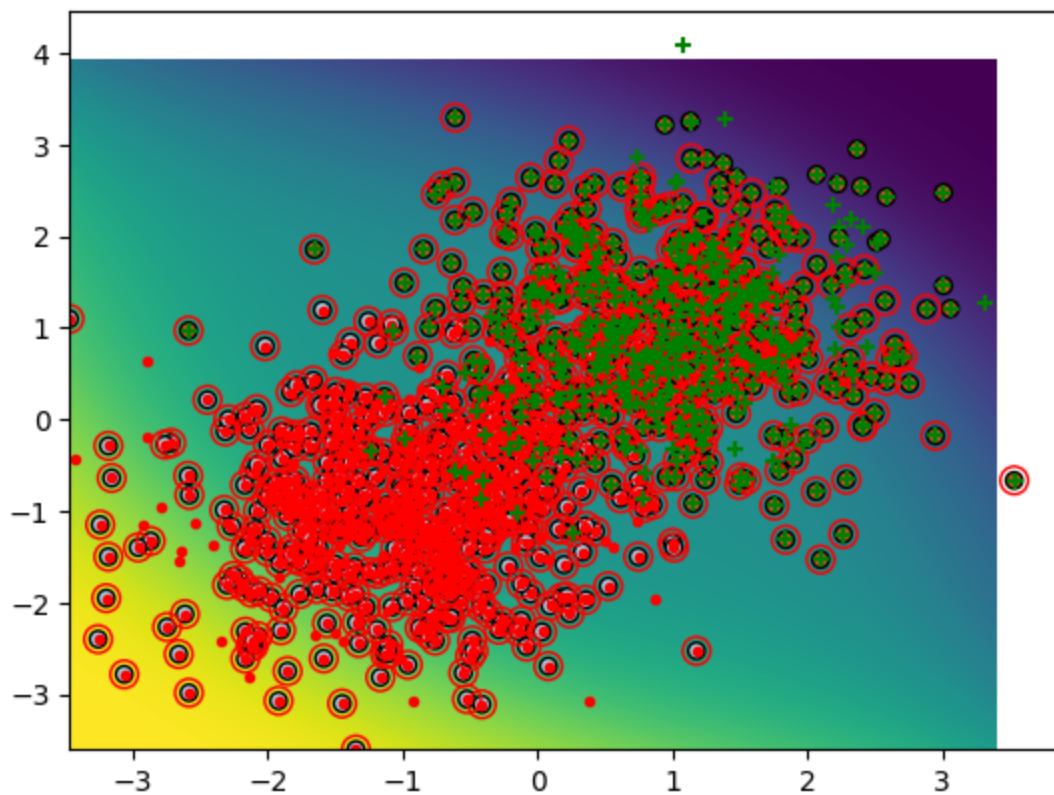
Modèle de SVM : poly, $C = 0.1$, $\gamma = 0.1$
Perceptron - Train Acc: 0.91, Test Acc: 0.91



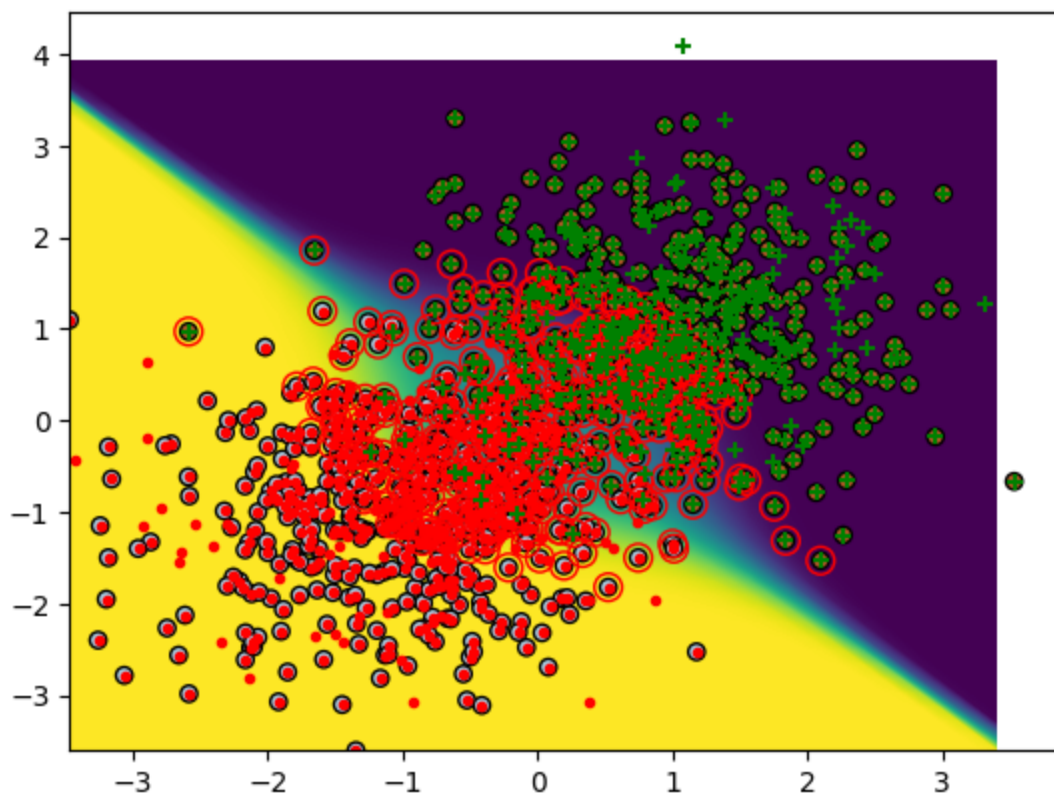
Modèle de SVM : poly, $C = 0.1$, $\gamma = 1$
Perceptron - Train Acc: 0.93, Test Acc: 0.96



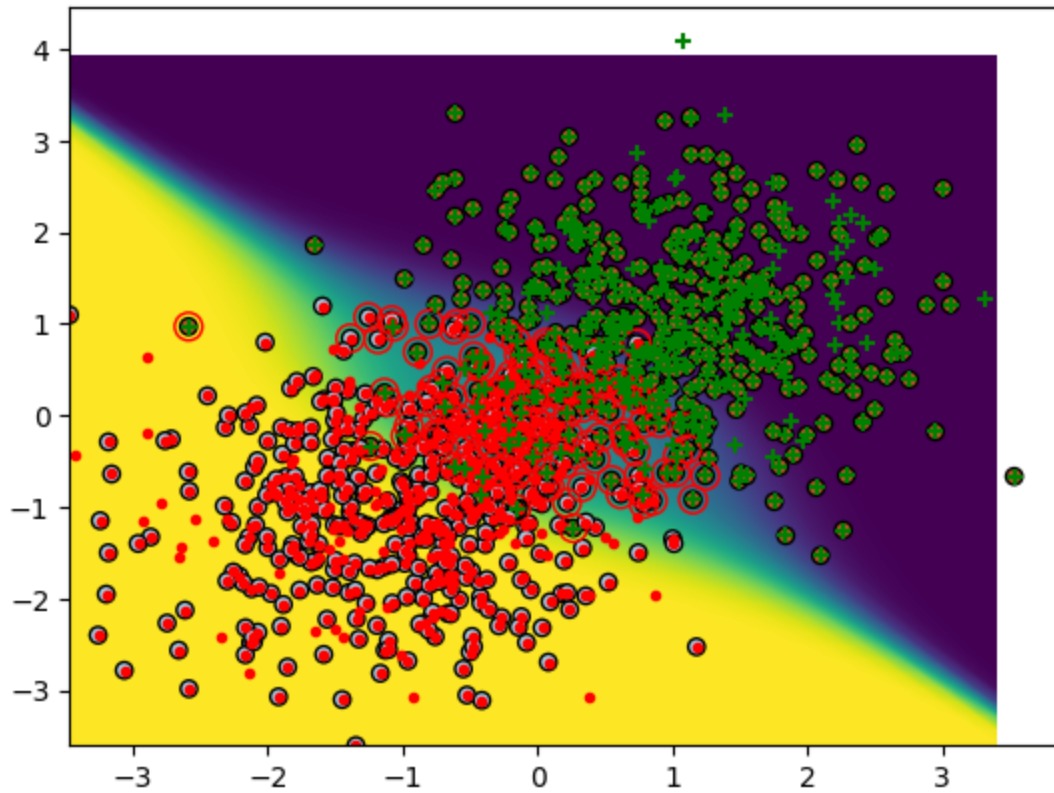
Modèle de SVM : poly, $C = 1$, $\gamma = 0.01$
Perceptron - Train Acc: 0.51, Test Acc: 0.46



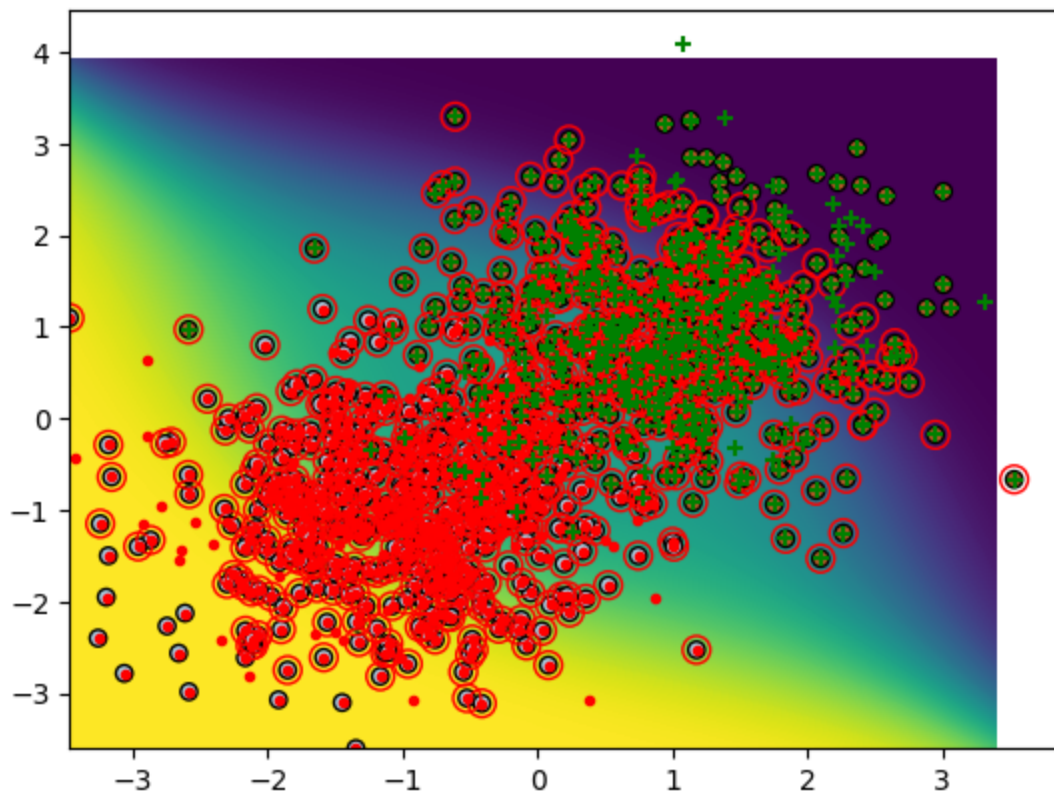
Modèle de SVM : poly, $C = 1$, $\gamma = 0.1$
Perceptron - Train Acc: 0.91, Test Acc: 0.91



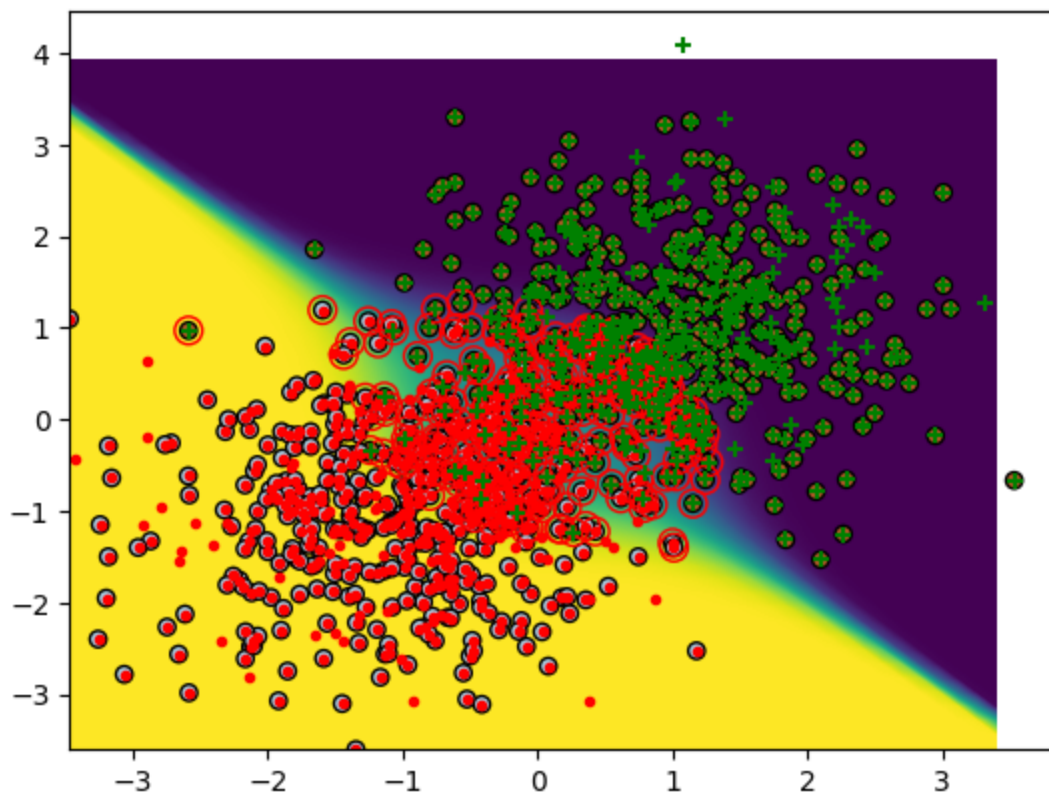
Modèle de SVM : poly, $C = 1$, $\gamma = 1$
Perceptron - Train Acc: 0.94, Test Acc: 0.96



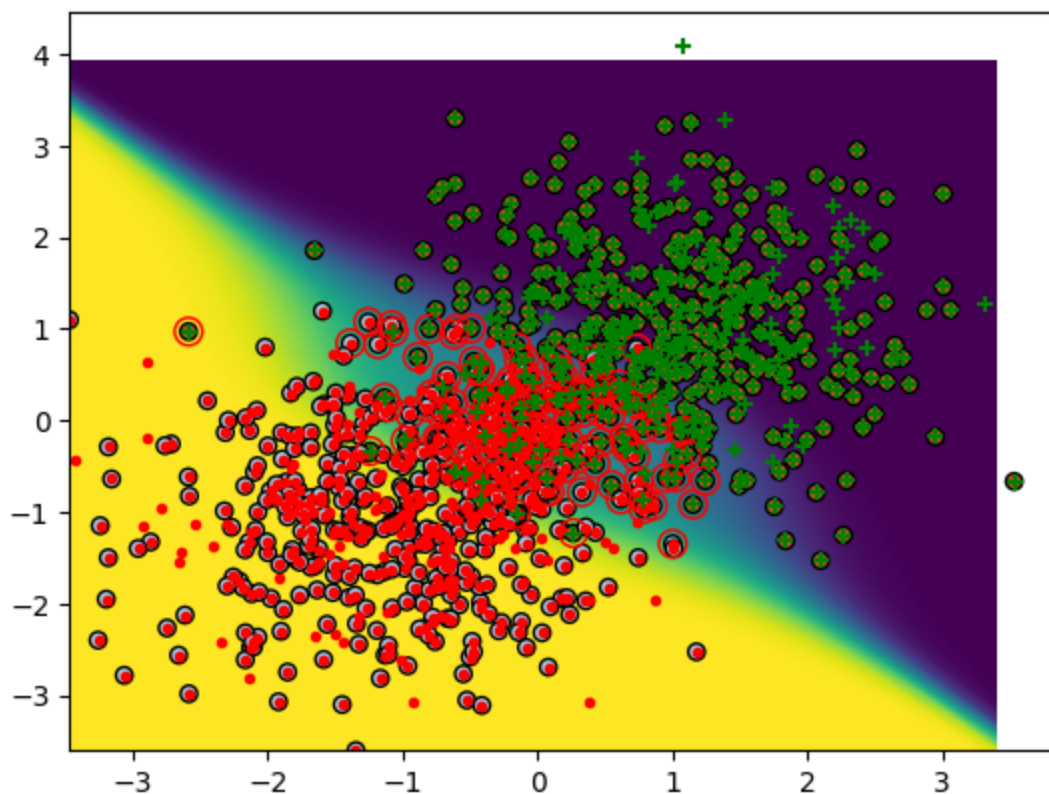
Modèle de SVM : poly, $C = 10$, $\gamma = 0.01$
Perceptron - Train Acc: 0.64, Test Acc: 0.60



Modèle de SVM : poly, $C = 10$, $\gamma = 0.1$
Perceptron - Train Acc: 0.93, Test Acc: 0.96



Modèle de SVM : poly, $C = 10$, $\gamma = 1$
Perceptron - Train Acc: 0.93, Test Acc: 0.96



```
In [224...] print("Avec sklearn on peut directement retrouver les paramètres optimaux :")  
  
for s in ["linear", "rbf", "poly"]:  
    params = {'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1]}
```

```
grid = GridSearchCV(SVC(kernel=s), params, cv=5)
grid.fit(x_train, y_train)
best_model = grid.best_estimator_
best_params = grid.best_params_
best_score = grid.best_score_
print(f"Meilleurs paramètre pour le modèle SVM {s}:", best_params)
```

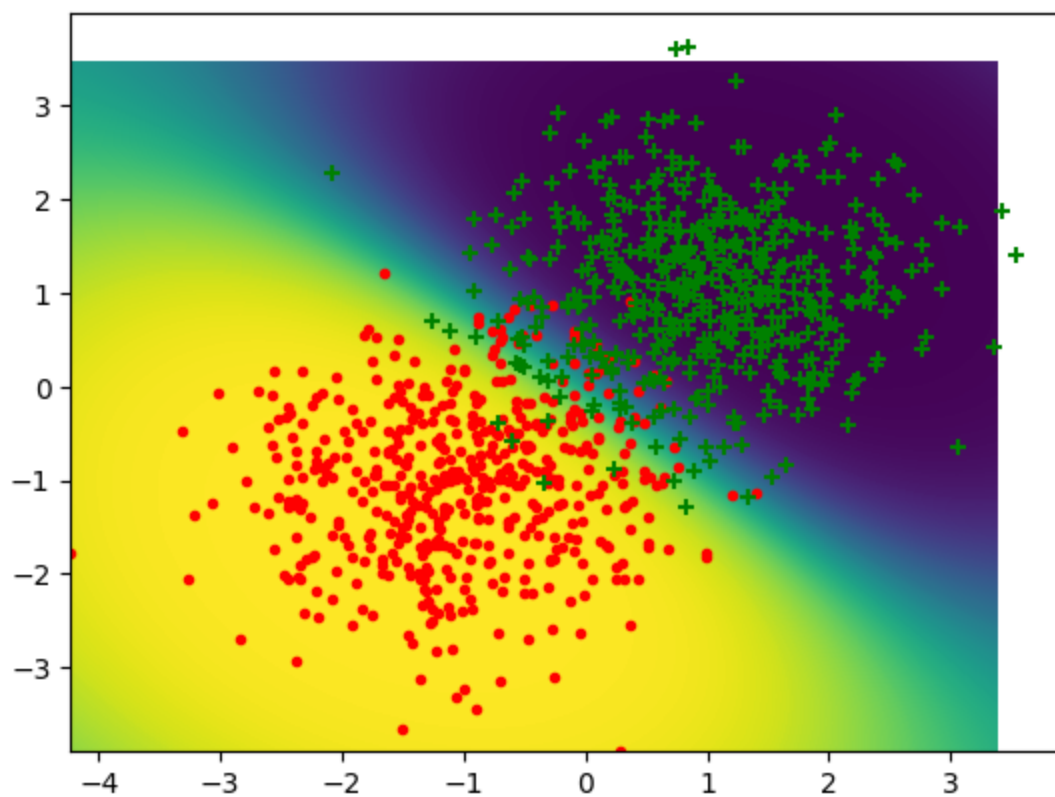
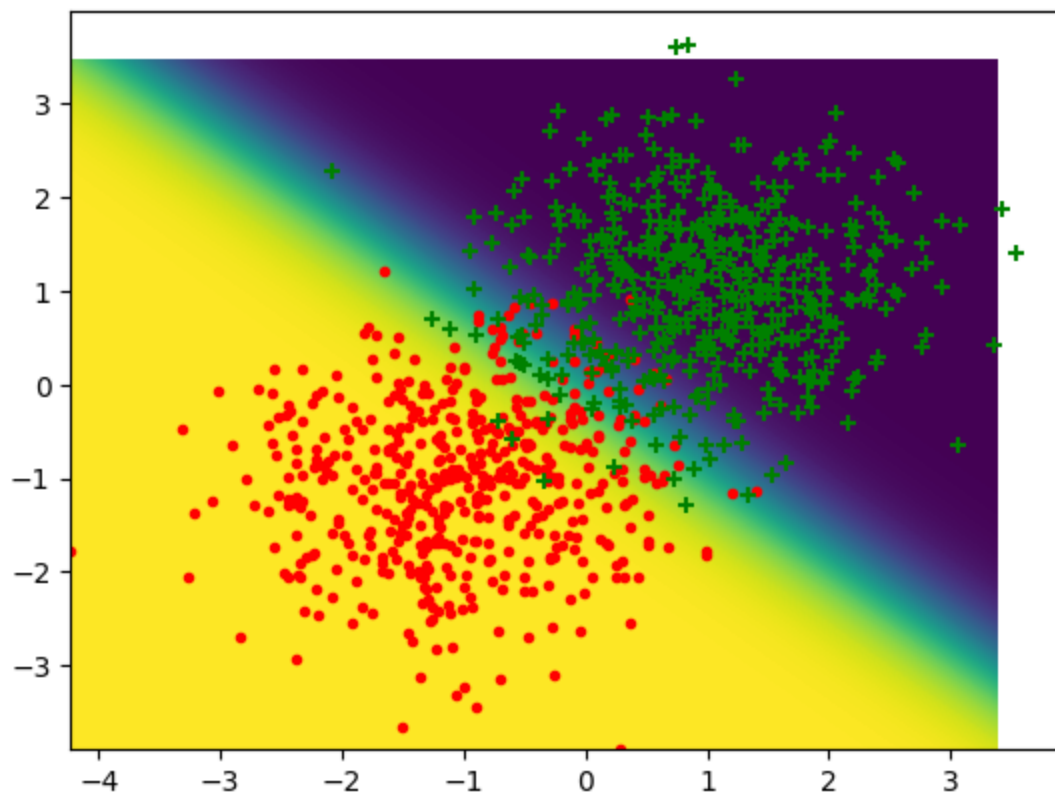
```
print("Ce qui s'observe sur les plots avec les frontières de décisions précé
```

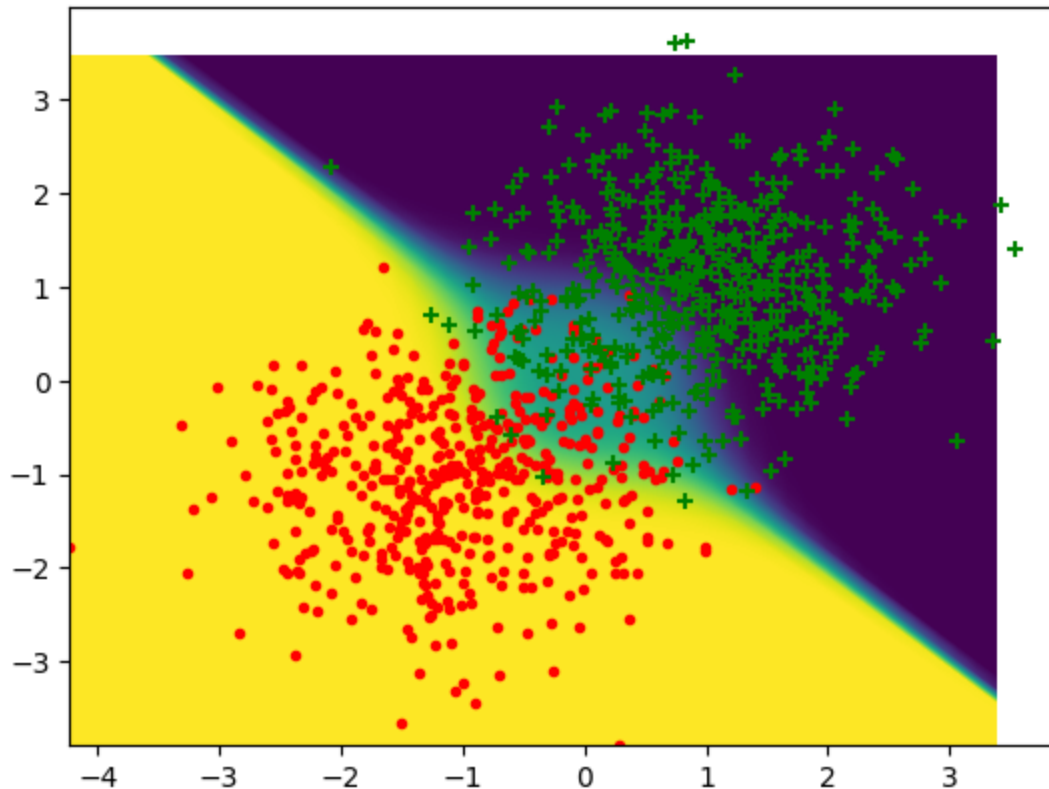
Avec sklearn on peut directement retrouver les paramètres optimaux :
Meilleurs paramètre pour le modèle SVM linear: {'C': 1, 'gamma': 0.01}
Meilleurs paramètre pour le modèle SVM rbf: {'C': 0.1, 'gamma': 0.1}
Meilleurs paramètre pour le modèle SVM poly: {'C': 1, 'gamma': 1}
Ce qui s'observe sur les plots avec les frontières de décisions précédents.

```
In [17]: # On utilise les paramètres optimaux
model = SVC(kernel='linear', C=1, gamma=0.01, probability=True)
model.fit(x_train, y_train)
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)
plot_data(x, y)
plt.show()

model = SVC(kernel='rbf', C=0.1, gamma=0.1, probability=True)
model.fit(x_train, y_train)
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)
plot_data(x, y)
plt.show()

model = SVC(kernel='poly', C=1, gamma=1, probability=True)
model.fit(x_train, y_train)
model.predict_proba(x_test)
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)
plot_data(x, y)
plt.show()
```





- frontières de décisions : Plus la frontière de décision est floue, plus les données sont difficilement séparable, les probabilités d'appartenance aux classes sont faibles. Une frontière nette indique une séparation puissante des données avec de fortes probabilités d'appartenance des points à la classe. Plus la marge est grande (C grand), plus le classifieur est tolérant aux erreurs et plus la frontière sera floue. À l'inverse, si C est faible, le classifieur sera plus strict et la frontière sera plus nette. Si gamma est faible, alors le classifieur est moins sensible aux points individuellement, la frontière sera plus large (et inversement pour gamma élevé) (C'est très visible sur la dernière image).
- Vecteurs supports : Les points de vecteurs supports sont proches de la frontière de décision. Plus il y a de vecteurs supports, plus le modèle est sensible à l'overfitting.

V - Kernels

```
In [ ]: def string_kernel(s, t, n, lambda_value=0.5):
        """
        Compute a string kernel based on common subsequences of length n.

        Params:
            s (str): first input string
            t (str): second input string
            n (int): length of the subsequences to compare
            lambda_value (float): weight factor for subsequences (default is 0.5)

        Returns:
```



```

        float: kernel value based on matching subsequences
    """

    similar_subsequences = []
    for i in range(len(s) - n + 1):
        for j in range(len(t) - n + 1):
            if s[i:i+n] == t[j:j+n]:
                similar_subsequences.append((i, j))

    kernel_value = 0
    for i, j in similar_subsequences:
        length_s = i + n
        length_t = j + n
        kernel_value += lambda_value ** (length_s + length_t)

    return kernel_value

```

```

In [ ]: def similarity_matrix(texts, n, lambda_value=0.5):
    """
    Compute a similarity matrix based on string kernels for a list of texts.

    Params:
        texts (list of str): list of text strings to compare
        n (int): length of subsequences for the string kernel
        lambda_value (float): weight factor for subsequences (default is 0.5)

    Returns:
        np.ndarray: matrix of pairwise similarity values between texts
    """

    num_texts = len(texts)
    similarity_matrix = np.zeros((num_texts, num_texts))

    for i in range(num_texts):
        for j in range(num_texts):
            similarity_matrix[i, j] = string_kernel(texts[i], texts[j], n, lambda_value)

    return similarity_matrix

```

```

In [ ]: def plot_similarity_matrix(similarity_matrix, texts):
    """
    Plot the similarity matrix as a heatmap.

    Params:
        similarity_matrix (np.ndarray): matrix of similarity values between
        texts (list of str): list of text strings (used for labels on axes)

    Side Effects:
        Displays a heatmap of the similarity matrix
    """

    plt.figure(figsize=(10, 8))
    sns.heatmap(similarity_matrix, xticklabels=texts, yticklabels=texts, cmap='magma')
    plt.title("similarity matrix between texts")
    plt.show()

```

```
In [19]: # Données
texts = [
    "Le soleil brillait sur la mer calme ce matin",
    "Une étrange créature surgit de la forêt profonde",
    "Le jeune garçon lisait un livre sous l'arbre",
    "Des éclairs illuminaient le ciel noir pendant l'orage",
    "Le chat dormait paisiblement sur le canapé",
    "Un dragon majestueux volait au-dessus des montagnes",
    "Elle écrivait dans son journal tous les soirs",
    "Une porte secrète menait à un monde oublié",
    "Les enfants jouaient dans le parc après l'école",
    "Un vieux grimoire contenait des sorts puissants",
    "Le vent soufflait fort dans les rues désertes",
    "La sorcière préparait une potion dans son chaudron",
    "Ils regardaient les étoiles dans la nuit silencieuse",
    "Une armée de squelettes sortit du sol poussiéreux",
    "La pluie tombait doucement sur les vitres embuées"
]

labels = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]

indices = np.arange(len(texts))
train_idx, test_idx, y_train, y_test = train_test_split(indices, labels, tes

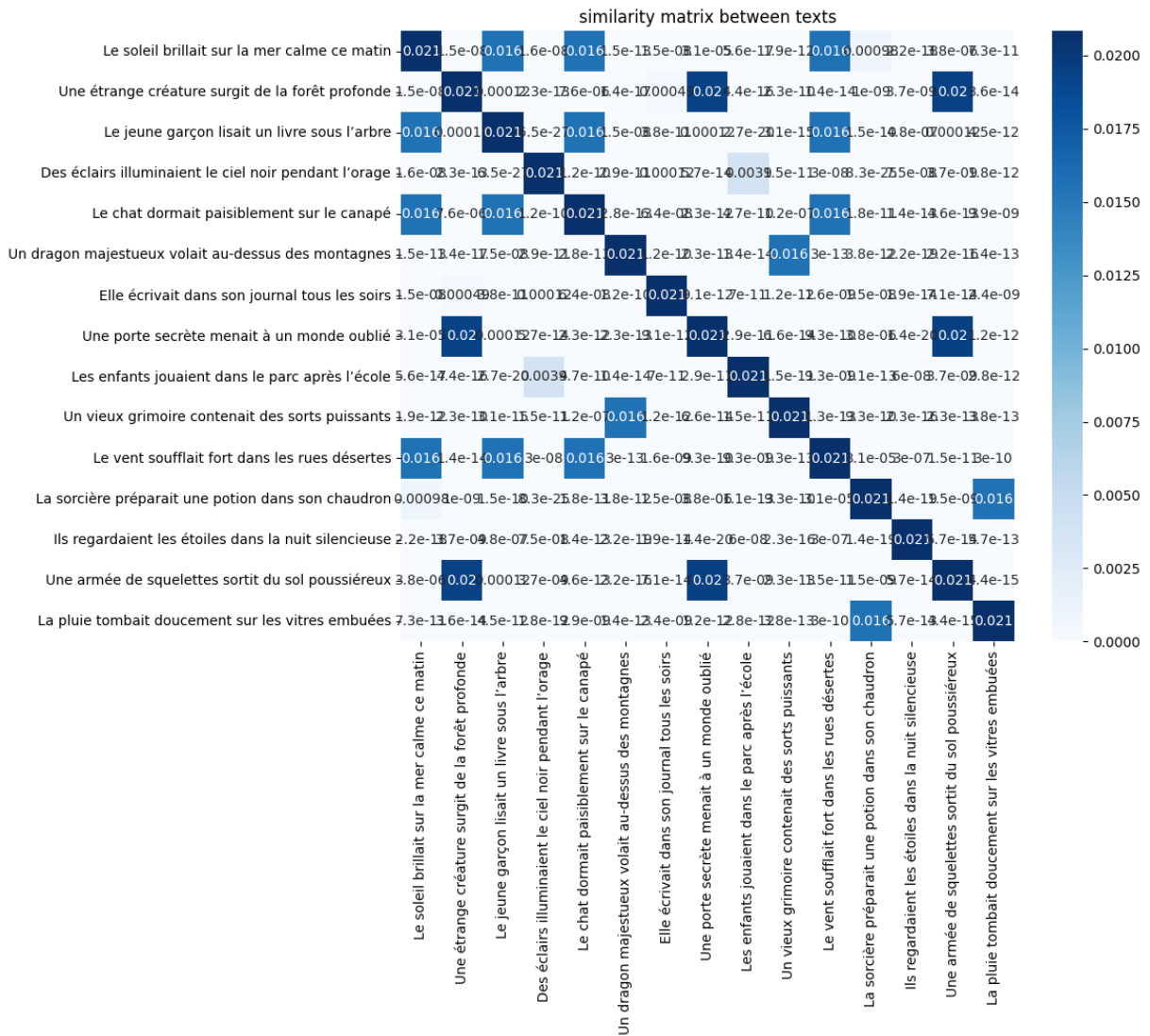
similarity_matrix_result = similarity_matrix(texts, n=3, lambda_value=0.5)
plot_similarity_matrix(similarity_matrix_result, texts)

x_train = similarity_matrix_result[np.ix_(train_idx, train_idx)]
x_test = similarity_matrix_result[np.ix_(test_idx, train_idx)]

# apprentissage
svm_model = SVC(kernel='precomputed')
svm_model.fit(x_train, y_train)

train_score = svm_model.score(x_train, y_train)
test_score = svm_model.score(x_test, y_test)

print(f"Training score: {train_score:.2f}")
print(f"Test score: {test_score:.2f}")
```



Training score: 0.58

Test score: 0.33